Springer Undergraduate Texts
in Mathematics and Technology

SUMAT



Leo Liberti
Carlile Lavor

Euclidean Distance Geometry

An Introduction

Springer

Springer Undergraduate Texts in Mathematics and Technology

Series editors

Jonathan M. Borwein, University of Newcastle, Newcastle, Australia
H. Holden, Norwegian University of Science and Technology, Trondheim, Norway
V.H. Moll, Tulane University, New Orleans, LA, USA

Editorial Board

Lisa Goldberg, University of California, Berkeley, CA, USA
Armin Iske, University of Hamburg, Hamburg, Germany
Palle E.T. Jorgensen, The University of Iowa, Iowa City, IA, USA
Stephen M. Robinson, University of Wisconsin at Madison, Madison, WI, USA

Springer Undergraduate Texts in Mathematics and Technology (SUMAT) publishes textbooks aimed primarily at the undergraduate. Each text is designed principally for students who are considering careers either in the mathematical sciences or in technology-based areas such as engineering, finance, information technology and computer science, bioscience and medicine, optimization or industry. Texts aim to be accessible introductions to a wide range of core mathematical disciplines and their practical, real-world applications; and are fashioned both for course use and for independent study.

More information about this series at http://www.springer.com/series/7438

Leo Liberti · Carlile Lavor

Euclidean Distance Geometry

An Introduction

Springer

Leo Liberti
CNRS LIX
École Polytechnique
Palaiseau
France

Carlile Lavor
Department of Applied Mathematics
(IMECC-UNICAMP)
University of Campinas
Campinas
Brazil

Part of the work on this book was carried out at the IBM "T.J. Watson" Research Center, Yorktown Heights NY, USA.
Part of the work on this book was carried out at the Dept. of Comp. Sci., Duke University, Durham NC, USA.

ISSN 1867-5506 ISSN 1867-5514 (electronic)
Springer Undergraduate Texts in Mathematics and Technology
ISBN 978-3-319-60791-7 ISBN 978-3-319-60792-4 (eBook)
DOI 10.1007/978-3-319-60792-4

Library of Congress Control Number: 2017943252

Mathematics Subject Classification (2010): 51K99, 51K05

© Springer International Publishing Switzerland 2017
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Printed on acid-free paper

This Springer imprint is published by Springer Nature
The registered company is Springer International Publishing AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

This book is dedicated to Prof. Nelson Maculan, who showed us the ropes of Distance Geometry and brought us together.

Contents

1 Motivation . . . 1
1.1 How it all started . . . 1
1.2 Setting up *Mathematica* . . . 3
1.3 Four examples . . . 3
1.3.1 Clock synchronization . . . 3
1.3.2 Sensor network localization . . . 4
1.3.3 Structural biology . . . 4
1.3.4 Big data . . . 5
1.3.5 What these problems have in common . . . 5
1.4 Solving the Clock Synchronization Problem . . . 6
1.5 Exercises . . . 7

2 The Distance Geometry Problem . . . 9
2.1 Computing all pairwise distances from points . . . 9
2.2 Computing points from all pairwise distances . . . 9
2.2.1 Ill-posedness . . . 9
2.2.2 No solution . . . 10
2.3 The fundamental problem of DG . . . 11
2.3.1 The input as a weighted graph . . . 11
2.3.2 Formalization of the DGP . . . 11
2.4 A quadratic system of equations . . . 12
2.4.1 The number of solutions . . . 13
2.4.2 Computational complexity of the DGP . . . 14
2.5 Direct solution methods . . . 16
2.5.1 A global optimization formulation . . . 16
2.6 Exercises . . . 18

3 Realizing complete graphs . . . 19
3.1 Cliques . . . 19
3.2 Realizing $(K+1)$-cliques in $\mathbb{R}^{K-1}$ . . . 19
3.2.1 The trilateration system in $\mathbb{R}^{K-1}$ . . . 20
3.2.2 Solving the linear system . . . 21
3.2.3 Iterative realization of complete graphs . . . 21
3.3 Realizing $(K+1)$-cliques in $\mathbb{R}^K$ . . . 23
3.3.1 Basic and nonbasic columns . . . 23
3.3.2 Expressing basics as linear functions of nonbasics . . . 23

3.3.3 The K-lateration system in $\mathbb{R}^K$ . . . 24
3.3.4 Differences between $\mathbb{R}^K$ and $\mathbb{R}^{K-1}$ . . . 26
3.3.5 The realization algorithm . . . 27
3.3.6 The assumption on the rank of A . . . 27
3.4 Exercises . . . 28

4 Discretizability . . . 31
4.1 The volume of simplices . . . 31
4.1.1 Length and area: K-volume for $K \leq 2$ . . . 31
4.1.2 The Cayley-Menger determinant . . . 32
4.2 Realizing quasi-cliques . . . 32
4.2.1 Flat simplices and zero volume . . . 32
4.3 Realizing K-laterative graphs in $\mathbb{R}^K$ . . . 33
4.3.1 Trilateration orders . . . 34
4.3.2 Trilaterative DGP . . . 35
4.3.3 The number of solutions of the TDGP . . . 35
4.3.4 Sensor network localization . . . 36
4.4 Realizing $(K-1)$-laterative graphs in $\mathbb{R}^K$ . . . 36
4.4.1 The shape of protein backbones . . . 37
4.4.2 Discretizable DGP . . . 37
4.4.3 A Branch-and-Prune algorithm . . . 38
4.4.4 Some examples . . . 39
4.4.5 Finding all realizations . . . 41
4.4.6 Worst-case complexity . . . 41
4.4.7 Best-case complexity . . . 41
4.5 Exercises . . . 41

5 Molecular distance geometry problems . . . 43
5.1 Contiguous $(K-1)$-lateration orders . . . 43
5.1.1 The generalized DMDGP . . . 43
5.1.2 Realizing ^{K}DMDGP graphs . . . 44
5.1.3 Feasibility of Next . . . 44
5.2 Partial reflection symmetry . . . 45
5.2.1 Isometry and congruence . . . 46
5.2.2 The discretization group . . . 47
5.2.3 The pruning group . . . 49
5.2.4 A symmetry-aware BP . . . 51
5.2.5 Number of realizations of ^{K}DMDGP graphs . . . 52
5.3 Fixed-parameter tractability . . . 52
5.3.1 BP tree width . . . 52
5.3.2 The BP seems polynomial on proteins . . . 54
5.4 Exercises . . . 54

6 Vertex orders . . . 57
6.1 Existence of trilateration orders . . . 57
6.1.1 Problem hardness . . . 57
6.1.2 A Fixed-Parameter Tractable algorithm . . . 60
6.2 Existence of contiguous trilateration orders . . . 61
6.2.1 Problem hardness . . . 62
6.2.2 A mathematical programming formulation . . . 63
6.3 Exercises . . . 64

7 Flexibility and rigidity . . . 67
7.1 Some preliminary notions . . . 67
7.2 Rigidity of frameworks . . . 68
7.3 The rigidity matrix . . . 69
7.3.1 The rank of the rigidity matrix . . . 69
7.3.2 Regular and singular realizations . . . 70
7.3.3 The nullity of the rigidity matrix: infinitesimal rigidity . . . 71
7.3.4 Asimow and Roth's theorems . . . 73
7.3.5 Generic rigidity . . . 74
7.4 Graph rigidity on the line and in the plane . . . 75
7.4.1 Graph rigidity on a line . . . 76
7.4.2 General position . . . 76
7.4.3 Abstract rigidity . . . 76
7.4.4 Laman's theorem . . . 78
7.5 Exercises . . . 79

8 Approximate realizations . . . 81
8.1 The weighted adjacency matrix . . . 81
8.2 Matrix completion . . . 81
8.3 Overall method structure . . . 82
8.4 Approximate Completion Methods . . . 82
8.4.1 Constant completion . . . 82
8.4.2 Shortest paths . . . 83
8.5 Approximate realization methods . . . 83
8.5.1 Classic Multidimensional Scaling . . . 83
8.5.2 Proximity adjustment . . . 87
8.6 Approximate projection methods . . . 87
8.6.1 Principal Components Analysis . . . 87
8.6.2 Gaussian random projections . . . 88
8.6.3 The Johnson–Lindenstrauss lemma . . . 88
8.7 Isomap . . . 90
8.8 Stochastic Proximity Embedding . . . 90
8.9 Exercises . . . 91

9 Taking DG further . . . 93
9.1 Modeling signal processing problems . . . 94
9.2 Theory of solution uniqueness . . . 95
9.3 Combinatorial methods . . . 95
9.4 Optimization-based solution methods . . . 95
9.5 Debitum Gratitudinis (DG) . . . 96

Appendix: Mathematical notions . . . 97

References . . . 117

Index . . . 121

Introduction

What do clocks, wireless devices, atoms, and submarines have in common? They move: the clocks move time forward (so to speak), wireless devices usually move on a plane (like an office floor), or at least on a two-dimensional surface (like a mountainous region), atoms move in three-dimensional space, and so do submarines. We are interested in these seemingly disparate entities when they move together: each computer on a network has a clock, wireless devices move as part of a wireless network, atoms form molecules, and the sort of submarines we look at are unmanned, and they move in fleets. A second very distinctive feature of these sets of moving entities is that it is useful to know the position of each entity with respect to the others: we want to know the absolute time of each clock in order to synchronize them, we need to trace each wireless device in the network for routing purposes, we want to find the geometrical shape of each molecule as it largely determines its function with respect to its environment (e.g., a cell), and we would like to control a fleet of unmanned submarines in order to accomplish a given mission. The third decisive common feature is that for each of these entities we can obtain estimates of some of their pairwise distances. The fundamental question underlying this book is the following: *given a subset of pairwise distances and the dimension of the surrounding space* (the one-dimensional time line for clocks, the two-dimensional plane for wireless devices, and the three-dimensional space for atoms and submarines), *can we find positions for all entities yielding the given distances in the space of the given dimension?*

Throughout this book, we look at the static version of this problem: namely, we suppose we can access a "snapshot" of some of the distances, including values and incidence to entities, and we want to compute corresponding spatial positions for all the entities. This is an accurate representation for the clock synchronization problem, where distances correspond to time discrepancies between pairs of clocks, as well as for the protein conformation problem, as long as we pretend that atoms do not vibrate too much that the protein is at rest and that we can trust the nuclear magnetic resonance experiments to provide some of the interatomic distances. Proteins move in space, but the relative positions of its atoms are often unchanged, i.e., they often display a *rigid motion*. Wireless sensors networks and fleets of unmanned submarines move in space, but rarely do they keep all of their pairwise distances fixed. One way to deal with the issue is to make sure that the algorithms that compute the positions from the given distances work very efficiently (so they can run in a time-step).

In abstract terms, we are given a weighted graph and a number K of dimensions, and we want to find positions for the vertices in $\mathbb{R}^K$ such that the edges, drawn as segments, have lengths equal to the corresponding weights. This is the fundamental problem of a field called *Euclidean Distance Geometry*. The name "Distance Geometry" refers to a concept of geometry based on distances rather than points. As for the word "Euclidean," it has something to do with our choice of drawing edges as segments, without constraining their incidence angles. If we had decided to draw edges only using

vertical and horizontal segments incident at right angles, the word "Euclidean" might have been replaced by "taxicab" or "Manhattan"—but this setting has fewer applications.

An issue which appears to be quite well studied in Euclidean Distance Geometry is the uniqueness of the solution. This feature is obviously very desirable in the wireless network and submarine fleet applications, since our objective is to recover the actual position in space of devices and submarines, which cannot help but occupy a single position in space. With proteins, however, which are really our "pet application," this issue is somewhat less important. Molecules naturally come in different isomers, which means that the same chemical composition can give rise to multiple shapes. Many interesting isomers involve a difference in *chirality*, i.e., two molecules can be geometric reflections of each other. With proteins, which consist of a backbone with some side chains appended, the reflections can also be *partial*: Two isomeric backbones have the same shape until atom $v - 1$, and then the part of the backbone from atom v onwards is a reflection with respect to the plane defined by three atoms before v. These simple geometric operations can make the difference between a safe medicine and a deadly poison. Therefore, it makes sense to work out the shape of all of the proteins that are consistent with the observed interatomic distances. Rather than solution uniqueness, we are interested in finding all solutions.

This book is aimed at intermediate undergraduate students, starting graduate students, any researcher who would like to know something about the theory and algorithms used in Euclidean Distance Geometry, and any practitioner who needs to actually compute positions of entities for which he or she knows some of the pairwise distances. Its objective is to teach the basics of this field, without going in too many details and yet providing readers with some useful methodologies as well as with a sense of *why* they work.

Note that *this is a textbook, rather than a research monograph*. The authors have taught Ph.D.-level courses about this material, but also drew on it to teach B.Sc. and M.Sc. courses in other fields, such as Mathematical Programming. We strove to keep the book short; proofs, when present, are given informally within the text and are often complemented by pictures. All algorithms in this book (and many more) have been implemented in the *Mathematica* computer programming language and are available online, at[1]

http://www.lix.polytechnique.fr/~liberti/intro_dg

Each chapter ends with some exercises. We added a schematic appendix containing all necessary preliminary mathematical notions.

Incredibly, this book is the first of its kind, i.e., it is the first (teaching oriented) textbook on Euclidean Distance Geometry. The first book on the subject of Distance Geometry was written by Blumenthal in 1953 [17], with entirely different objectives: it is part research monograph and part compendium of a budding field. Distance Geometry in 1953 focused—due to the almost total lack of computers at the time—on very different issues than it does today. We also note that Blumenthal wrote a short didactical textbook on Euclidean geometry in 1961 [18], to which he added a chapter on Distance Geometry that borders on the same topics of the 1953 book. A mathematically advanced research monograph on issues similar to those treated here was written by Crippen and Havel in 1988 [31], a book which is now out of print and very hard to find. Like this book, [31] is also motivated by finding the shape of proteins. The most recent research-oriented book on this subject was written by Dattorro around 2004 [33], who keeps the book continually updated: [33] is mostly an online book, although the author will print, bind, and send you a copy (for a fee to cover costs) if you ask for one. Dattorro's book is also about convex optimization and therefore takes an approach that is complementary to ours. The most immediately remarkable difference between [33] and this book is in the

[1] If you are reading this chapter as a PDF on your screen, copying and pasting the URL is likely to result in some wrong characters (particularly the tilde and the underscore).

writing style. Dattorro's book is an almost encyclopedic research monograph: commended for the specialists in the field who need all the details at their fingertips, but perhaps not ideal for the budding Euclidean Distance geometer. Some books about graph rigidity [57, 118] and oriented matroids [16] also cover some of the subjects we discuss. We think that our own surveys [80, 84, 86] nicely complement the material in this book. We also cowrote a very short didactical monograph (directed mostly at starting undergraduates and even finishing high-school students) which covers some of the topics of this book, mostly restricted to the 2D plane and 3D space [77].

The reason why we write "incredibly," in noting this as the first teaching book on Euclidean Distance Geometry, is that the subject itself is as old as the ancient Greeks and involved mathematicians such as Heron, Euler, Cauchy, Cayley, and Gödel [81], to name only the most famous. This field also produced methods such as classic Multidimensional Scaling and the Johnson–Lindenstrauss lemma, both widely used today in the context of the "big data" revolution. It contributed to win the 2002 Nobel Prize in Chemistry to Wütrich [121] "for his development of nuclear magnetic resonance spectroscopy for determining the three-dimensional structure of biological macromolecules in solution." From our point of view, there should be *hundreds* of teaching books on Distance Geometry around!

Our own interest in this field started in 2004, when one of us, then working at Politecnico di Milano, paid a visit to the other, then at the Universidade Estadual do Rio de Janeiro. That visit yielded a modest book chapter [78], after which we scratched our heads and asked ourselves, "what now"? Twelve years and many dozens of research papers later, we realize we only just started to brush the surface. Humbled by the gargantuan size of the task ahead, we decided to spend some of our time teaching other people what we know, in the hope of advancing this wonderful field. This book is the result of our teaching efforts.

New York, Paris — Leo Liberti
Campinas, Durham — Carlile Lavor
2013–2017

Chapter 1
Motivation

This book is a basic introduction to Distance Geometry (DG): it gives an overview of the mathematical theory of DG. Our point of view derives from our motivation to apply DG methods to the problem of finding the structure of proteins given some of the interatomic distances. The presentation, however, is largely application independent and applies to many settings, such as localization of wireless sensors in a network, control of unmanned underwater vehicles, visualization of high-dimensional data, stability of bar-and-joint structures, and more. The prerequisites are elementary linear algebra and Euclidean geometry; most prerequisites are summarized in Appendix A.

1.1 How it all started

In Euclid's original view, geometry was based on points and lines [47]. This view was brought forward by Descartes, who gave a more quantitative interpretation of points with the Cartesian plane and its axes [36]. Calculus, mathematical analysis, linear algebra all make use of a concept of geometry which is based on points (sometimes called vectors). And yet, when Greek farmers squabbled over the extent of their triangular fields, they had an easier time calculating the side lengths rather than the coordinates of the vertices. The Alexandria colons circa 100AD seemed to be the most belligerent and brought their fights before the wise Heron, who, tired of the hellish waste of time, devised a formula for computing the area of a triangle using nothing but the side lengths [59] (forget the "base," "height," and "divided by two", which always confused all of the farmers). We might as well ascribe to Heron the official birth of DG, although some would insinuate that similar methods for estimating the area of triangles were present in Egypt well before 100AD.

A few centuries later, Arthur Cayley took Heron's formula for triangles in the plane and generalized it, through a determinant, to simplex volumes in any dimension [25]. Karl Menger, who worked at the beginning of the twentieth century and was fascinated by the (then) fashionable axiomatization of mathematics, picked up on the Cayley determinant and used it to try his own axiomatization of geometry through distances [93][1]: so that, now, the Cayley determinant is actually called[2] "Caley–Menger determinant." Menger, who is best known for organizing a popular seminar in Vienna [95] in contrast to the Vienna Circle (which had become politicized and downright dangerous), apparently only made a single disciple with his work on DG: Leonard Blumenthal, who devoted his working life to clarifying [17] the work of his advisor, which remained obscure both in the German original and in

[1] We suspect that Menger chose an axiomatization of geometry by distances because all of the other axiomatizations had been already taken by people like Hilbert, Bernays, Tarski, or Carnap…☺.

[2] This was probably one of Menger's smartest career moves.

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_1

(his own) English translation [94]. So obscure, in fact, that attempts at explaining it are still ongoing [20, 81].

While giving M.Sc.-level courses at the University of Vienna, Menger welcomed to his class a student who then became one of the most celebrated mathematicians of all time, Kurt Gödel. Gödel proved two incredibly deep theorems as a Ph.D. student: his *completeness theorem*, which states that any logically valid first-order formula has a formal proof, and his *incompleteness theorem*, which perversely states that there are true first-order formulæ that cannot have a proof.[3] Even more incredibly, Gödel never had a Ph.D. student, nor a coauthor, except for a single paper[4] on DG, where he discusses how to adapt to the sphere the statement that each quadruplet of points on a plane has zero volume. Gödel wrote a few other abstracts in DG, the most interesting of which contains a devious fixed point argument to show that if four points can be realized in $\mathbb{R}^3$ (but not in $\mathbb{R}^2$), then they can also be realized on the surface of a sphere with geodesic curved sides having the same lengths [53].

This admittedly sketchy historical account did nothing so far to justify a contemporary interest in DG. So why should we write a book about it? Two breakthroughs, both related to "Big Data," will give us a better motivation. Isaac Schoenberg, the inventor of splines, unearthed in 1935 the relationship between Euclidean distance matrices (EDMs) and positive semidefinite (PSD) matrices [109]. This gave rise to the incredibly successful *multidimensional scaling* technique for visualizing high-dimensional data (see Sect. 8.5.1). Schoenberg's paper bears the title *Remarks to Maurice Fréchet's Article "Sur La Definition Axiomatique D'Une Classe D'Espace Distanciés Vectoriellement Applicable Sur L'Espace De Hilbert"*. What is really remarkable about Schoenberg's remarks is that no one even remembers Fréchet's original paper, but everyone uses multidimensional scaling.

The second breakthrough follows a similar pattern: Johnson and Lindenstrauss' 1984 paper *Extensions of Lipschitz mappings into a Hilbert space* [62] focuses on a rather complicated theorem concerning infinite dimensional spaces. To prove the theorem, the authors spend a couple of pages on a surprising lemma, now called the *Johnson-Lindenstrauss Lemma* (see Sect. 8.6.3). This lemma states that, given a set X of n vectors in $\mathbb{R}^m$, you can premultiply the vectors by a $k \times m$ matrix T, where each component is sampled from a normal distribution with zero mean and $\frac{1}{k}$ variance, and, provided k is $O(\epsilon^{-2} \ln n)$ with some given $\epsilon \in (0, 1)$, you get:

$$\forall x, y \in X \quad (1-\epsilon)\|x-y\|_2 \leq \|Tx-Ty\|_2 \leq (1+\epsilon)\|x-y\|_2.$$

Why is this surprising? Well, suppose you want to cluster 100,000 images using an algorithm that uses nothing but Euclidean distances. The thumbnail 100×100 RGB images are actually vectors in $\mathbb{R}^{30,000}$. Now, if you set an error tolerance at 10%, i.e., $\epsilon = 0.1$, you could pick k to be around $100 \times \ln(100{,}000) \approx 1152$. So, instead of working with vectors having thirty thousand components, you could work with vectors having just over a thousand components. And, if your clustering algorithm is only a heuristic, who knows whether the 10% error is even hurting your results? In other words, this is an eminently sellable technique, and we think that Google, Yahoo!, and Facebook are likely to use it a lot.[5] Johnson's and Lindenstrauss' result followed the same pattern as Schoenberg's: no one remembers the actual theorem, but everyone knows the lemma.

Architecture, statics, and the worry of engineers that their next bridge might collapse and they might be sent to prison or worse pushed DG toward another direction: which constructions are resistant to external forces, and which will crumble (or crumple, depending on materials) under their action? For example, which bar-and-joint structures are rigid or flexible? Rigid structures will only have finitely many incongruent realizations in space, whereas flexible structures will flex, and hence have

[3] Explaining away the apparent contradiction in terms between the completeness and incompleteness theorems is even more perversely left as an "easy exercise" for the reader—allowed solution time: approximately 2 years. Hint: consult [50].

[4] Calling this work a "paper" is overkill—it is more like a one page abstract in Menger's seminar proceedings [95].

[5] This belief is based on no other assumption that, if we were in their place, we would use this technique a lot.

uncountably many. Maxwell defined "force diagrams" based on rigidity notions [92], and a graphical algorithm to solve them [91]. A famous 1766 conjecture of Euler's [48] stated that all three-dimensional polyhedra must be rigid. As shown in a wonderful proof by Cauchy [24], Euler was right insofar as one uses the definition of polyhedron as an intersection of half-spaces. But if one is willing to consider face incidence lattice-based definitions, then a "polyhedron" might also be a nonconvex set. Bob Connelly, the proud inventor of a flexible (triangulated) sphere, has the peculiar distinction of having proved Euler wrong [27].

Finally, computers are the real reason why DG is practically important today: as ever, much of the number-crunching that no one could have ever carried out by hand suddenly became possible. Thus, motivated to have our computers help us learn DG, we step into the next section without further ado.

1.2 Setting up *Mathematica*

We use *Mathematica* (v.9.0.1) as a programming tool to perform mathematical experiments and implement algorithms. Most of the commands we use are standard in *Mathematica*; those which are not are contained in the file `intro_dg.m` (which can be downloaded from http://www.lix.polytechnique.fr/~liberti/intro_dg). To load up `intro_dg.m` in your *Mathematica* kernel, supposing it is in folder `MyFolder`, type[6]:

```
Get["MyFolder/intro_dg.m"];
```

in your open *Mathematica* notebook.

1.3 Four examples

We dive right into the heart of the matter by presenting four examples of problems which can be solved using DG tools. The common traits of these examples are discussed below, in Sect. 1.3.5, to give a chance to the reader to work them out independently.

1.3.1 Clock synchronization

Alice just told you her watch is wrong by 5 minutes, Bob's is wrong by 7 with respect to Alice's, and Charles' by 3 with respect to Alice's and 4 with respect to Bob's. Exasperated, you check the atomic clock public Web site to find that it is precisely 16:27. Can you find out the time on Alice's, Bob's, and Charles' watches? Is there only one solution, or can there be many?

[6] Use the forward slash "/" as a directory separator for both Unix and Windows.

1.3.2 Sensor network localization

WIFI-enabled smartphones can create what is known as an "ad-hoc network," i.e., they can create a WIFI network where each communication is peer-to-peer, as long as the distance between two smartphones is not excessive. Also, smartphones can estimate pairwise distances (with close enough peers) by measuring how much battery they use to send/receive a data packet: the higher the battery consumption, the larger the distance. The network administrator at headquarters must locate each person in the building at any time: he or she defines a protocol that has every corporate smartphone send out the distance to each of its neighbors to a central server. At a certain time, the server measures the pairwise distances below:

Pair	$\{A, B\}$	$\{A, C\}$	$\{A, D\}$	$\{B, D\}$	$\{B, S\}$	$\{D, S\}$
Distance	3	7	5	4	3	3

where A, B, C, D are smartphones, and S is the server. Setting the server's position as the origin $(0, 0)$, can you find the positions of all the smartphones, supposing they are all on the same floor? Hint: see Fig. 1.1.

1.3.3 Structural biology

Designing drugs to cure a specific disease is all about knowing what a certain protein can do to an organism. Because proteins bind to certain sites in cells, their function depends on their shape. Proteins are sets of atoms, some pairs of which share a chemical bond. Chemical experiments tell us what the set of atoms is, and what the covalent bonds connecting these atoms are. Then, if we number the atoms as $1, 2, 3, \ldots, n$, we can infer: (i) the set of bonds $\{u, v\}$, where u, v are atoms in $\{1, \ldots, n\}$; (ii) the distance between u and v (for each bond pair); (iii) the angle defined by two bonds $\{u, v\}$ and $\{v, w\}$ incident to an atom v. If you imagine atoms as joints and bonds as bars, you can visualize this setting as a mechanical structure in space: this structure bends and folds until it reaches a state of lowest energy, with a corresponding geometrical shape, which is what we would like to find out. When the protein is at rest, we can perform more complicated experiments, such as nuclear magnetic resonance (NMR), to determine all interatomic distances up to a certain threshold value (around 5Å, i.e., 5×10^{-10} meters). Armed with all this information, how can we find the shape of the protein? *Hint*: if you want to perform some experiments with *Mathematica*, see Fig. 1.2.

```
G = Graph[{A <-> B, A <-> C, A <-> D, B <-> D, B <-> S, D <-> S},
  EdgeWeight -> {3, 7, 5, 4, 3, 3}, VertexLabels -> "Name"]
```

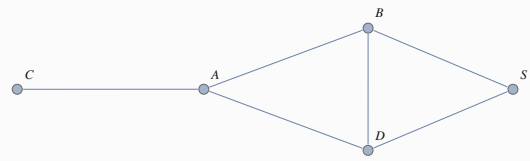


Fig. 1.1 *Mathematica* code to represent the relations of the sensor network.

```
Grid[Partition[Flatten[Table[GraphPlot3D[SpiralGraph[s, r+s]],
    {s ,0.4,0.1,−0.1}, {r ,0.6,1.8,0.4}]],4]]
```

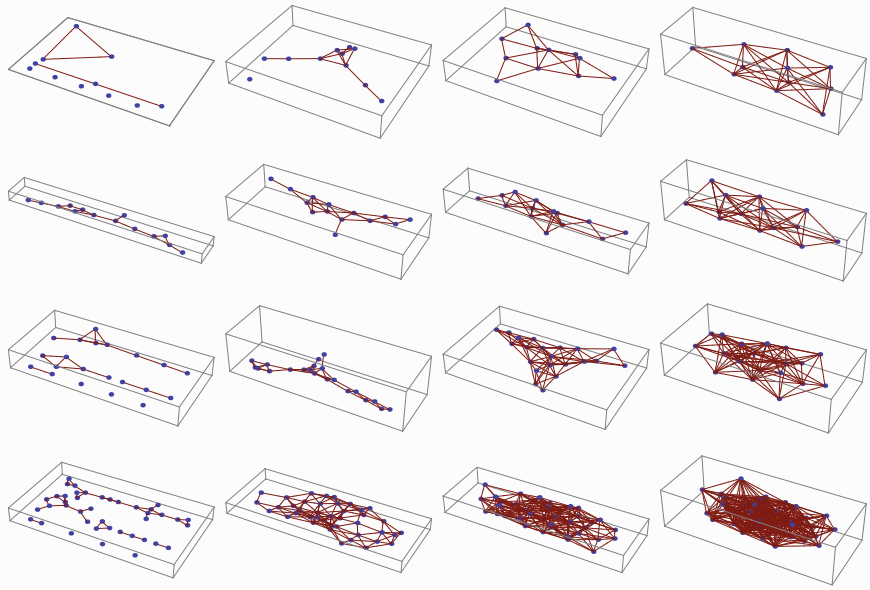


Fig. 1.2 `SpiralGraph[s,r]` generates points along a spiral-like curve in 3D, spaced by s, and relates two points if they are closer together than r (this has some similarities with a protein backbone graph). The closer s is to zero, the more points are generated; the greater r, the more relations are generated. Good values of s, r ensure that each point is related to at least six other points. `SpiralGraph[]` uses default values $s = 0.02$ and $r = 0.7$.

1.3.4 Big data

The JCN corporate database has a historical sales table of their mainframe computers dating all the way back to the system's creation, in the 70s. Each row of the table represents a sale and has a set of 350 columns, which are far from being independent. Your boss told you to prepare an accurate visual representation of this table, so he can show the upper management. Of course you cannot show a picture of a 350-dimensional space. What can you do? *Hint*: see Fig. 1.3.

1.3.5 What these problems have in common

The first problem is an instance of the clock synchronization problem (CSYP) [110], the second is of the sensor network localization problem (SNLP) [66], the third is a description of the molecular distance geometry problem (MDGP). These problems [84] all belong to the field of DG. The input of each one is a set of distances assigned to a set object pairs, and the required output is the geometric position of each object on a one-dimensional (1D), two-dimensional (2D), or three-dimensional (3D) space, respectively.

```
(* SalesTable is an $n \times 350$ array *)
Proj3D = (1/Sqrt[3]) RandomVariate[NormalDistribution[0, 1], {350, 3}];
SalesTable3D = SalesTable . Proj3D;
ListPlot3D[SalesTable3D, Mesh->None, InterpolationOrder->0,
 ColorFunction->"SouthwestColors"]
```

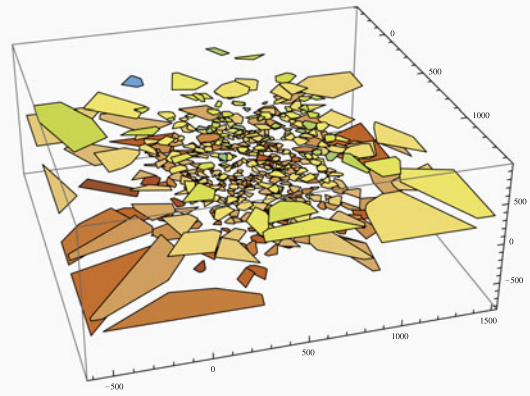


Fig. 1.3 Standard Gaussian random projection can be a good way to decrease the dimensionality of a set of points while approximately keeping its shape similar (see Sect. 8.6.3 for more information). Note that `SalesTable` was also randomly generated in the above experiment, so this picture, though visually appealing, is not particularly significant!

The last problem is different, insofar as the input is the geometric position of each object in a 350D space, and the output is a set of accurate geometrical positions in 3D space. If we take "accurate" to mean "the distances between pairs of objects must be approximately preserved," then even the fourth problem can be seen as a DG problem in two or three dimensions.

All the problems involve the following concepts: pairs, distances, and geometric positions in one or more dimensions.

1.4 Solving the Clock Synchronization Problem

Let us see how to tackle the CSYP instance. We let Alice's watch be A, Bob's B, Charles' C, and the atomic clock be S. Let x_A, x_B, x_C, x_S the time shown by each clock. We then have:

$$|x_A - x_B| = 7 \tag{1.1}$$
$$|x_A - x_C| = 3 \tag{1.2}$$
$$|x_A - x_S| = 5 \tag{1.3}$$
$$|x_B - x_C| = 4 \tag{1.4}$$
$$x_S = 16{:}27. \tag{1.5}$$

In this context, the position is the time shown by each clock. We start with the pair involving x_S, whose position is known: the only such pair is $\{A, S\}$, with distance 5. Hence, x_A could be either 16:22 or 16:32. The other pairs incident to A are $\{A, B\}$ and $\{A, C\}$. If $x_A = 16{:}22$, then x_B is either 16:15 or 16:29, and x_C is either 16:19 or 16:25. If $x_A = 16{:}32$, then $x_B \in \{16{:}25, 16{:}39\}$ and

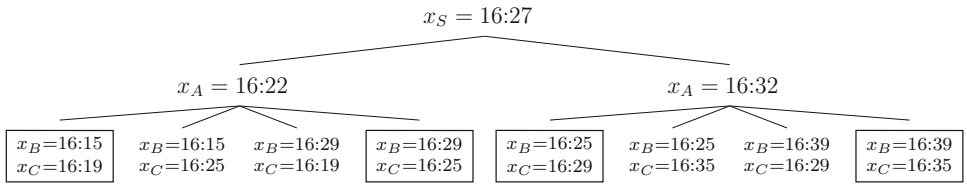


Fig. 1.4 Four solutions of the clock synchronization problem.

$x_C \in \{16{:}29, 16{:}35\}$. So far, the situation can be represented by the tree in Fig. 1.4. We now consider the last distance constraint $|x_B - x_C| = 4$. Four out of the eight possibilities for x_B, x_C are consistent with this information: $x_B = 16{:}15$ and $x_C = 16{:}19$, $x_B = 16{:}29$ and $x_C = 16{:}25$, $x_B = 16{:}25$ and $x_C = 16{:}29$, and $x_B = 16{:}39$ and $x_C = 16{:}35$. So the problem has four possible distinct solutions.

1.5 Exercises

1.5.1 Exercise

Can you solve the SNLP instance given above (the second problem) by adapting the procedure used for the CSYP?

1.5.2 Exercise

There is a firm, called *Metronom Incorporated*, which controls the accuracy of 3D positions of certain critical parts of various industrial tools from the lengths of some of their components. A specific tool consists of five connected bars having unit lengths, held together by four joints. How many bars is each joint adjacent to? Provide a set of mathematical equations describing the relative position of each joint. Solve the system numerically (e.g., with *Mathematica*). How many solutions does this system have? Does the answer change if you remove all solutions deriving from rotations, translations, and reflections?

1.5.3 Exercise

After the British Petroleum oil spillage in the gulf of Mexico, a fleet of underwater autonomous vehicles (UAV), or unmanned submarines, was deployed in an attempt to fix the problem with the broken pump. Since GPS does not work under water, all the submarines can use, in order to sense their surroundings, is the sonar "pings," which, by emitting sounds in a certain direction and measuring the time interval before the echo and its intensity, provide an estimation of the distance to the target. Write a system of equations describing the position of each submarine in the fleet as a function of time, assuming $d_{ij}(t)$ is the distance between submarine i and j at time t.

1.5.4 Exercise

Continuing Exercise 1.5.3, assume the fleet contains six submarines; for a given time instant t_0, assign various distance values to the constants $d_{ij}(t_0)$ and then solve the problem numerically (e.g., with *Mathematica*). How often does the resulting system turn out to have a valid solution? What is the maximum number of solutions of the system? Does this number change if you remove all solutions deriving from rotations, translations, and reflections?

1.5.5 Exercise

Geographically close electromagnetic waves superpose and generate a field which is an aggregation of the frequencies and amplitudes of the superposed waves. In order for cellular communication technology to work, each cellphone is assigned a frequency to communicate with the closest fixed radio

base (the *anchor*). A pair of cellphones are close whenever they are assigned to the same anchor. Close cellphones must be assigned different frequencies (represented by natural numbers, say $1, \ldots, M$) in order to minimize the loss of signal. In addition to this, each pair of close phones is assigned a certain absolute frequency difference in terms of the natural numbers representing the frequencies: for example, if cellphone A and B have a frequency difference equal to 3 in absolute value, then cellphone A could be assigned frequency 1 and cellphone B frequency 4; or 2 and 5, respectively, and so on. Write a mathematical system of equations to find the relative frequency number of cellphones communicating with a single anchor, and then write a mathematical system of equations for all cellphones in the network (which has many anchors). Supposing you were able to solve the problem for just one anchor, how would you solve the problem for the whole network?

1.5.6 Exercise

Continuing Exercise 1.5.5, assume there are three anchors, each having five cellphones, and $M = 10$. Find sets of absolute frequency differences to assign to the various pairs of close cellphones, so that each anchor system has a solution. What is the solution of the whole network? How many different solutions can you find for each anchor system? And for the whole network?

Chapter 2
The Distance Geometry Problem

The Distance Geometry Problem (DGP) is an *inverse problem*. The corresponding "direct problem" is to compute some pairwise distances of a given set of points. Whereas the direct problem is obviously trivial (just carry out the computation), the inverse problem is generally difficult to solve.

2.1 Computing all pairwise distances from points

Consider these four points in $\mathbb{R}^2$: $A = (4, 0)$, $B = (0, 3)$, $C = (-4, 0)$, $D = (0, -3)$, as shown in Fig. 2.1. Their pairwise distances are easy to compute (see Fig. 2.2):

$$\overline{AB} = 5, \overline{AC} = 8, \overline{AD} = 5, \overline{BC} = 5, \overline{BD} = 6, \overline{CD} = 5. \tag{2.1}$$

2.2 Computing points from all pairwise distances

Now, consider the inverse problem: you are given all the distances as in (2.1), with each value assigned to a pair of point names, you are told that they refer to a 2D vector space, and you are asked to compute four points A, B, C, D in $\mathbb{R}^2$ that give rise to those distances.

2.2.1 Ill-posedness

Notice that the inverse problem is ill-defined: it may have one or more solutions or even have no solution at all. For example, the points $A = (5, 1)$, $B = (1, 4)$, $C = (-3, 1)$, $D = (1, -2)$ also yield the same distances as $A = (4, 0)$, $B = (0, 3)$, $C = (-4, 0)$, $D = (0, -3)$ (see Fig. 2.3). And so do all the infinitely many points

$$A = (4 + \alpha, 0 + \beta), B = (0 + \alpha, 3 + \beta), C = (-4 + \alpha, 0 + \beta), D = (0 + \alpha, -3 + \beta)$$

for any scalars $\alpha, \beta \in \mathbb{R}$.

We can also rotate the point configuration in Fig. 2.1 by any given angle ϑ around any given center c, obtaining four different points always yielding the same distances as in (2.1).

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_2

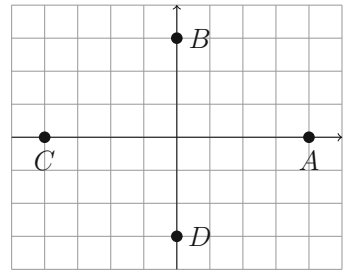


Fig. 2.1 Configuration of four points in $\mathbb{R}^2$.

```
x = {{4,0},{0,3},{−4,0},{0,−3}}; MatrixForm[EuclideanDistanceMatrix[x]]
```

$$\begin{pmatrix} 0 & 5 & 8 & 5 \\ 5 & 0 & 5 & 6 \\ 8 & 5 & 0 & 5 \\ 5 & 6 & 5 & 0 \end{pmatrix}$$

Fig. 2.2 Computing a distance matrix from an array of 2D points.

```
x = {{4,0},{0,3},{−4,0},{0,−3}}; Dx = EuclideanDistanceMatrix[x];
y = Map[(# + {1,1})&, x]; Dy = EuclideanDistanceMatrix[y];
PartialEDMError[Dx,Dy]
```

$$0$$

Fig. 2.3 Translations preserve pairwise distances. The `Map` function applies the function in the first argument (`(#+{1,1})&`) to each of the elements of the list (`x`) in the second argument. In general, if f is a function and $x = (x_1, \ldots, x_n)$ is a list, then `Map[(f[#])&, x]` is the list $(f(x_1), \ldots, f(x_n))$.

2.2.2 No solution

If we modify (2.1) so that $\overline{AC} = 11$, then the inverse problem has no solution at all: since $\overline{AB} = \overline{BC} = 5$ but $\overline{AC} = 11$, and $5 + 5$ is *not* greater than 11, these values contradict the triangle inequality.[1] This means that, no matter how hard we try, we shall never find three points A, B, C having those values as pairwise distances.

Ill-definedness notwithstanding, we shall see that it is possible to efficiently compute the correct point sets when all pairwise distances are given.

[1] See the metric axiom (Axiom 3) in Appendix A.6.

2.3 The fundamental problem of DG

In real problems, it very rarely happens that we are given all pairwise distances. The fundamental problem of DG is as follows: is there a set of points in a vector space of given dimension which yields a given subset pairwise distances, each with its assigned point name pair?

2.3.1 The input as a weighted graph

We formalize this problem using graphs to encode point names, their pairs, and the corresponding distances (see Fig. 2.4). Each point name is a vertex of the graph, and pairs of vertices are edges. Each edge is assigned to a distance value. This yields a weighted graph $G = (V, E, d)$ where V is the set of vertices of the graph, E is the set of edges, and d is the edge weight function. Graphs which are inputs to DG problems are generally *simple*, i.e., they do not have loops or parallel edges (see Appendix A.8.2). The first metric axiom (see Appendix A.6) states that the distance between a point and itself is zero; so it would not make much sense to have loops in the graph. Also, since any metric is a function, it must be well-defined; it must map any given argument to a unique value. So, if we had parallel edges between two vertices, we would have to assign the same distance value to all of them. But then we can simply replace all parallel edges with just one edge, without loss of generality. Because of these two observations, we can require the graph G to be simple.

We can also assume the input graph to be connected; in case it is not, then the graph really consists of a set of different subgraphs, each of which is connected. In order to solve the DGP on a disconnected graph, it suffices to solve it separately for each connected component. This reduces the DGP on disconnected graphs to a set of DGPs on connected graphs, showing that the connectivity assumption on G yields no loss of generality.

2.3.2 Formalization of the DGP

A *realization* is a function which maps a set of vertices to a Euclidean space of some given dimension.

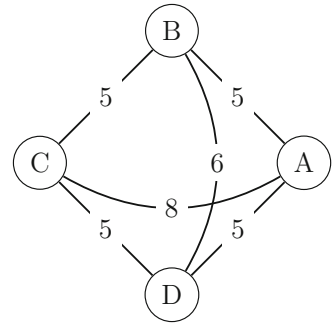


Fig. 2.4 Graph encoding the distances (2.1).

```
x = {{4,0},{0,3},{−4,0},{0,−3}}; {MatrixForm[x], MatrixForm[Transpose[x]]}
```

$$\left\{ \begin{pmatrix} 4 & 0 \\ 0 & 3 \\ -4 & 0 \\ 0 & -3 \end{pmatrix}, \begin{pmatrix} 4 & 0 & -4 & 0 \\ 0 & 3 & 0 & -3 \end{pmatrix} \right\}$$

Fig. 2.5 Since we wish x_i to be the column vector representing the ith point of the realization x, in *Mathematica* we write x as a list of lists: each of the n elements of x is itself a list with K elements, as shown in the code above. *Mathematica*, however, encodes matrices by row: x is shown (on the left) as an $n \times K$ matrix; to make sure matrix computations are dimensionally correct, we have to either multiply on the right, as in the product (`SalesTable . Proj3D` in Fig. 1.3), or use the transpose, as above.

DISTANCE GEOMETRY PROBLEM (DGP). Given a simple, connected, weighted graph $G = (V, E, d)$ and an integer $K > 0$, is there a realization $x : V \to \mathbb{R}^K$ such that:

$$\forall \{u, v\} \in E \quad \|x(u) - x(v)\| = d_{uv} \; ? \tag{2.2}$$

Since a realization certifies a positive answer to a DGP problem instance, realizations are sometimes also called "solutions" or "certificates."

We usually write x_u, x_v instead of $x(u), x(v)$ and limit our attention to the Euclidean distance $\|x_u - x_v\|_2$. Moreover, since each x_v is in $\mathbb{R}^K$ for any $v \in V$, x can in fact be written as a rectangular $K \times n$ array:

$$x = \begin{pmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{K1} & \cdots & x_{Kn} \end{pmatrix},$$

where $n = |V|$, and the vth column is the K-vector x_v (see Fig. 2.5). With a slight abuse of notation, we often write $x \in \mathbb{R}^{Kn}$ to mean that x is a $K \times n$ matrix. This means that elements of $\mathbb{R}^{Kn}$ are not to be seen as linear sequences of Kn elements; as a consequence, the rank of x is to be considered as varying between 0 and K rather than between 0 and 1.

2.4 A quadratic system of equations

If $\|\cdot\|$ is the Euclidean norm, then Eq. (2.2) becomes[2]:

$$\forall \{u, v\} \in E \quad \sqrt{\sum_{k \le K} (x_{uk} - x_{vk})^2} = d_{uv}. \tag{2.3}$$

The fourth metric axiom (see Appendix A.6) states that distances are always nonnegative, so we can

[2]The symbol $\sum_{k \le K}$ stands for $\sum_{k=1}^{K}$.

square both sides of Eq. (2.3) and obtain:

$$\forall \{u, v\} \in E \quad \sum_{k \le K} (x_{uk} - x_{vk})^2 = d_{uv}^2, \tag{2.4}$$

which is a multivariate polynomial system of equations of degree 2.

2.4.1 The number of solutions

Any realization x satisfying Eq. (2.3) can be translated and rotated in $\mathbb{R}^K$ without any change to the pairwise distances, implying that the cardinality of the solution set is not only infinite, but uncountably so (see Appendix A.1).

However, what if we were to discount the effect of translations and rotations? How many distinct realizations would a given weighted graph have in $\mathbb{R}^K$ then? The answer depends on the structure of the graph, as well as on the edge weight function: there could be no realization, a unique realization, finitely many, or uncountably many (see Fig. 2.6). A result in real algebraic geometry [10] precludes the only other possibility, i.e., a countable infinity of realizations.

2.4.1 Example

Consider the complete graph shown in Fig. 2.6, left: there is no way to move a vertex while at least two other vertices remain fixed. In other words, aside from rotating or translating the whole drawing, there is a unique realization of the given graph (note that reflecting the drawing with respect to the axis given by the two fixed vertices yields a drawing which can be superposed on the original drawing by rotations and translations—only with two swapped vertex labels). Consider now the triangle graph given in Fig. 2.6, above middle: keeping two vertices fixed does not fix the third one, which can be reflected across the fixed edge, as shown in Fig. 2.6, below middle, yielding a new drawing which cannot be superposed on the original drawing using rotations and translations only.

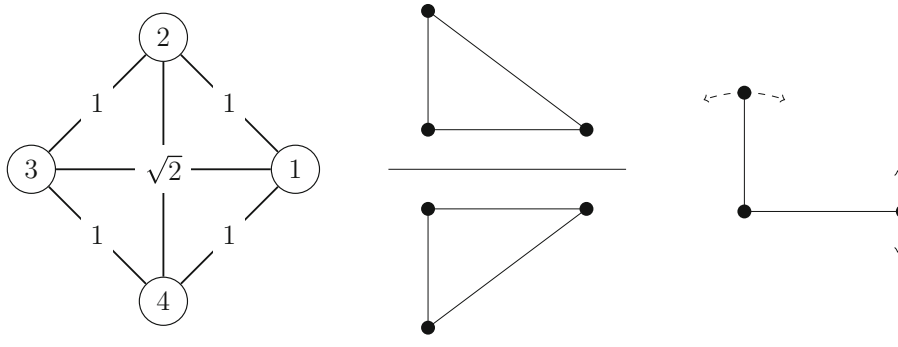


Fig. 2.6 In $\mathbb{R}^2$: a unique solution (left), two solutions (center), and uncountably many solutions (right). Finally, consider the graph in this figure, right the arrows show the directions of the continuous movements which the corresponding vertices can make without changing any of the given edge lengths, and while keeping the other two vertices fixed.

2.4.2 Computational complexity of the DGP

The DGP is **NP**-hard [107] (see Appendix A.9). The proof works as follows: we pick another **NP**-hard problem P, and we show that there is a polynomial transformation (also known as *reduction*, see Sect. A.9.6) from P to DGP such that P is a YES instance if and only if the reduced DGP instance is YES. This implies that if we could solve DGP, then P could be solved by means of a polynomial numbers of calls to a solution algorithm for DGP, which is another way of saying that DGP is at least as hard P modulo a polynomial amount of computational effort. Since P is assumed to be **NP**-hard, then DGP must also be in the same complexity class. Saxe's proof reduces the PARTITION problem to the DGP for $K = 1$.

PARTITION. Given a sequence $A = (a_1, \ldots, a_n)$ of nonnegative integers, is there a subset $I \subseteq \{1, \ldots, n\}$ such that $\sum_{i \in I} a_i = \sum_{i \notin I} a_i$?

This is the first nontrivial proof we present. The reader should be familiar with basic notions of computational complexity, such as reduction and hardness. Read Appendix A.9 before moving on.

For a given instance $(a_1, \ldots, a_n)$ of PARTITION, consider a simple cycle $C = (V, E)$ with $|V| = |E| = n$ and an edge weight function d such that $d_{i,i+1} = a_i$ for $i \leq n-1$, and $d_{n1} = a_n$: by arbitrarily setting $K = 1$, we make this weighted cycle graph into a DGP instance.

Assume that the given PARTITION instance is a YES instance: we show that the corresponding DGP instance is also a YES instance, i.e., there is a realization x of C in 1D (i.e., on a line). We construct a realization x of C in $\mathbb{R}$ inductively, as follows:

1. we fix $x_1 = 0$;
2. if we know the position x_i for $i < n$ and $i \in I$, we let $x_{i+1} = x_i + d_{i,i+1}$ (right placement), else $x_{i+1} = x_i - d_{i,i+1}$ (left placement).

Following the same induction on i, we easily prove that x is a valid realization for C; it suffices to assume it is correct as far as vertex i and conclude, by the induction hypothesis, that it is also correct for vertex $i+1$, since we place x_{i+1} at distance $d_{i,i+1}$ from i. This induction holds as far as we are applying rule 2 above, i.e., until $i = n-1$, which yields position for $i+1 = n$. In order for the realization to be valid, however, we also need to ensure that the distance d_{1n} is preserved. To this end, we define a dummy index $n+1$ to be equivalent to index 1: the position x_{n+1}, computed according to rule 2 where $d_{n,n+1} = d_{n1} = d_{1n}$, should turn out to be equal to x_1, which we still need to prove.

We have:

$$\begin{aligned}
\sum_{i \in I}(x_{i+1} - x_i) &= \sum_{i \in I} d_{i,i+1} \quad \text{[by defn. of the cycle graph]} \\
&= \sum_{i \in I} a_i = \sum_{i \notin I} a_i \quad \text{[Partition instance is YES]} \\
&= \sum_{i \notin I} d_{i,i+1} = \sum_{i \notin I}(x_i - x_{i+1}).
\end{aligned}$$

This implies that the first and the last terms are equal, hence:

$$\begin{aligned}
0 &= \sum_{i \in I} (x_{i+1} - x_i) - \sum_{i \notin I} (x_i - x_{i+1}) \\
&= \sum_{i \in I} (x_{i+1} - x_i) + \sum_{i \notin I} (x_{i+1} - x_i) \quad \text{[sign change in 2nd term]} \\
&= \sum_{i \leq n} (x_{i+1} - x_i) \quad \text{[grouping terms from both sums]} \\
&= (x_{n+1} - x_n) + (x_n - x_{n-1}) + \ldots + (x_2 - x_1) \\
&= x_{n+1} + (x_n - x_n) + \ldots + (x_2 - x_2) - x_1 = x_{n+1} - x_1,
\end{aligned}$$

implying $x_{n+1} = x_1$ as claimed. So the DGP instance is YES.

Now, assume that the given PARTITION instance is a NO instance, and suppose, to get a contradiction, that the corresponding DGP instance is a YES instance. So we suppose there is a realization x of C in 1D. Since we are realizing on a line, for any two points x_u, x_v, we either have $x_u \leq x_v$ or $x_u > x_v$. Let $F = \{\{u, v\} \in E \mid x_u \leq x_v\}$, so that $E \setminus F$ will only contain edges $\{u, v\}$ for which $x_u > x_v$. Because C is a cycle, starting with any vertex v, we must be able to walk the cycle from v back to itself passing through every vertex; for this to hold, the walk must have one direction over all edges in F and the opposite direction for all edges in $E \setminus F$. Since there is a unique position x_v for each vertex v, the distance walked in one direction must be equal to the distance walked in the opposite direction. Hence:

$$\begin{aligned}
& \sum_{\{u,v\} \in F} (x_v - x_u) = \sum_{\{u,v\} \in E \setminus F} (x_u - x_v) \\
\Rightarrow \quad & \sum_{\{u,v\} \in F} |x_u - x_v| = \sum_{\{u,v\} \in E \setminus F} |x_u - x_v| \\
\Rightarrow \quad & \sum_{\{u,v\} \in F} d_{uv} = \sum_{\{u,v\} \in E \setminus F} d_{uv}. \qquad (2.5)
\end{aligned}$$

By definition, every edge in E has the form $\{i, i+1\}$ for $i < n$ or $\{n, 1\}$: let J be the set of all $i < n$ such that $\{i, i+1\}$ is in F, and let it also contain n if $\{n, 1\}$ is in F. Then, Eq. (2.5) becomes:

$$\sum_{i \in J} a_i = \sum_{i \notin J} a_i,$$

which implies that J is a solution for the given PARTITION instance, against the assumption that it was a NO instance. Hence, the corresponding DGP instance must also be a NO instance, as claimed.

Lastly, it is easy to note that the transformation of a PARTITION instance into the corresponding DGP instance can be carried out in time bounded by a polynomial in n, since for each $i \leq n$ we construct a vertex and an edge in the cycle graph.

This means that we have a polynomial-time (also known as *polytime*) transformation to turn PARTITION instances to DGP instances with $K = 1$ so that YES instances map to YES instances and NO instances map to NO instances. In other words, if we could solve the DGP in polytime, then we could exploit this polytime transformation to solve PARTITION in polytime too. But since PARTITION is **NP**-hard [51], then DGP must also be **NP**-hard with $K = 1$. And since the case $K = 1$ determines a subset of instances of the DGP, the DGP itself must be **NP**-hard.

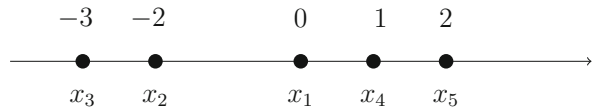


Fig. 2.7 Realization of a cycle in 1D.

2.4.2 Example
Consider the PARTITION *instance* $a_1 = 2$, $a_2 = 1$, $a_3 = 4$, $a_4 = 1$, $a_5 = 2$. *We construct the cycle* C *over* $\{1, 2, 3, 4, 5\}$ *with edges* $\{i, i+1\}$ *for* $i \leq 4$ *and* $\{5, 1\}$ *(which closes the cycle), weighted by* a_i *for each* $i \leq 4$ *and* $d_{51} = a_5$. *We realize* C *with* $x_1 = 0$, $x_2 = -2$, $x_3 = -3$, $x_4 = 1$, $x_5 = 2$, *as shown in Fig. 2.7. Now, the set* F *of edges* $\{u, v\}$ *with* $u < v$ *and* $x_u \leq x_v$ *is* $\{3, 4\}$ *and* $\{4, 5\}$, *so* $J = \{3, 4\}$, *and it is easy to verify that* $\sum_{i \in J} a_i = 4 + 1 = 5 = 2 + 1 + 2 = \sum_{i \notin J} a_i$, *showing that* (a_3, a_4) *and* (a_1, a_2, a_5) *is the desired partition.*

Saxe also proved that the DGP is **NP**-hard for any fixed value of K, using a more complicated reduction from a different **NP**-hard problem.

2.5 Direct solution methods

The simplest approach to find the set of solutions of Eq. (2.4) is to attempt to solve the system of equations directly. In general, and for given $K \geq 2$, there is evidence that a closed-form solution where every component of x is expressed by radicals (i.e., using integers combined with sum, difference, product, fraction, power, and roots) is not possible [6, 9].

Numerically, Eq. (2.4) is difficult to solve directly.

2.5.1 A global optimization formulation

Instead, we can formulate the problem as a global optimization (GO) problem, where we minimize the sum of the errors over all the equations:

$$\min_{x \in \mathbb{R}^{Kn}} \sum_{\{u,v\} \in E} \left(\sum_{k \leq K} (x_{uk} - x_{vk})^2 - d_{uv}^2 \right)^2 . \tag{2.6}$$

If the global optimum x^* has value zero, then every term in the sum has error zero, which implies that x^* is a realization of the given weighted graph $G = (V, E, d)$. Conversely, if x^* is a realization of G, then Eq. (2.6) has value zero. Moreover, Eq. (2.6) being a sum of nonnegative values, it cannot be negative itself. From all this, we conclude that this formulation of the problem identifies a realization if and only if the global optimum has value zero.

What if G cannot be realized in $\mathbb{R}^K$? Then, the global optimum x^* of Eq. (2.6) will yield a function value strictly greater than zero. From a geometrical point of view, x^* provides the smallest change to the edge weighting of G that does yield a realization.

Equation (2.6) aims at minimizing a multivariate polynomial of degree four. It turns out that solving such problems is hard at a computational complexity point of view, as well as from a practical one.

```
y = Map[(DGPSystemApproxGlobal[G,2,#])&,
  {"NelderMead","DifferentialEvolution","SimulatedAnnealing","RandomSearch"}];
t = Map[(RelativePartialEDMError[A, EuclideanDistanceMatrix[y [[#]]]]) &,Range[4]];
i = Flatten[Position[t,Min[t ]]][[1]];
GraphPlot[G, VertexCoordinateRules->y[[i]] ]
```

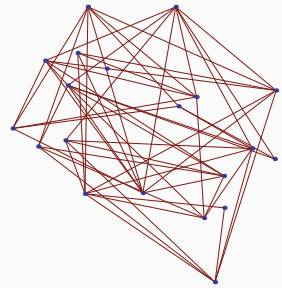


Fig. 2.8 G is a random graph created as in Fig. 2.9. We solve the corresponding DGP by means of Eq. (2.6) using each of the four GO methods in *Mathematica*, then evaluate their mean relative per-distance error with respect to the distances in G, pick the minimum, and plot the corresponding realization. To understand what the definition of i does, use *Mathematica* to print out `Min[t]`, `Position[t,Min[t]]`, and `Flatten[Position[t,Min[t]]]`. For any list `L`, `L[[i]]` returns its ith element.

Some general-purpose GO approaches have been tested in [78]. Unfortunately, none of them scales well to medium or large instances. We test some of these methods (Fig. 2.8) on random graphs (Fig. 2.9).

One of the challenges of GO is that many of the existing methods—specially the most efficient ones—do not provide a guarantee of optimality. In other words, we may find a solution x' of Eq. (2.6) which is not a global optimum, but simply a local one. Such a solution can be considered as an

```
n = 20;
m = Round[N[0.3*n (n - 1)/2]];
G = RandomGraph[{n, m}, EdgeWeight -> RandomReal[{0, 10}, m]];
A = Normal[WeightedAdjacencyMatrix[G]];
GraphPlot[G, VertexLabeling -> True]
MatrixPlot[A]
```

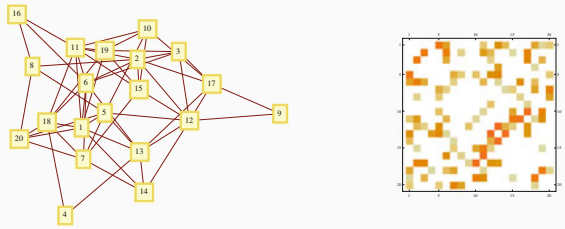


Fig. 2.9 Creating a random graph with n vertices and m edges in *Mathematica*. The pictures show a drawing of the graph in 2D (left, made by *Mathematica*), and the sparsity structure of the distance matrix (right).

approximate realization of G, the approximation error of which depends on the value of Eq. (2.6) at x': the closer to zero, the better the approximation. This does not imply that there is a provable guarantee that the approximation error is bounded by the objective function value; it may happen occasionally that a very small but nonzero objective function value corresponds to a large error in the realization (e.g., if there exist two extremely different realizations having almost exactly the same edge lengths). But, empirically speaking, this is unlikely.

2.6 Exercises

2.6.1 Exercise

Consider a triangle graph ($V = \{1, 2, 3\}$, $E = \{\{1, 2\}, \{1, 3\}, \{2, 3\}\}$) with unit edge weights $d_{12} = d_{13} = d_{23} = 1$. Let the position of vertex $v = 1$ be the origin. Write down the systems in Eq. (2.4) for $K \in \{1, 2, 3\}$, and solve these systems, either algebraically or numerically (e.g., using *Mathematica*). How many solutions are there for $K = 1$? For $K = 2$? And for $K = 3$?

2.6.2 Exercise

Consider two disjoint triangle graphs with unit edge weights, with the position of vertex $v = 1$ fixed at the origin. Write down the system in Eq. (2.4) for $K = 2$ and provide a solution. How many solutions are there? Now, fix another vertex position, say $v = 2$: how many solutions?

2.6.3 Exercise

Consider two disjoint triangle graphs with unit edge weights sharing a vertex, as in Fig. 2.10. Write down the system in Eq. (2.4) for $K = 2$ and provide a solution. How many solutions are there? Now, by fixing the position of vertex $v = 3$, how many solutions? And finally by fixing the position of vertex $v = 5$, how many solutions?

2.6.4 Exercise

Prove (from first principles) that a simple cycle on n vertices has n edges.

2.6.5 Exercise

Prove that any simple cycle can be decomposed into two simple paths with no common edges. How many such decompositions are there?

2.6.6 Exercise

Consider the procedure used in Sect. 2.4.2 to reduce a PARTITION instance into a DGP one. Apply it to the following PARTITION instances: (1, 1, 1, 3, 2) and (1, 2, 3, 4, 5, 6). Derive corresponding DGP instances in 1D and solve them. Use the solutions to decide whether the original PARTITION instances are YES or NO.

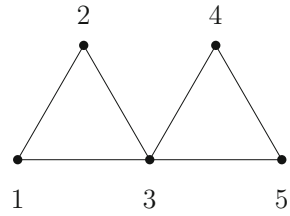


Fig. 2.10 Graph consisting of two triangle graphs sharing a vertex.

Chapter 3
Realizing complete graphs

In this chapter, we consider the DGP on a very specific class ofgraphs: the $(K+1)$-cliques, i.e., complete graphs on $K+1$ vertices, where K is the dimension of the embedding space $\mathbb{R}^K$.

3.1 Cliques

In a one-dimensional space (i.e., a line), we have $K+1=2$. The complete graph on two vertices is $\mathbf{K}_2 = (\{1,2\}, \{\{1,2\}\})$, i.e., the graph consisting of two vertices and one edge between them (see Fig. 3.1, left). In 2D, the complete graph on three vertices is a triangle (Fig. 3.1, middle); in 3D, we have a tetrahedron (Fig. 3.1, right).

Cliques have a recursive structure: let $\mathbf{K}_{K+1}$ be a $(K+1)$-clique, and let v be any vertex in $V(\mathbf{K}_{K+1})$. Then, the graph induced by $V(\mathbf{K}_{K+1}) \setminus \{v\}$ is a K-clique. For example, a tetrahedron minus one of its vertices and its three adjacent edges is a triangle, and a triangle minus one of its vertices and its two adjacent edges is simply a segment (see Fig. 3.1).

3.2 Realizing $(K+1)$-cliques in $\mathbb{R}^{K-1}$

In general, if a $(K+1)$-clique has a realization in $\mathbb{R}^{K-1}$, it is unique up to translations and rotations; whereas, if it has a realization in $\mathbb{R}^K$, then a reflection will yield a different realization (this statement will be made clearer in the course of this chapter). In this sense, realizing $(K+1)$-cliques in $\mathbb{R}^{K-1}$ can be considered to be simpler than realizing them in $\mathbb{R}^K$, which is the reason why we study this problem first.

Cliques are often used as "basic building blocks" for constructing realizations iteratively. Consider a 3-clique on $V = \{1,2,3\}$, where $d_{12}=1$, $d_{13}=2$, and $d_{23}=1$. Then, a possible realization on the real line $\mathbb{R}$ which satisfies all the given distances is $x_1=0$, $x_2=1$, $x_3=2$.

One way to obtain the value for x_3 given the values for x_1 and x_2 (say $x_1=0, x_2=1$) and the distances d_{13} and d_{23} is *trilateration*: from $d_{13} = \|x_3 - x_1\| = 2$ and $d_{23} = \|x_3 - x_2\| = 1$, we get:

$$x_3^2 - 2x_1x_3 + x_1^2 = 4 \tag{3.1}$$
$$x_3^2 - 2x_2x_3 + x_2^2 = 1. \tag{3.2}$$

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_3

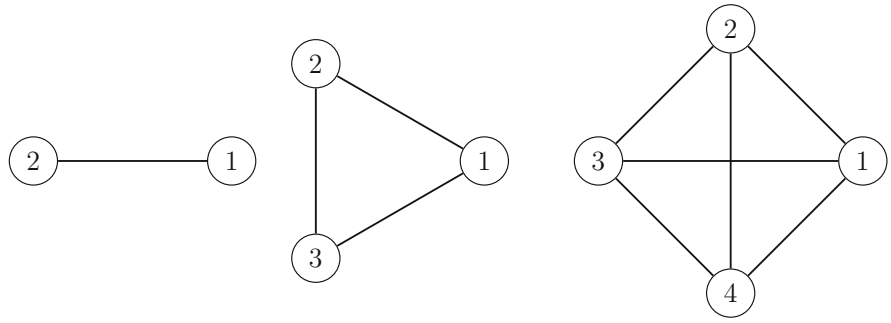


Fig. 3.1 2-clique (left), a 3-clique (center), a 4-clique (right).

Now subtract Eq. (3.2) from (3.1), to obtain:

$$\begin{aligned} 2x_3(x_1 - x_2) &= x_1^2 - x_2^2 - 3 \\ \Rightarrow \quad 2x_3 &= 4, \end{aligned}$$

whence $x_3 = 2$.

This example can be generalized to $(K+1)$-cliques: in $\mathbb{R}^{K-1}$, we need to know the position of K vertices (and the distances from these K vertices to the $(K+1)$-st) to find the position of the $(K+1)$-st.

3.2.1 The trilateration system in $\mathbb{R}^{K-1}$

Let $x_1, \ldots, x_K \in \mathbb{R}^{K-1}$ be given positions for K vertices of a $(K+1)$-clique, and, for all $j \leq K$, let $d_{j,K+1}$ be the distance value associated with the edge $\{j, K+1\}$. Let y be the position in $\mathbb{R}^{K-1}$ of the $(K+1)$-st vertex; then, y must satisfy the K quadratic equations $\forall j \leq K \ \|y - x_j\|^2 = d_{j,K+1}^2$ in the $K-1$ unknowns $y_1, \ldots, y_{K-1}$:

$$\left.\begin{aligned} \|y\|^2 - 2x_1 \cdot y + \|x_1\|^2 &= d_{1,K+1}^2 \\ &\vdots \\ \|y\|^2 - 2x_K \cdot y + \|x_K\|^2 &= d_{K,K+1}^2. \end{aligned}\right\} \tag{3.3}$$

For any $h \leq K$, let e_h be the hth equation in the above system: we then take differences and form a new system $\forall h < K \ (e_h - e_K)$ which has $K-1$ equations in $K-1$ unknowns:

$$\left.\begin{aligned} 2(x_1 - x_K) \cdot y &= \|x_1\|^2 - \|x_K\|^2 - d_{1,K+1}^2 + d_{K,K+1}^2 \\ &\vdots \\ 2(x_{K-1} - x_K) \cdot y &= \|x_{K-1}\|^2 - \|x_K\|^2 - d_{K-1,K+1}^2 + d_{K,K+1}^2. \end{aligned}\right\} \tag{3.4}$$

This is a linear system of the form

$$Ay = b, \tag{3.5}$$

where $A = (2a_{ij})$ is a square $(K-1) \times (K-1)$ matrix with $a_{ij} = x_{ij} - x_{Kj}$ for all $i, j < K$, and $b = (b_1, \ldots, b_{K-1})^\top$ with $b_i = \|x_i\|^2 - \|x_K\|^2 - d_{i,K+1}^2 + d_{K,K+1}^2$ for all $i < K$.

3.2.2 Solving the linear system

If A is nonsingular (see Appendix A.5.5), then it has an inverse A^{-1}. Then, by left-multiplying both sides of Eq. (3.5) by A^{-1}, we get $A^{-1}Ay = A^{-1}b$, which implies that the position of the $(K+1)$-st vertex is $x_{K+1} = y = A^{-1}b$. The worst-case computational complexity of solving a square linear system of size $(K-1) \times (K-1)$ is $O(K^3)$. In practice there are better methods, depending on the sparsity and structure of the matrix A [35].

If A is singular, it means that the rows a_i are not all linearly independent, which in turn implies that the difference vectors $x_i - x_K$ are not linearly independent (for $i < K$). For example, if $K = 2$, it means that $x_1 - x_2 = 0$, i.e., $x_1 = x_2$, or, in other words, the segment between x_1 and x_2 is a single point. Since we are embedding in $\mathbb{R}^{K-1} = \mathbb{R}$, i.e., the real line, geometrically the situation is that x_3 could be either left or right of $x_1 = x_2$, but we cannot tell which. If $K = 3$, the singularity of A implies that the triangle on x_1, x_2, x_3 is either a segment in the plane (if the rank of A is 1) or a single point (if the rank of A is 0). In the former case, x_4 could be either side of the line containing the segment; in the latter, x_4 could be on the circle centered at $x_1 = x_2 = x_3$ with radius $d_{14} = d_{24} = d_{34}$. This trend holds for higher values of K: the singularity of A implies that there may be multiple solutions for x_{K+1}.

Notice that Eq. (3.3) implies Eq. (3.5) but not vice versa. In particular, if Eq. (3.3) has a solution, then Eq. (3.5) has the same solution. But Eq. (3.5) always has a unique solution, as long as A is nonsingular; whereas Eq. (3.3) may not have a solution at all. Thus, the solution of Eq. (3.5) must be verified to be feasible with respect to Eq. (3.3). It turns out that it is sufficient to verify the solution with respect to the Kth equation of Eq. (3.3). To see this, we introduce a nonstandard notation which will hopefully make things clearer: let $[i]$ stand for the ith equation in Eq. (3.3). Then, we can say that Eq. (3.5) is obtained as:

$$\left.\begin{aligned} [1] - [K] \\ \vdots \\ [K-1] - [K], \end{aligned}\right\} \tag{3.6}$$

Let $(\star)$ be the system consisting of Eq. (3.6) and $[K]$. If we add $[K]$ to each of the equations in Eq. (3.6), we obtain Eq. (3.3), so $(\star)$ implies and is implied by Eq. (3.3). Hence, if y solves Eq. (3.5) and is also feasible with respect to $[K]$, then y solves Eq. (3.3). Conversely, if y is infeasible in $[K]$, then it is also infeasible in $(\star)$, and hence in Eq. (3.3).

For later reference, we let $\mathsf{Next}_d^{K-1}(x_1, \ldots, x_K) = x_{K+1}$ be the position of the $(K+1)$-st vertex of a d-weighted $(K+1)$-clique in $\mathbb{R}^{K-1}$. If Eq. (3.3) is infeasible at the solution found by solving Eq. (3.5), Next will return the symbol $\emptyset$ meaning "no solution."

3.2.3 Iterative realization of complete graphs

We now apply the results of the previous sections to the task of realizing complete weighted graphs $G = (V, E, d)$ in $\mathbb{R}^K$ (where d is an edge weight function) whenever $K < n = |V|$. The resulting algorithm (Alg. 1) is called RealizeComplete.

We first assume there is a $(K+1)$-clique $\mathbf{K}(G)$ in G for which we know a realization: we shall call this the *initial* clique. Without loss of generality, we let the initial clique be defined on $V(\mathbf{K}(G)) = \{1, \ldots, K+1\}$ and be realized as $x = (x_1, \ldots, x_{K+1})$.

The initial clique $\mathbf{K}(G)$ is used in Algorithm 1 to start an iterative process where each subsequent vertex is realized using distances to $K+1$ vertices the positions of which have been found during previous steps (see Fig. 3.2). In Algorithm 1, $N(i)$ is the set of vertices adjacent to vertex i, and we

```
G = Graph[{1<->2,1<->3,1<->4,1<->5,2<->3,2<->4,2<->5,3<->4,3<->5,4<->5},
EdgeWeight->{1,2,1,1/Sqrt[2],2,1,1/Sqrt[2],(Sqrt[15]+Sqrt[3])/2,(Sqrt[15]-1)/2,(Sqrt[3]+1)/2}];
x0 = {{1/2,1/2}, {1/2,-1/2}, {1/2+Sqrt[15]/2,0}};
x = RealizeComplete[G, 2, x0];
GraphPlot[G, VertexLabeling -> True, VertexCoordinateRules -> x]
```

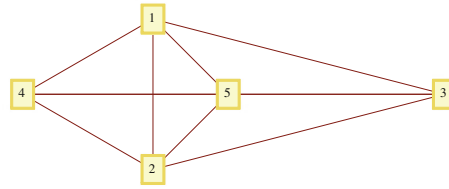


Fig. 3.2 Complete weighted graph G on five vertices, an initial realization x^0 in $\mathbb{R}^2$ for vertices $\{1, 2, 3\}$, and the corresponding realization x of G in $\mathbb{R}^2$, found by RealizeComplete

use the symbol $\emptyset$ to denote the infeasibility of a vertex position. Note that in Sect. 3.2.2, we defined $\mathsf{Next}_d^{K-1}(x_1, \ldots, x_K) = x_{K+1}$ as the position of vertex $K+1$ of the d-weighted $(K+1)$-clique in $\mathbb{R}^{K-1}$. It therefore follows that $\mathsf{Next}_d^K(x_{i-K-1}, \ldots, x_{i-1})$, when $i = K+2$, denotes the position of vertex $K+2$ of the d-weighted $(K+2)$-clique in $\mathbb{R}^K$. The worst-case complexity of Algorithm 1 is

Algorithm 1 $x = \mathsf{RealizeComplete}(G, d, K, x)$

```
// realize next vertex iteratively
for i ∈ {K + 2, ..., n} do
  // use (K + 1) immediate adjacent predecessors to compute a unique position for x_i
  x_i = Next_d^K(x_{i-K-1}, ..., x_{i-1})
  // check that x_i is feasible w.r.t. other distances
  for {j ∈ N(i) | j < i} do
    if ||x_i - x_j|| ≠ d_ij then
      // if not, mark infeasible and exit loop
      x_i = ∅
      break
    end if
  end for
  if x_i = ∅ then
    // return infeasible flag
    return ∅
  end if
end for
// return feasible realization
return x
```

$O(K^3 n)$ (for each of n vertices, we have to solve a $K \times K$ linear system). If no feasible realization of G exists in $\mathbb{R}^K$, Algorithm 1 returns $\emptyset$.

The process of trilateration in $\mathbb{R}^K$ is called *K-lateration* [46]. Algorithm 1 was discussed in [41].

3.3 Realizing $(K+1)$-cliques in $\mathbb{R}^K$

In Sect. 3.2, we showed how to realize $(K+1)$ cliques in $\mathbb{R}^{K-1}$. This was used iteratively in Sect. 3.2.3 to realize complete graphs in $\mathbb{R}^K$. So this method can be used, for example, with $K=2$ to realize a triangle on a line.

Consider this, however. Most triangles can only be realized in the plane or higher dimensional spaces. Only very special triangles can be realized on a line: namely the "flat" triangles where the side lengths are such that $d_{12}+d_{23}=d_{13}$. Likewise, most tetrahedra can only be realized in 3D or higher dimensional spaces: only flat tetrahedra can be realized on a plane. From this point of view, it would seem more natural to look for a realization of $(K+1)$-cliques in $\mathbb{R}^K$ rather than $\mathbb{R}^{K-1}$.

Starting from Eq. (3.3), we can carry out the same reasoning as in Sect. 3.2.1, with the exception that all vectors are now in $\mathbb{R}^K$ instead of $\mathbb{R}^{K-1}$. We obtain a system like Eq. (3.4), which is a linear system in the form $Ay=b$, except that in this case A is $(K-1)\times K$, $y=x_{K+1}\in\mathbb{R}^K$ and $b\in\mathbb{R}^{K-1}$. Since this system has one more unknown than equations, we cannot solve it in the same way as before.

3.3.1 Basic and nonbasic columns

Let us assume that A has rank $K-1$, which means that exactly $K-1$ of the K columns of A are linearly independent, and exactly one depends linearly on the others (see Appendix A.4). Without loss of generality, we can rearrange the columns of A so the linearly dependent column is last. So A looks like:

$$\left(\begin{array}{ccc|c} a_{11} & \dots & a_{1,K-1} & a_{1K} \\ \vdots & \ddots & \vdots & \vdots \\ a_{K-1,1} & \dots & a_{K-1,K-1} & a_{K-1,K} \end{array}\right) = (B|N),$$

where B is a nonsingular square $(K-1)\times(K-1)$ matrix, and N is a column which depends linearly on the columns of B. The columns of B are called *basic*, and the column in N is called *nonbasic*.

3.3.2 Expressing basics as linear functions of nonbasics

Partitioning a rectangular matrix A into basic and nonbasic columns $(B|N)$ is useful because it allows us to write a closed-form expression for the affine space of solutions of the corresponding system $Ay=b$. Let y_B be the vector consisting of those components of y corresponding to the indices of the columns in B, and y_N be the single component (i.e., the scalar) corresponding to the index of column N, so that $y=(y_B,y_N)$. Then, $Ay=b$ can be written as $By_B+Ny_N=b$, which, by the nonsingularity of B, yields $y_B=B^{-1}b-B^{-1}Ny_N$.

Since N is the Kth column of A, we have:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_{K-1} \end{pmatrix} = B^{-1}b - B^{-1}\begin{pmatrix} a_{1K} \\ \vdots \\ a_{K-1,K} \end{pmatrix} y_K. \tag{3.7}$$

If we let $B^{-1} = (\beta_{hj})$ and $b = (b_1, \dots, b_{K-1})$, then the system above is:

$$\forall h < K \quad y_h = \sum_{j \le K-1} \beta_{hj}(b_h - a_{jK} y_K). \tag{3.8}$$

Notice that, in Eq. (3.8), every variable y_h (with $h < K$) is expressed in terms of the single variable y_K. Geometrically, this means that the solution set is a 1D affine space, i.e., a line.

One word of warning is due: if $a_{jK} = 0$ for each $j \le K$, then Eq. (3.7) reduces to[1]:

$$(y_1, \dots, y_{K-1})^\top = B^{-1} b, \tag{3.9}$$

which does *not* express y_K in function of $y_1, \dots, y_{K-1}$. This case, however, is simpler to handle than the general one: it suffices to compute $y_1, \dots, y_{K-1}$ by means of Eq. (3.9), and then compute y_K by replacing $y_1, \dots, y_{K-1}$ in any of the distance equations $\|y - x_h\|^2 = d^2_{h,K+1}$ for any $h \le K$.

3.3.3 The K-lateration system in $\mathbb{R}^K$

Our aim is to solve Eq. (3.3) when $a_{jK} \neq 0$ for at least one $j \le K$. In Sects. 3.3.1–3.3.2, we obtained an explicit description of the first $K-1$ components of $y = x_{K+1} \in \mathbb{R}^K$ in function of the last component y_K. In order to find the values of y_K that yield feasible solutions for Eq. (3.3), we combine Eq. (3.8) with any quadratic equation in Eq. (3.3) (e.g., the first):

$$\left.\begin{aligned} \forall h < K \quad y_h &= \sum_{j<K} \beta_{hj}(b_h - a_{jK} y_K) \\ d^2_{1,K+1} &= \sum_{h \le K} y_h^2 - 2 \sum_{h \le K} x_{1h} y_h + \|x_1\|^2. \end{aligned}\right\} \tag{3.10}$$

Notice that the last equation in (3.10) is simply $\|y - x_1\|^2 = d^2_{1,K+1}$, and therefore represents a K-dimensional sphere centered at x_1 with radius $d_{1,K+1}$. Moreover, we chose x_1 without loss of generality: in fact any $i \in \{1, \dots, K\}$ would have worked. The intersection of a line with a sphere, if it exists, consists in general of two points (see Fig. 3.3), which may become a single point if the line is tangent to the sphere. So, if Eq. (3.10) admits a solution at all, we should expect it to be more likely to admit two solutions.

[1] Here is a setup where this might happen in practice. If the linear system $Ay = b$ arises from taking equation differences $\forall 1 < h \le K$ ($[h] - [1]$), which is different but equivalent to Eq. (3.6), and x_1 is arbitrarily set to 0, then an application of RealizeClique (Algorithm 2) would yield x_h with the last $K - h + 1$ components set to zero, for each $h \le K$, which would cause a last zero column to appear in the matrix $A = 2(x_h - x_1)^\top$. Choosing $h = 1$ in this case would yield $\|y\|^2 = d^2_{1,K+1}$, i.e., $y_K = \pm\sqrt{d^2_{1,K+1} - \sum_{j<K} y_j^2}$.

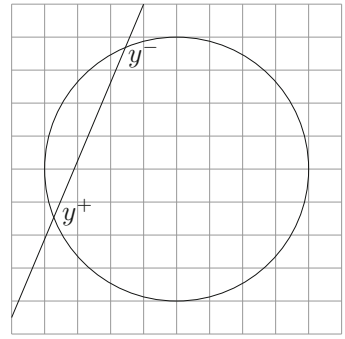


Fig. 3.3 Line intersects a circle in two points y^+, y^-.

We replace each y_h (for $h < K$) in the last equation of (3.10) with the corresponding linear forms in y_K from the first $K-1$ equations of (3.10). This yields a quadratic equation in function of y_K:

$$\lambda y_K^2 - 2\mu y_K + \nu = 0, \quad \text{where} \tag{3.11}$$

$$\lambda = 1 + \sum_{h,j<K} \beta_{hj}^2 a_{jK}^2 \tag{3.12}$$

$$\mu = x_{1K} + \sum_{h,j<K} \beta_{hj} a_{jK} (\beta_{hj} b_h - x_{1h}) \tag{3.13}$$

$$\nu = \sum_{h,j<K} \beta_{hj} b_h (\beta_{hj} b_h - 2x_{1h}) + \|x_1\|^2 - d_{1,K+1}^2. \tag{3.14}$$

Solving Eq. (3.11) yields two roots y_K^+, y_K^-. If they are complex, then the line does not intersect the sphere. If they are real and coincide, the line is tangent to the sphere; if they are real and distinct, they yield two points y^+, $y^- \in \mathbb{R}^K$ (see Fig. 3.3). The other components (indexed in $1, \ldots, K-1$) of y^+, y^- are given by replacing the symbol y_K in Eq. (3.7) with the values y_K^+, y_K^-. The two points y^+, y^- provide distinct realizations $x^+ = (x_1, \ldots, x_K, y^+)$ and $x^- = (x_1, \ldots, x_K, y^-)$ for the $(K+1)$-clique in $\mathbb{R}^K$.

By construction, it is clear that Eq. (3.3) implies Eq. (3.10). We show the converse, i.e., (3.10) implies (3.3). Since the linear system in (3.8) is $(K-1) \times K$, it has $K-1$ basic variables $(y_1, \ldots, y_{K-1})$ and one nonbasic variable y_K. Moreover, the system in Eq. (3.8) is by construction equivalent to:

$$\left.\begin{aligned} [1] - [K] \\ \vdots \\ [K-1] - [K], \end{aligned}\right\} \tag{3.15}$$

where $[\ell]$ stands for the ℓth equation in Eq. (3.3)—bear in mind that Eq. (3.6) is a $(K-1) \times (K-1)$ system, whereas Eq. (3.15) is $(K-1) \times K$. Eq. (3.10) is, therefore, equivalent to Eq. (3.15) together with one of the quadratic equations (e.g. the ith) of Eq. (3.3). If $i = K$, by what was remarked below Eq. (3.6) in Sect. 3.2.2, we would immediately conclude that Eq. (3.10) and Eq. (3.3) are equivalent. Notice, however, that $([j] - [K]) - ([h] - [K]) = [j] - [h]$ for any $j, h \leq K$. So the linear system Eq. (3.6) can be transformed into an equivalent linear system $\forall j \leq K$ $([j] - [i])$ for any $i \leq K$. This shows that Eq. (3.10) and Eq. (3.3) are equivalent independently of the choice of i.

We let $\mathsf{Next}_d^K(x_1, \ldots, x_K) = \{y^+, y^-\}$ be the set of distinct positions of vertex $K+1$ of the d-weighted $(K+1)$-clique in $\mathbb{R}^K$, which could have cardinality $0, 1, 2$ according to the roots of (3.11).

3.3.4 Differences between $\mathbb{R}^K$ and $\mathbb{R}^{K-1}$

At first sight, it might appear surprising that a $(K+1)$-clique has at most one realization (not considering translations and rotations) in $\mathbb{R}^{K-1}$, and at most two in its "proper" ambient space $\mathbb{R}^K$.

Suppose that the 3-clique over $\{1, 2, 3\}$ has already been realized in 2D (shaded area in Fig. 3.4, left). We now use Eq. (3.11) to compute a realization of the 4-clique over $\{1, 2, 3, 4\}$ in $\mathbb{R}^3$. The fact that we get two distinct roots for $y = x_4$ corresponds to vertex 4 being in two possible positions (above or below the shaded area in Fig. 3.4, left). This yields two tetrahedra that are *reflections* (see Appendix A.5.6) of each other with respect to the plane containing the realization of the 3-clique over $\{1, 2, 3\}$. Notice that, in general, neither tetrahedron can be superposed onto the other using translations and rotations only. Notwithstanding, the two tetrahedra are *congruent*, i.e., all pairwise distances are the same.

If, however, the edge weight function over the 4-clique is such that the tetrahedron it yields can be realized in $\mathbb{R}^2$, then it appears clear from Fig. 3.4 (right) that the two reflected tetrahedra are in fact equal to a single flat tetrahedron. This corresponds to the two roots y_K^+, y_K^- taking the same value.

This discussion also justifies the choice of using the same name Next for the operator that computes the position in either $\mathbb{R}^{K-1}$ or $\mathbb{R}^K$ of vertex $K+1$ of a $(K+1)$-clique, given a realization of vertices $\{1, \ldots, K\}$ in $\mathbb{R}^{K-1}$. More precisely, $x_{K+1} = \mathsf{Next}_d^{K-1}(x_1, \ldots, x_K)$ is the (unique) position of the $(K+1)$-st vertex of a clique realized in $\mathbb{R}^{K-1}$ as a *simplex* (i.e., a polyhedron in $\mathbb{R}^{K-1}$ which is the convex hull of its K vertices) with zero K-dimensional volume (also known as a *flat* simplex), whereas $\mathsf{Next}_d^K(x_1, \ldots, x_K)$ describes the set of at most two positions of the $(K+1)$-st vertex of a clique realized in $\mathbb{R}^K$ as a simplex with nonzero K-dimensional volume.

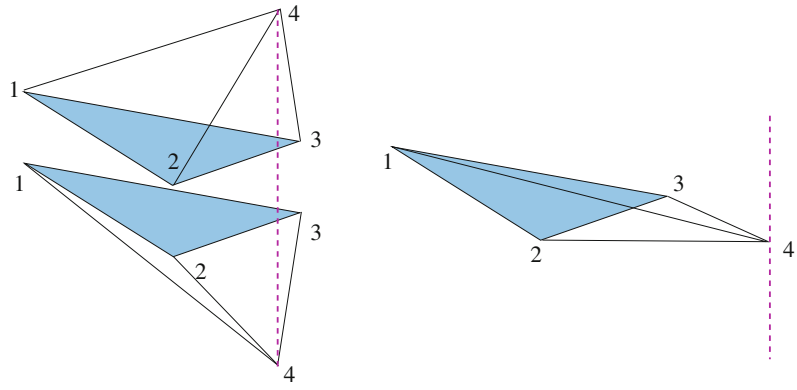


Fig. 3.4 Two realizations in $\mathbb{R}^3$ (left), but just one in $\mathbb{R}^2$ (right).

3.3.5 The realization algorithm

In Sect. 3.2.3, we assumed that a realization for an initial $(K+1)$-clique had to be given in order to realize a complete graph G in $\mathbb{R}^K$. We relax that assumption by showing how to realize the initial $(K+1)$-clique H from the edge weight function d on G.

First, we arbitrarily place x_1 at the origin. Since $\{1\}$ is a 1-clique, we can now use the results in the preceding sections to find two alternative positions (each reflection of the other with respect to x_1) for x_2 in $\mathbb{R}$. Geometrically, either x_2 is on the negative or on the positive side of the origin x_1; we arbitrarily choose the positive side, so $x_2 = (d_{12}, 0, \ldots, 0)$. We repeat the reasoning for x_3: we find two positions for x_3 on the (arbitrary, but convenient) plane defined by the first two coordinate axes, and again arbitrarily choose the positive side with respect to the line containing x_1, x_2. This continues until x_{K+1} is computed (see Fig. 3.5). At each iteration $\ell \leq K+1$, we work in the space of dimension $\ell - 1$ defined by the coordinate axes $2, \ldots, \ell$, compute up to two positions for x_ℓ, and arbitrarily choose one (see Algorithm 2). If (3.11) has no real roots, Algorithm 2 returns the infeasibility flag $\emptyset$.

Algorithm 2 $x = \mathsf{RealizeClique}(H, d, K)$

```
// realize 1 at the origin
x_1 = (0, ..., 0)
// realize next vertex iteratively
for ℓ ∈ {2, ..., K + 1} do
  // using all (adjacent) predecessors, compute two points in R^{ℓ−1} for ℓ
  S = Next_d^{ℓ−1}(x_1, ..., x_{ℓ−1})
  if S = ∅ then
    // warn if infeasible
    return ∅
  end if
  // arbitrarily choose one of the two points
  choose any x_ℓ ∈ S
end for
// return feasible realization
return x
```

Notice that Algorithm 2 is not deterministic ("choose any…"). An arbitrary but deterministic choice of elements in S is, however, implicit. We might say Algorithm 2 describes a class of algorithms (each for every set of arbitrary choices) that all realize cliques in $\mathbb{R}^K$. Indicatively, a convenient rule for disambiguation might aim to reduce poor scaling of the vector components and the distance data.

If no feasible realization exists in $\mathbb{R}^K$ of the given clique, Algorithm 2 will detect it because Eq. (3.11) will have complex roots.

3.3.6 The assumption on the rank of A

We close the chapter with a remark on the first sentence of Sect. 3.3.1, i.e., that A is assumed to have rank $K-1$. Suppose this assumption fails, and that $\operatorname{rk} A = r < K-1$: What are the consequences?

We assume that the given DGP instance is feasible, and so is the linear system $Ay = b$ corresponding to the instance (Eq. (3.5) in Sect. 3.2.1, also see Sect. 3.3.3). A is first reduced to a rectangular $r \times K$ matrix by removing linearly dependent rows of the system. The decomposition $A = (B|N)$ into basic and nonbasic columns yields a square nonsingular $r \times r$ matrix B of linearly independent columns, and a $r \times (K-r)$ matrix N of linearly dependent columns. We then express y_B (the r components

of y corresponding to the column indices of B in A) as linear functions of y_N (the remaining $K-r$ components of y) by means of the system $y_B = B^{-1}b - B^{-1}Ny_N$. The solution set for y_B is a $(K-r)$-dimensional hyperplane in $\mathbb{R}^K$. This hyperplane is to be intersected with $K-r$ spheres in $\mathbb{R}^K$. Since there is no guarantee that this intersection has finitely many elements (as is the case whenever $r = K-1$), we are unable to define a combinatorial realization algorithm such as Algorithm 2.

```
G = CompleteGraph[4, EdgeWeight->ConstantArray[1, 6]]; x = RealizeClique[G];
GraphPlot3D[G, VertexLabeling->True, VertexCoordinateRules->x,
 Axes->True, AxesStyle->Gray]
```

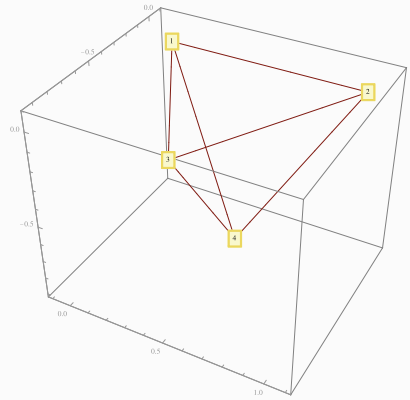


Fig. 3.5 4-clique realization in $\mathbb{R}^3$. The *Mathematica* implementation of RealizeClique only has one argument (the graph G), since the weight d is encoded in the graph option `EdgeWeight`, and the ambient space dimensionality K is simply $|V(G)|-1$.

We can get a different perspective by looking at Eq. (3.4): if $\operatorname{rk} A = r < K-1$, then $K-1-r$ equations are linearly dependent on the others. We can assume without loss of generality that $x_{K-1}-x_K$, $x_{K-2}-x_K$, …, $x_{r+1}-x_K$ are linearly dependent on x_1-x_K, …, x_r-x_K. This means that the points $x_{r+1},\ldots,x_{K-1}$ are in the same r-hyperplane defined by $x_1,\ldots,x_r$ and x_K.

In general, the assumption on the rank of A is essential to the validity of the realization algorithms in the whole of this book.[2]

3.4 Exercises

3.4.1 Exercise

Prove that a $(K+1)$-clique minus one of its vertices is a K-clique. By induction, prove that a $(K+1)$-clique minus h of its vertices (where $h < K$) is a $(K+1-h)$-clique.

[2]In most of our papers, e.g., [84], we formulate this assumption in a more general way, stating that certain statements hold "with probability 1." This is a reference to the fact that if d were uniformly sampled from a compact set, it would yield matrices A having full rank with probability 1.

3.4.2 Exercise

Consider the 4-clique on $V = \{1, 2, 3, 4\}$ with the following distances:

$$d_{12} = 3 \; d_{13} = 4 \quad d_{14} = 4$$
$$d_{23} = 5 \; d_{24} = 5 \; d_{34} = 4\sqrt{2}.$$

Can you realize it in 2D? Can you realize it in 3D? Can you realize it in 4D? Can you realize it in $\mathbb{R}^K$ for general K? If your answer is no, say why. If your answer is yes, provide a realization.

3.4.3 Exercise

Let A be the matrix appearing in Eq. (3.5). Prove that, if $K = 3$ and A has rank 1, then x_4 (denoted by y in Eq. (3.5)) could be either side of the line containing the segment containing x_1, x_2, x_3. Prove also that if $K = 3$ and A has rank 0, x_4 could be on the circle having x_1 as center and d_{14} as radius. What are the other alternatives to these possibilities?

3.4.4 Exercise

Consider the 3-clique having edge weights $d_{12} = d_{13} = 1$ and $d_{23} = 3$. Prove it does not have any realization in $\mathbb{R}^K$ for any $K > 0$. Now write down the linear system (3.5) and prove it has a solution. State whether Eq. (3.5) is a necessary or a sufficient condition for Eq. (3.3) to have a solution.

3.4.5 Exercise

Using *Mathematica* or another software, generate many complete graphs with 10 vertices, assign a set of random weights to the edges, and use Algorithm 1 to determine whether they are realizable or not. What is the fraction of realizable graphs over 10 randomly generated edge weight functions? What about 100 randomly generated edge weight functions? Do your answers change if you generate complete graphs having 20 vertices?

3.4.6 Exercise

Consider a 3-clique on $V = \{1, 2, 3\}$ having unit edge weights, with vertices 1, 2 placed at $x_1 = (0, 0)$ and $x_2 = (1, 0)$ in $\mathbb{R}^2$. At the end of Sect. 3.3.2, we mentioned that the possible positions for x_3 are all on a line. Draw this line on the Euclidean plane. Try different values of d_{13}, d_{23} such that the 3-clique stays realizable: how does the line for x_3 change? Can you infer any property about the angle this line makes with the line defined by x_1, x_2?

3.4.7 Exercise

In the context of RealizeClique (Algorithm 2), derive the linear system $Ay = b$ (see Sect. 3.3, p. 23) by means of subtracting the *first* (instead of the Kth, as in Eq. (3.15)) equation from equations $2, \ldots, K$. Prove that the results in Sects. 3.3.1–3.3.3 carry through unchanged (aside from the indices). Modify RealizeClique and Next accordingly, and find the realization corresponding to the EDM

$$D = \begin{pmatrix} 0 & 21.7560 & 16.8062 & 13.4524 \\ 21.7560 & 0 & 15.1852 & 19.2491 \\ 16.8062 & 15.1852 & 0 & 11.0920 \\ 13.4524 & 19.2491 & 11.0920 & 0 \end{pmatrix},$$

starting with x_1 positioned at the origin. Does the code work? If not, perform the computation manually: what are the values of a_{jK} (for $j < K$) in Eq. (3.7)?

3.4.8 Exercise

In a $(K+1)$-clique which is realizable in $\mathbb{R}^K$, prove that the line containing x_{K+1} (defined in Eq. (3.8)) is always orthogonal to the affine subspace defined by $x_1, \ldots, x_K$. Prove that the result holds even if we order the vertices $1, \ldots, K+1$ of the clique differently.

3.4.9 Exercise

In Sect. 3.3.5, we mentioned that Algorithm 2 describes a class of algorithms, one for every arbitrary set of choices of $x_\ell \in S$. Consider a 5-clique with unit edge weights, and run Algorithm 2 with every possible choice set. How many realizations of the clique can you generate in $\mathbb{R}^4$, assuming vertex 1 is positioned at the origin?

3.4.10 Exercise

Verify empirically whether the "numerically advantageous choice" of x_ℓ in Sect. 3.3.5 (page 27) is really numerically advantageous. Create a test set of at least twenty feasible K-clique instances (for various values of K and of the edge weights), and run Algorithm 2 with the advised choice of x_ℓ. Compare the results with results obtained with x_ℓ chosen differently.

Chapter 4
Discretizability

We remarked in Sect. 2.5.1 that direct methods, which search for solutions in the continuous space $\mathbb{R}^{Kn}$ of all realizations, do not always work all that well. In Chap. 3 we discussed a discrete method for realizing complete graphs in polytime. In this chapter we look at larger classes of graphs for which the search space can be shown to be discrete.

4.1 The volume of simplices

A K-simplex is the realization of a $(K+1)$-clique in $\mathbb{R}^K$. K-simplices are K-dimensional generalizations of segments, triangles, and tetrahedra. They can therefore be associated with measures such as length, area, and volume.

In general, we shall employ the notion of *K-volume* $\Delta_K(S)$ of a closed and bounded[1] set $S \subseteq \mathbb{R}^K$. Intuitively, this is the K-dimensional generalization of the standard three-dimensional volume. Formally, the K-volume of S is the Lebesgue measure associated with the set S [64].

4.1.1 Length and area: K-volume for K ≤ 2

If S is a 1-simplex consisting of two points at distance d_{12}, its 1-volume (or length) is simply $\Delta_1(S) = d_{12}$.

If S is a 2-simplex (a triangle), its 2-volume (or area) can be computed in function of the lengths d_{12}, d_{13}, d_{23} of the sides of the triangles using Heron's formula:

$$\Delta_2(S) = \frac{1}{4}\sqrt{(d_{12}+d_{13}+d_{23})(d_{13}+d_{23}-d_{12})(d_{12}+d_{23}-d_{13})(d_{12}+d_{13}-d_{23})}. \tag{4.1}$$

Notice that 1- and 2-volumes of simplices can be computed using distances alone. This can be generalized to higher values of K, as shown below.

[1] In fact, of a *measurable* set.

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_4

4.1.2 The Cayley-Menger determinant

If S is a K-simplex with edge weights d_{ij} (where $i < j \leq K$), its K-volume is:

$$\Delta_K(S) = \sqrt{\frac{(-1)^{K+1}\mathsf{CM}(d)}{2^K (K!)^2}}, \tag{4.2}$$

where $\mathsf{CM}(d)$ is the *Cayley–Menger determinant* (CM determinant) of S [25, 94], given by:

$$\mathsf{CM}(d) = \begin{vmatrix} 0 & 1 & 1 & \cdots & 1 \\ 1 & 0 & d_{12}^2 & \cdots & d_{1,K+1}^2 \\ 1 & d_{12}^2 & 0 & \cdots & d_{2,K+1}^2 \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & d_{K+1,1}^2 & d_{K+1,2}^2 & \cdots & 0 \end{vmatrix}, \tag{4.3}$$

sometimes also denoted by $\begin{vmatrix} 0 & 1 \\ 1 & (d_{ij}^2) \end{vmatrix}$.

4.2 Realizing quasi-cliques

A *quasi-clique* is a graph with $K+1$ vertices and $\frac{K(K+1)}{2} - 1$ edges for some positive integer K. Quasi-cliques can be defined in other ways: (i) a $(K+1)$-clique with a missing edge; (ii) two K-cliques with $K-1$ common vertices (see Fig. 4.1).

4.2.1 Flat simplices and zero volume

Let $G = (V, E, d)$ be a weighted quasi-clique with $V = \{v_1, \ldots, v_{K+2}\}$. Using definition (i) above, we assume without loss of generality that its missing edge is $\{v_1, v_2\}$. So let $\bar{G}$ be the $(K+2)$-clique on V and extend d to $\bar{G}$ by setting $d_{12} = \delta$.

By the discussion in Sect. 3.3.4, $\bar{G}$ can be realized in $\mathbb{R}^K$ only if it is a flat $(K+1)$-simplex, i.e., its $(K+1)$-volume in $\mathbb{R}^{K+1}$ is zero. Since the volume is proportional to the absolute value of the Cayley–Menger determinant, this implies $\mathsf{CM}(d) = 0$. Because the determinant is a polynomial in

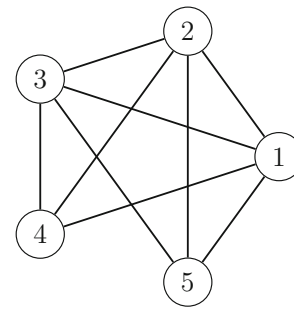


Fig. 4.1 A quasi-clique with $K = 4$, with 5 vertices and 9 edges: the missing edge is $\{4, 5\}$, and there are two 4-cliques having $\{1, 2, 3\}$ in common.

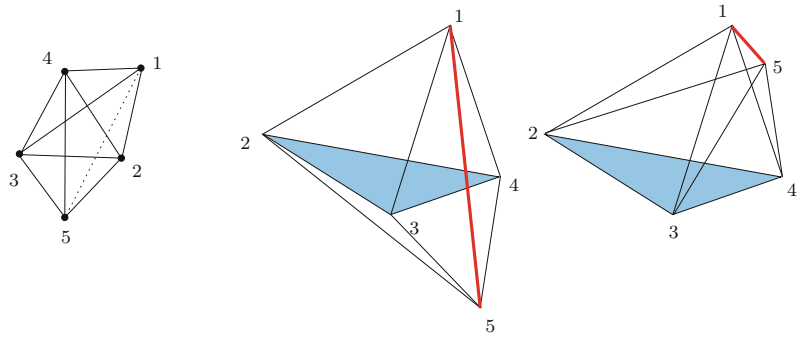


Fig. 4.2 Two realizations of a quasi-clique with $K = 3$ (the missing edge is dotted on the left, and bold on the center and on the right).

d_{ij}, but all d_{ij}'s are given constants apart from $d_{12} = \delta$, $\mathsf{CM}(d)$ turns out to be a quadratic polynomial in δ, say $p(\delta) = 0$: this provides a way to compute up to two values for the missing distance δ (see Fig. 4.2).

Once d_{12} is known, we can simply realize the $(K+2)$-clique $\bar{G}$ in $\mathbb{R}^K$ using Alg. 1.

4.2.1 Example

Let $G = (\{1,2,3,4\}, \{\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\}, d)$ be a weighted quasi-clique on 4 vertices, which we aim to realize in $\mathbb{R}^2$. We get the following Cayley–Menger determinant

$$\begin{vmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & \delta & d_{13}^2 & d_{14}^2 \\ 1 & \delta & 0 & d_{23}^2 & d_{24}^2 \\ 1 & d_{13}^2 & d_{23}^2 & 0 & d_{34}^2 \\ 1 & d_{14}^2 & d_{24}^2 & d_{34}^2 & 0 \end{vmatrix} = 0. \tag{4.4}$$

This yields the quadratic equation: $a\delta^2 - b\delta - c = 0$, where $a = d_{34}^2$, $b = d_{13}^2 d_{23}^2 - d_{14}^2 d_{23}^2 - d_{13}^2 d_{24}^2 + d_{14}^2 d_{24}^2 - d_{13}^2 d_{34}^2 - d_{14}^2 d_{34}^2 - d_{23}^2 d_{34}^2 - d_{24}^2 d_{34}^2 + d_{34}^4$, and $c = d_{13}^2 d_{14}^2 d_{23}^2 - d_{14}^4 d_{23}^2 - d_{14}^2 d_{23}^4 - d_{13}^4 d_{24}^2 + d_{13}^2 d_{14}^2 d_{24}^2 + d_{13}^2 d_{23}^2 d_{24}^2 + d_{14}^2 d_{23}^2 d_{24}^2 - d_{13}^2 d_{24}^4 - d_{13}^2 d_{14}^2 d_{34}^2 + d_{14}^2 d_{23}^2 d_{34}^2 + d_{13}^2 d_{24}^2 d_{34}^2 - d_{23}^2 d_{24}^2 d_{34}^2$. This allows us to compute two values for $\delta = d_{12}$. Geometrically, they correspond to the distance between the positions x_1, x_2 of vertices 1, 2 depending on whether they are on the same side as x_3, or on opposite sides.

See Fig. 4.3 for an example in $\mathbb{R}^3$ using *Mathematica*.

4.3 Realizing K-laterative graphs in $\mathbb{R}^K$

In Chap. 3, we presented an algorithm for realizing complete graphs G based on computing the position of the ith vertex v_i (for $i \leq n = |V|$) in $\mathbb{R}^K$ from the positions of its $K+1$ predecessors $U_i = \{v_{i-K-1}, \ldots, v_{i-1}\}$ with the assumption that $\{v_{i-k}, v_i\} \in E(G)$ for all $k \in \{1, \ldots, K+1\}$. Alg. 1 repeatedly applies the operator $\mathsf{Next}_d^K(U_i)$, where $|U_i| = K+1$. Such a statement obviously implies the existence of a vertex order on V.

If G is complete, *every* vertex order $(v_1, \ldots, v_n)$ on $V(G)$ is such that v_i is adjacent to all its predecessors for each $i > K+1$ (in particular, it is adjacent to at least $K+1$ predecessors). On the

```
G = Graph[{1<−>3,1<−>4,1<−>5,2<−>3,2<−>4,2<−>5,3<−>4,3<−>5,4<−>5},
    EdgeWeight−>{1,1,1,0.7,0.7,0.7,1,1,1}]
A = Normal[WeightedAdjacencyMatrix[G]];
n = Length[A];
v = Array[w, {n, n}];
delta = CayleyMengerVarsSymbolic[A, v];
sol = Select[Flatten[z/.Solve[CayleyMengerDeterminantSymbolic[A,v]==0,delta]], (#>=0)&]
ewl = EdgeWeightList[A];
ewl2 = Map[(ReplacePart[ewl, Position[ewl, 0] −> #]) &, sol];
CG = Map[(CompleteGraph[n,EdgeWeight−>ReplacePart[ewl,Position[ewl,0]−>#]]) &, sol];
X = Map[Take[#, n − 2] &, Map[RealizeClique[#] &, CG], {2}];
Map[GraphPlot3D[G, VertexLabeling −> True, VertexCoordinateRules −> #] &, X]
```

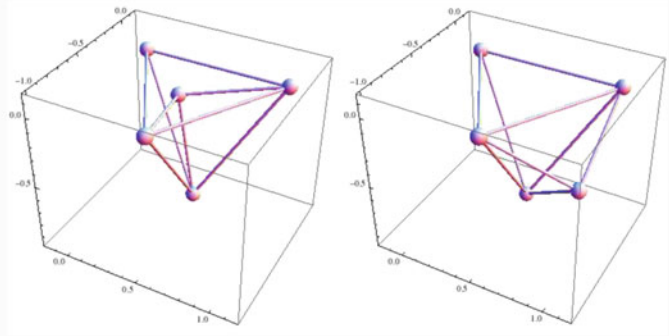


Fig. 4.3 Two possible realizations in $\mathbb{R}^3$ of a quasi-clique on five vertices. The method works as follows: (i) find set `sol` of solutions for `delta`; (ii) replace the missing (zero) distance in the list of edge weights by each value in `sol`; (iii) consider the set `CG` of the two weighted complete graphs on five vertices corresponding to the two edge weight sets; (iv) construct the set `X` of two possible realizations of the quasi-clique (each is a realization of one of the cliques in `CG`).

other hand, Alg. 1 only needs *one such order* in order to be applicable. In this section, we are going to exploit this fact.

4.3.1 Trilateration orders

Some notation: if $<$ is an order on V and $v \in V$, let $\rho(v)$ be the rank of v with respect to $<$, and $\gamma(v)$ be the set of predecessors of v in $<$. Also, let $U(v) = N(v) \cap \gamma(v)$ be the set of adjacent predecessors of v. Finally, let $\theta_K(v) = \{u \in U(v) \mid \exists i \in \{1, \ldots, K\} \ (\rho(u) = \rho(v) - i)\}$ be the set of the K immediate (or contiguous) predecessors of v.

Since for an ordered set V there is a bijection between vertices and their ranks, we sometimes slightly abuse notation and write statements like $v - h$ or $v > K$ to mean $\rho(v) - h$ or $\rho(v) > K$.

Given a graph $G = (V, E)$, a vertex order $<$ on V is a *K-lateration order* if:

1. the first $K + 1$ vertices induce a $(K + 1)$-clique $\mathbf{K}(G)$ in G;
2. any vertex $v > K + 1$ has $|U(v)| > K$.

A graph G is a *K-laterative* if G has a K-lateration order on $V(G)$. A K-laterative graph may have more edges than those that are necessary to guarantee the existence of the K-lateration order, obviously. A K-laterative graph is *minimal* if it has as few edges as possible.

4.3.2 Trilaterative DGP

The TRILATERATIVE DISTANCE GEOMETRY PROBLEM (TDGP) is the subset of DGP instances (G, d, K) for which G is K-laterative, and a K-lateration order on G is given.

Given a TDGP instance (G, d, K), where G is a trilaterative graph weighted by d, $K > 0$, and a vertex order $<$ is given by definition of trilaterative graph, a realization x of G in $\mathbb{R}^K$ can be found (or shown not to exist) in polytime by Alg. 3 (also see Fig. 4.4). Although the choice of U' in Alg. 3

Algorithm 3 $x = \mathsf{RealizeTrilaterative}(G, d, K)$.

```
// realize the initial clique
(x_1, ..., x_{K+1}) = RealizeClique(K(G), d, K)
// realize the next vertex iteratively
for i ∈ {K + 2, ..., n} do
  // find K + 1 immediate adjacent predecessors
  let U' ⊆ U(i) with |U'| = K + 1, and W = {x_j | j ∈ U'}
  // use them to compute the unique position of x_i
  x_i = Next_d^K(W)
  // check that x_i is feasible w.r.t. other distances
  for j ∈ U(i) ∖ U' do
    if ‖x_i − x_j‖ ≠ d_ij then
      // if not, mark infeasible and exit loop
      x_i = ∅
      break
    end if
  end for
  if x_i = ∅ then
    // return infeasible flag
    return ∅
  end if
end for
// return feasible realization
return x
```

appears nondeterministic, it really is not, as x_i will be the same for whatever U', so long as $x_i \neq \emptyset$. If this were not the case, a contradiction could be derived using Cayley–Menger determinants as a measure of a simplex volume.

Alg. 3 is a polytime algorithm. However, we cannot formally state that the TDGP is in **P**. The problem is that the components of the vectors of a realization in $\mathbb{R}^K$ may be irrational numbers even though d maps edges to rational values. Accordingly, all realization algorithms (including Alg. 3) are approximate rather than precise methods. Informally, we say that the TDGP is an easy (or tractable) problem.

Since Alg. 3 uses RealizeClique and Next, the assumption discussed in Sect. 3.3.6 on the rank of A being full holds. Specifically, the output of Next must be a unique point.

4.3.3 The number of solutions of the TDGP

Once the initial clique $\mathbf{K}(G)$ is realized, the rest of Alg. 3 yields exactly one vector for each x_i ($i \in \{K + 2, \ldots, n\}$). If RealizeClique is used deterministically, as specified in Sect. 3.3.5, then Alg. 3 yields either no solution (if (G, d) has no realization in $\mathbb{R}^K$) or exactly one solution.

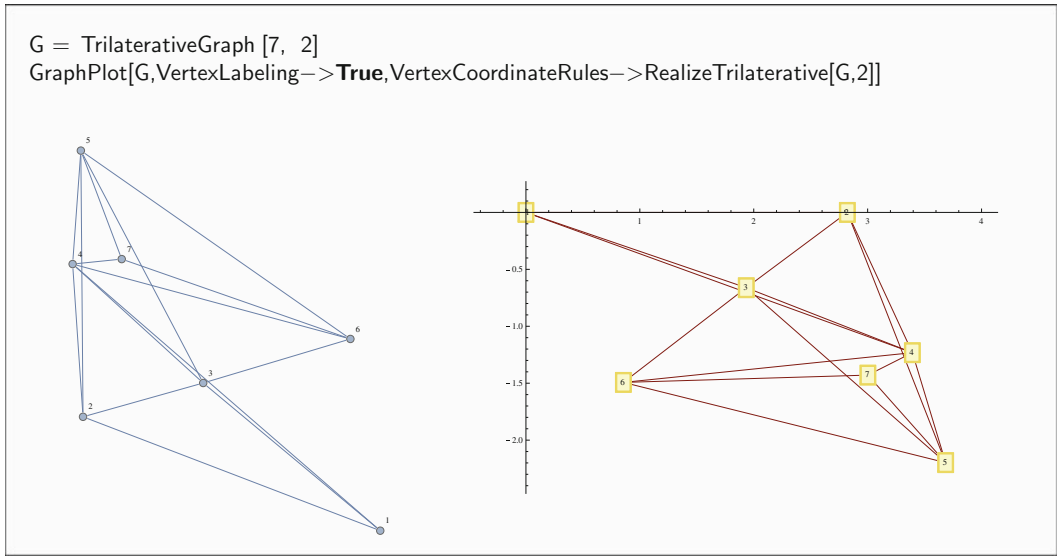


Fig. 4.4 Creating a 2-laterative graph with random edge weights from a random realization (left); reconstructing a congruence (see Appendix A.5.6) of that realization using the 2-lateration order (1, 2, 3, 4, 5, 6, 7) (right).

On the other hand, RealizeClique makes at most 2^K arbitrary choices (exactly 2^K if all Eq. (3.3.5) have distinct real roots), and from each distinct initial clique, we obtain a distinct realization. So the number of solutions of any TDGP instance is bounded by 2^K. In fact, it is exactly 2^{K-h}, where h is the number of times Eq. (3.3.5) has nondistinct real roots.

4.3.4 Sensor network localization

The concept of trilateration is used in the application field known as sensor network localization (SNL) [46], mainly because a realization for the initial $(K+1)$-clique is usually given, and hence, the problem is likely to have a unique solution. Uniqueness is important because the problem arises when finding the geographical positions of a set of mobile sensors which are out and about on the 2D surface of the physical world. If the corresponding DGP had multiple solutions, we would not be able to choose which really corresponds to the actual physical situation.

The SNL PROBLEM (SNLP) is the subset of TDGP instances with $K = 2$. A realization of some 3-clique is usually known because any sensor network is routed to a wired network by means of a set of *anchors*, i.e., wired base stations. In the SNLP, we assume that there are at least three anchors and that a 2-lateration order exists for the network (i.e., the next vertex is adjacent to three predecessors), which must be realized in $\mathbb{R}^2$.

4.4 Realizing $(K-1)$-laterative graphs in $\mathbb{R}^K$

What happens if the given graph G does not have sufficiently many edges to ensure the existence of a K-lateration order? Can we still derive a combinatorial algorithm to realize such graphs? A particularly interesting case is that of realizing $(K-1)$-laterative graphs in $\mathbb{R}^K$.

4.4.1 The shape of protein backbones

The behavior of living organisms can be influenced at the cell level by proteins: these are molecules that bind to specific sites on the cell surface and trigger chemical reactions which modify cellular behavior. It turns out that the function of proteins is intimately related to their 3D structure, or shape. Proteins consist of a backbone and some side chains. Once a structure is known for the backbone, the problem of fitting the side chains can be solved as a separate problem, so we focus on determining the backbone shape.

The protein backbone is a set $V = \{v_1, \ldots, v_n\}$ of atoms with a certain order. We know distances for consecutive pairs $\{v_{i-1}, v_i\}$ (for $i \leq n$), and we know the angles on consecutive triplets (v_{i-2}, v_{i-1}, v_i). This means that we know everything about consecutive triangles, including distances for pairs $\{v_{i-2}, v_i\}$. If the distances for pairs $\{v_{i-3}, v_i\}$ cannot be determined using, e.g., nuclear, magnetic resonance (NMR) experiments [40, 108], it turns out that there is an alternative vertex order for which v_i is adjacent to at least three predecessors [84, 100], indexed in the range $\{1, \ldots, i-1\}$.

Hence, protein backbones can be modeled by weighted 2-laterative graphs (each vertex is adjacent to at least three predecessors), but need to be realized in 3D (see Fig. 4.5). In general, Alg. 3 cannot be used.

4.4.2 Discretizable DGP

The DISCRETIZABLE DISTANCE GEOMETRY PROBLEM (DDGP) [100] is the subset of DGP instances (G, d, K) for which G is $(K-1)$-laterative, and a $(K-1)$-lateration vertex order on G is given. The realization of G is sought in $\mathbb{R}^K$.

The DDGP can be solved by an algorithm which is deceptively similar to Alg. 3. Essentially, we apply Next_d^K to a set W of K, instead of $K+1$, adjacent predecessors. By the definitions of Next for different K given in Sect. 3.2.1–3.3.3, and by the discussion in Sect. 3.3.4, applying Next_d^K to a set of K points generates *up to two* possible positions for the next vertex v.

In Alg. 2, we simply disregarded one of the two positions arbitrarily, since each application of Next was applied to *all* adjacent predecessors. In the present setting, there may be other adjacent predecessors, besides those in W, with distances to v that are feasible with one position but not the other: so we cannot arbitrarily disregard any of them.

Instead, we recursively consider each of the (at most two) feasible positions and try to construct a feasible realization from each. This generates a binary tree: at level v, the tree contains all possible positions for vertex v. The recursion terminates when v is the last vertex in the order. Each tree node at the last level is called a *leaf node*. Every path from the root of the tree to a leaf node encodes a feasible realization of G.

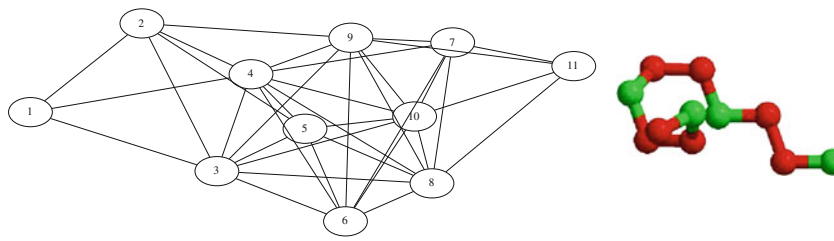


Fig. 4.5 An artificial protein backbone graph [74] (left), and a 3D realization (right).

4.4.3 A Branch-and-Prune algorithm

The algorithm for realizing DDGP instances calls itself recursively over the vertices. It consists of a recursion start (Alg. 4) and a recursive part (Alg. 5).

We store each realization into a solution set X, which is also the returned value of Alg. 4. Let $\mathbf{K}(G)$ be the initial K-clique over $\{1, \ldots, K\}$ given by Part 4.3.1 of the trilateration-order definition on page 34.

Algorithm 4 $X =$ RealizeDDGP(G, d, K)

```
// realize the initial clique
x = (x_1, ..., x_K) = RealizeClique(K(G), d, K − 1)
// initialize an empty solution set
X = ∅
// call the recursive part
RealizeDDGPRecursive(G, d, K, x, K + 1, X)
// return the set of realizations
return X
```

The recursive part, Alg. 5, takes as input: the DDGP instance (G, d, K), the current (partial) realization x, the index i of the next vertex to consider, and the current solution set X, which is actually used as an output of RealizeDDGPRecursive (X may be updated whenever $i = n$). As shown, RealizeDDGP finds all realizations of G. If only one realization is needed, the algorithm can be stopped as soon as X becomes nonempty.

Algorithm 5 RealizeDDGPRecursive(G, d, K, x, i, X)

```
// find K adjacent predecessors
let U' ⊆ U(i) with |U'| = K
W = {x_j | j ∈ U'}
// use them to compute two positions of x_i
S = Next^K_d(W)
// consider each position in turn
for y ∈ S do
  // check feasibility w.r.t. other distances
  for j ∈ U(i) \ U' do
    if ‖y − x_j‖ ≠ d_ij then
      // if not, remove from S
      S = S \ {y}
      break
    end if
  end for
  if y ∈ S then
    // y is feasible
    x_i = y
    if i = n then
      // recursion terminates, store realization
      X = X ∪ {x}
    else
      // recursive step
      RealizeDDGPRecursive(G, d, K, x, i + 1, X)
    end if
  end if
end for
```

The choice of U' in Alg. 5 appears nondeterministic, but should be implemented deterministically. A possible choice may be to choose U' so that the distances from x_i to x_j whenever $j \in U$ have smallest variance (intuitively, this might help the solution of linear system by ensuring well-scaled data).

RealizeDDGP belongs to a class of algorithms called *Branch-and-Prune* (BP) [83]: it *branches* over a binary decision (in this case, the two positions of the ith vertex) and verifies each decision by checking distance feasibility: if not, it *prunes* the corresponding branch in the search tree.

Since Alg. 4–5 use RealizeClique and Next, the assumption discussed in Sect. 3.3.6 on the rank of A being full holds.

4.4.4 Some examples

The BP tree shown in Fig. 4.6 corresponds to the graph of Fig. 4.5. Since there are 4 paths from the root (node 1) to the leaf nodes (11, 28, 36, 53), $|X| = 4$. Because RealizeDDGP explores the search tree depth-first, the nodes in the leftmost path are numbered according to the levels they are at. By Alg. 5, if a nonleaf node at level $i-1$ has fewer than two subnodes, there must be an edge $\{j, i\}$ where $j < i$ and $j \notin U'$. The input graph G (Fig. 4.5) has the following adjacent predecessor sets:

$$
\begin{aligned}
U(1) &= \varnothing \\
U(2) &= \{1\} \\
U(3) &= \{1, 2\} \\
U(4) &= \{1, 2, 3\} \\
U(5) &= \{2, 3, 4\} \\
U(6) &= \{3, 4, 5\}
\end{aligned}
$$

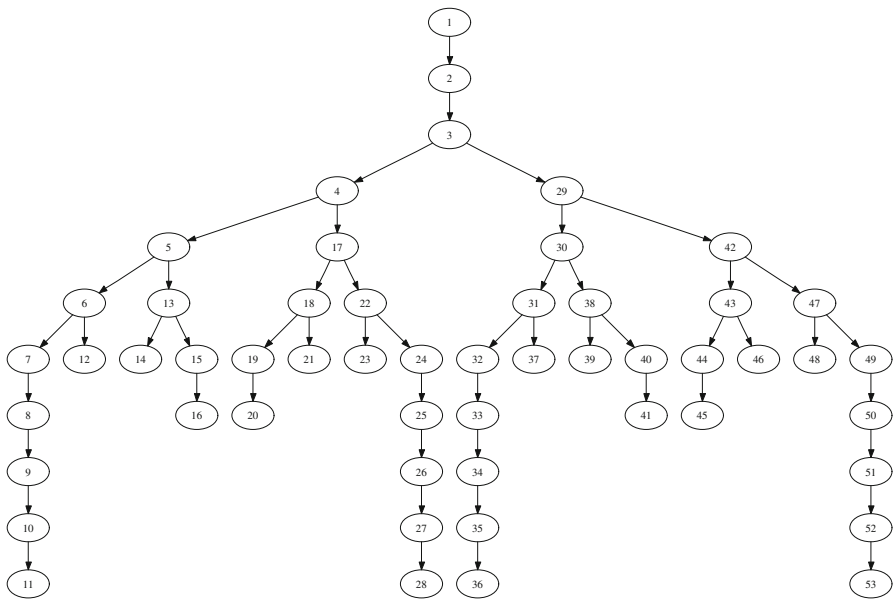


Fig. 4.6 A typical BP tree.

$$
\begin{aligned}
U(7) &= \{4, 5, 6\} \\
U(8) &= \{3, 4, 5, 6, 7\} \\
U(9) &= \{2, 3, 4, 5, 6, 7, 8\} \\
U(10) &= \{3, 4, 5, 6, 7, 8, 9\} \\
U(11) &= \{7, 8, 9, 10\}.
\end{aligned}
$$

Accordingly, we should expect a complete binary tree from level 3 to level 7, followed by some pruning at lower levels, which is consistent with Fig. 4.6.

In Fig. 4.7, we use *Mathematica* to build and realize a DDGP graph G with $n = |V| = 6$, $K = 2$, the 1-lateration order $1, \ldots n$, and one pruning edge. In DDGP instances, we partition the set E of edges into *discretization edges* E_D and *pruning edges* E_P. An edge $\{u, v\} \in E$ is in E_D if $u \in U'$ (see Alg. 5), and in E_P otherwise. Any DDGP instance graph G must have at least $\frac{1}{2}K(K-1) + K(n-K)$ discretization edges: $\frac{1}{2}K(K-1)$ edges in the initial clique $\mathbf{K}(G)$, and K edges incident to each of the $n - K$ vertices beyond $\mathbf{K}(G)$. Obviously, the more pruning edges G has, the closer it gets to being a trilaterative graph. If $|E_D| + |E_P| = \frac{1}{2}n(n-1)$, G is a complete graph. We compute and show the set of all realizations of G in $\mathbb{R}^2$ up to rotations and translations.

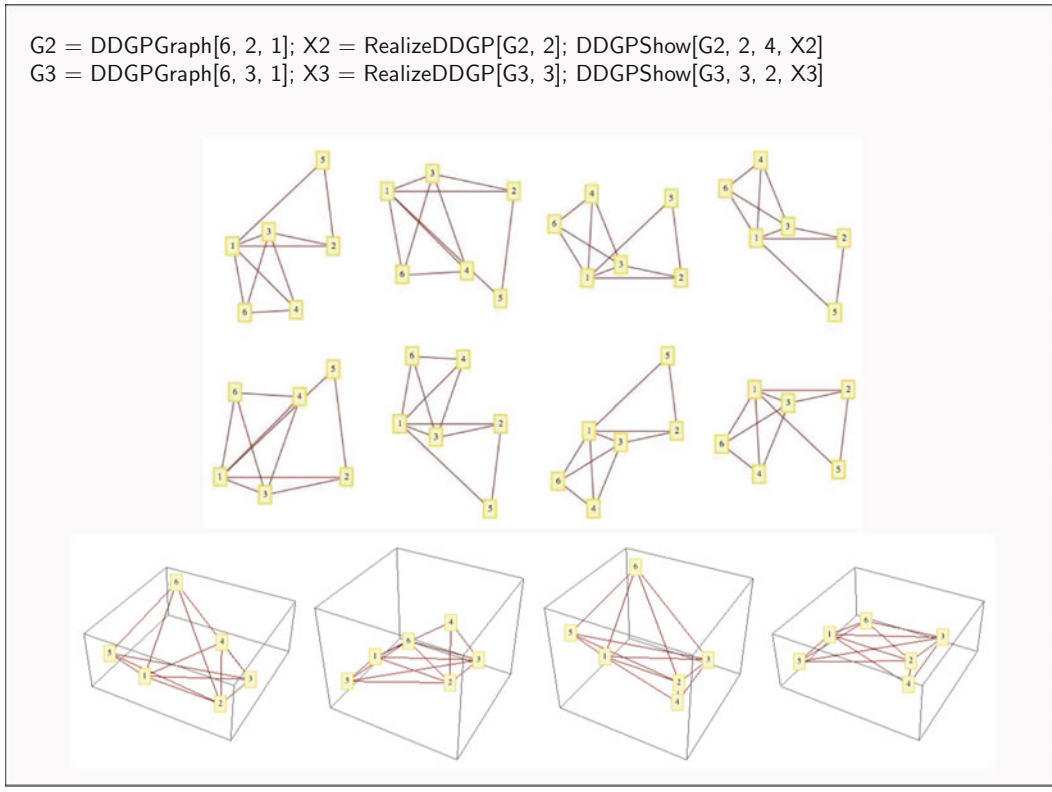


Fig. 4.7 Constructing a DDGP instance graph and displaying all its realizations up to rotations and translations. `DDGPGraph` with arguments n, K, m creates a DDGP graph with n vertices in K dimensions with m pruning edges chosen at random; `RealizeDDGP` with arguments G, K finds all realizations (up to translations and rotations) of a DDGP instance graph G in $\mathbb{R}^K$; `DDGPShow` with arguments G, K, p, X shows p-column display of the set of realizations X for graph G in $\mathbb{R}^K$.

4.4.5 Finding all realizations

All realizations found by RealizeDDGP are feasible, essentially by the correctness of Next. The fact that RealizeDDGP finds *all* realizations of G (up to translations and rotations) can be shown by contradiction.

Suppose there is a feasible realization z of G and that $\vartheta z \notin X$ for any rotation or translation ϑ. Let i be the smallest index such that $x_i \neq z_i$ for every $x \in X$ (we assume z is translated and rotated so that its alignment with realizations in X is as good as possible, meaning that i is made to be as large as possible for at least one $x \in X$). Because RealizeClique is correct, i cannot be $\leq K$. So $i > K$. But then that means that there is a feasible position for i that is not in the set S in Alg. 5, which means that Eq. 3.11 is a quadratic equation with more than two distinct real roots. This is a contradiction, which shows that if $z \notin X$, then z cannot be a feasible realization of G: hence, X contains all realizations of G up to rotations and translation, as claimed.

If we consider the deterministic version of RealizeClique, every vertex can be recursively placed in at most two positions: so at termination, we have $|X| \leq 2^{n-K}$. If we consider a nondeterministic version, where all possible choices of $x_\ell \in S$ (Alg. 2) are followed through, then $|X| \leq 2^{n-1}$ (there is a unique position—the origin—only for the first vertex). In the following, we shall assume that algorithms are deterministic, so that $|X| \leq 2^{n-K}$.

4.4.6 Worst-case complexity

The worst-case complexity is given by graphs which yield $|X| = 2^{n-K}$: this implies an exponential running time for RealizeDDGP. Notice $|X|$ decreases each time S is replaced by $S \smallsetminus \{y\}$ in Alg. 5. If G is a minimally trilaterative graph, then $U' = U(i)$ in Alg. 5, and the statement removing y from S can never be reached. So RealizeDDGP takes longest on the class of minimally trilaterative graphs.

4.4.7 Best-case complexity

Since a complete graph is also a $(K-1)$-laterative graph, if it can be realized in $\mathbb{R}^K$, then $|S| = 1$ at each recursive call of Alg. 5, which will yield a total of $n - K$ calls, a call trace equivalent to that of Alg. 1 and hence a polynomial (in fact, linear if K is fixed [41]) running time.

Moreover, a K-laterative graph is also a $(K-1)$-laterative graph, so, similarly, we get $|S| = 1$ at each recursive call, which implies a running time equivalence with Alg. 3, which is a polytime algorithm.

Other polynomial cases are discussed in [85]; interestingly, all protein instances we tested belong to a polynomial case.

4.5 Exercises

4.5.1 Exercise

Given two positive integers n, m with $n < m$ and a set $S \subseteq \mathbb{R}^m$ having affine hull dimension (see Appendix A.4.3) equal to n, what is the m-volume of S in $\mathbb{R}^m$? What can you say about the n-volume of S in $\mathbb{R}^n$, the n-volume of S in $\mathbb{R}^m$, and the m-volume of S in $\mathbb{R}^n$?

4.5.2 Exercise
Compute (using, e.g., *Mathematica*) the coefficients of the K-volume formula Eq. (4.2) for some K-simplices with unit edge weights, then square them and take their inverses, so they become integers. Verify the three-term integer sequence for $K \in \{1, 2, 3\}$ on Sloane's Online Encyclopedia of Integer Sequences.[2] What result do you get? Now, extend your sequence to $K \in \{1, 2, \ldots, 10\}$: do you still get the same result? [Hint: the *Mathematica* code necessary for this computation is a (longish) one-liner involving the built-in functions `Map`, `Table`, `Range` and `IdentityMatrix` as well as ordinary arithmetic operators].

4.5.3 Exercise
Compute (using, e.g., *Mathematica*) the possible volumes of the unit edge weight quasi-cliques on 4, 5, 6, 7, 8, 9, 10 vertices in dimensions 2, 3, 4, 5, 6, 7, 8 respectively.

4.5.4 Exercise
Prove that, in Alg. 3, x_i is independent of the choice of U' (as long as $|U'| = K + 1$).

4.5.5 Exercise
Prove that Alg. 3 is a polytime algorithm.

4.5.6 Exercise
Let CDGP be the subset of DGP instances that are defined on complete graphs. Prove that it is a subproblem of the TDGP. Is the CDGP a tractable problem?

4.5.7 Exercise
Prove that a complete graph is K-laterative for any K less than or equal to the number of vertices.

4.5.8 Exercise
Prove that a K-laterative graph is $(K - h)$-laterative for any K and $h < K$.

4.5.9 Exercise
Verify empirically whether the advice for a "good choice of U'" in Sect. 4.4.3 (page 38) is actually good or not. Create a test set of at least twenty feasible DDGP instances, generated either randomly or from real/realistic data, and solve them with the advised choice of U'. Compare the results with results obtained with U' chosen randomly.

[2] https://oeis.org.

Chapter 5
Molecular distance geometry problems

The MOLECULAR DISTANCE GEOMETRY PROBLEM (MDGP) [86] is the subclass of DGP instances with $K = 3$. Since the DGP is **NP**-hard for each K [107], the MDGP is also **NP**-hard.

The DISCRETIZABLE MDGP (DMDGP) [79] is the subclass of MDGP instances where G is 2-laterative and, for each vertex v_i with $i > 3$, the set $U(v_i)$ of adjacent predecessors to v_i contains the three immediate predecessors $\{v_{i-1}, v_{i-2}, v_{i-3}\}$. The DMDGP is shown to be **NP**-hard by a modification [79] of Saxe's proof (Sect. 2.4.2).

5.1 Contiguous $(K-1)$-lateration orders

Let $G = (V, E)$ be a graph, K be a positive integer, and $<$ be a $(K-1)$-lateration order; $<$ is *contiguous* if, for any $v \in V$ with $v > K$, the set $U(v)$ of adjacent predecessors of v contains the set $\theta_K(v)$ of the K immediate (contiguous) predecessors of v.

The assumption $\theta_K(v) \subseteq U(v)$ is motivated by the protein backbone model given in Sect. 4.4.1, where $K = 3$ and all distances between atoms v and $v-1$, $v-2$, $v-3$ are assumed to be known. Although this model is not perfect, it can serve as a first approximation. The basic DG theory given so far can be adapted to more precise models [84].

5.1.1 The generalized DMDGP

The generalized DMDGP (^{K}DMDGP) is the subclass of DGP instances (G, d, K) for which G has a (given) contiguous $(K-1)$-lateration order.[1]

It was shown in [85] that the ^{K}DMDGP is **NP**-hard. Since the ^{K}DMDGP is a subset of instances of the DDGP, the latter is also **NP**-hard by inclusion (in other words, if a problem has a hard subproblem, then it must be hard itself).

Contiguous $(K-1)$-lateration orders are also called ^{K}DMDGP orders, and if (G, d, K) is a ^{K}DMDGP instance, then G is also called a ^{K}DMDGP graph.

[1] In some of our past papers, e.g., [76, 79, 83, 100], we defined DDGP, DMDGP, and ^{K}DMDGP with one further condition, i.e. d satisfies strict simplex inequalities. This is equivalent to the condition on the rank of A in Sect. 3.3.6; its purpose is to eliminate those edge weight functions that prevent the application of the methods given in Ch. 3.

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_5

5.1.2 Realizing ^{K}DMDGP graphs

The good news is that there is a BP algorithm for realizing ^{K}DMDGP graphs, and that it is almost exactly the same as RealizeDDGP. We define the RealizeDMDGPRecursive algorithm to be like RealizeDDGPRecursive, with the first line replaced by:

$$\text{let } U = \{i - K, \ldots, i - 1\},$$

and of course the recursive step replaced by

$$\mathsf{RealizeDMDGPRecursive}(G, d, K, x, i + 1, X).$$

Then, a ^{K}DMDGP graph G can be realized in $\mathbb{R}^K$ by using Alg. 6, which is the same as Alg. 4 except that the recursive call is made to RealizeDMDGPRecursive. As for Alg. 4, Alg. 6 also requires the rank of A to be full, as discussed in Sect. 3.3.6. Also, the results in Sects. 4.4.5, 4.4.6, 4.4.7 all hold for ^{K}DMDGP graphs. However, ^{K}DMDGP graphs have more structure than trilaterative or DDGP graphs. We shall exploit this structure in the rest of this chapter (Fig. 5.1).

Algorithm 6 $X = \mathsf{RealizeDMDGP}(G, d, K)$

$x = (x_1, \ldots, x_K) = \mathsf{RealizeClique}(\mathbf{K}(G), d, K - 1)$
$X = \varnothing$
$\mathsf{RealizeDMDGPRecursive}(G, d, K, x, K + 1, X)$
return X

5.1.3 Feasibility of Next

Although the BP algorithm for the ^{K}DMDGP is almost identical to the one for the DDGP, there is a striking difference in their behaviors as concerns the detection of infeasibility along a given branch. In the case of DDGP (RealizeDDGPRecursive, Alg. 5), infeasibility of a branch can be detected either when testing whether $\|y - x_j\| \neq d_{ij}$ in the loop over $y \in S$, or when computing $S = \mathsf{Next}_d^K(W)$ before the loop. By contrast, in the case of ^{K}DMDGP, infeasibility of a branch can only be detected when testing whether $\|y - x_j\| \neq d_{ij}$. To see this, we show that the system solved by Next is always feasible.

First, we prove by induction that for each vertex $v \in V$ with $v > K$, the set $\bar{\theta}(v) = \theta_K(v) \cup \{v\}$ induces a $(K + 1)$-clique in G. If $v = K + 1$, $\theta_K(v)$ induces the initial clique $\mathbf{K}(G)$, so by definition of trilaterative order the claim holds. We assume that the claim holds for all vertices up to $v - 1$ (with $v > K + 1$) and shows that it holds for v. Since it holds for $v - 1$, $\bar{\theta}(v - 1)$ induces a $(K + 1)$-clique in G. Again by definition of trilaterative order, v is adjacent to its K immediate predecessors $\theta_K(v)$: but since $\bar{\theta}(v - 1)$ is a $(K + 1)$ clique, and $\bar{\theta}(v - 1) = \theta_K(v) \cup \{v - K - 1\}$, then $\theta_K(v)$ induces a K-clique in G, which means that $\bar{\theta}(v)$ is a $(K + 1)$-clique, as claimed.

Thus, for each $v > K$, the given edge weight function d lists all of side lengths of the K-simplex induced by $\bar{\theta}(v)$. A generalization to $\mathbb{R}^K$ of the triangular inequality, (see Appendix A.6) which states that no side of a triangle can exceed in length the sum of the other two (Eq. (3)), is that the value under the square root in the K-volume formula (Eq. (4.2)) is always nonnegative. All YES instances are such that the K-volume is a real number (rather than a complex one). Since G is assumed to be a YES instance, none of its weighted cliques can yield a NO instance. Thus Next must yield at least one solution.

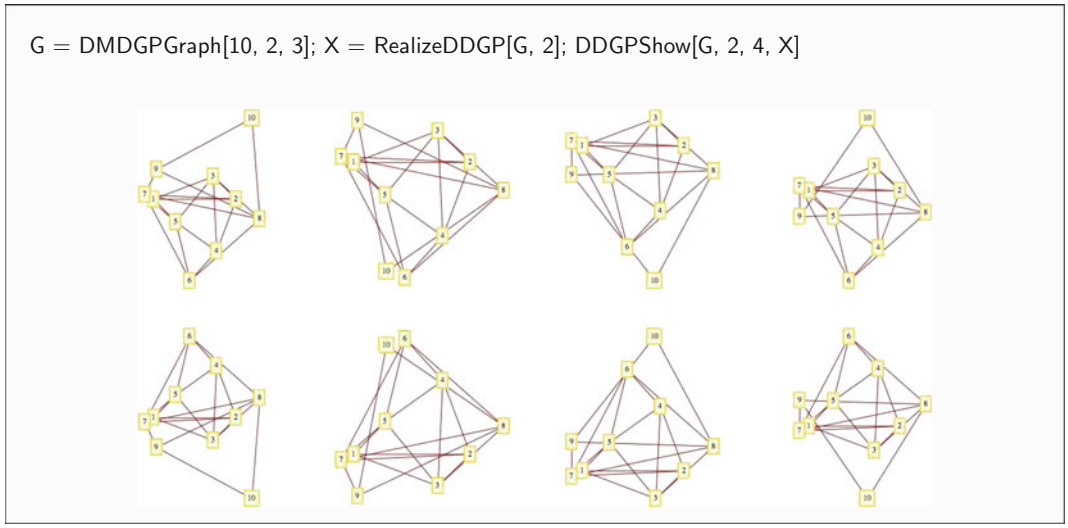


Fig. 5.1 Our *Mathematica* implementation of Branch-and-Prune automatically adapts to whether the input graph G is DDGP or ^{K}DMDGP.

5.2 Partial reflection symmetry

The simplest possible ^{K}DMDGP graph is simply a chain of triangles in $\mathbb{R}^2$, as shown in Fig. 5.2. The chain of three triangles in Fig. 5.2 shows a plane realization of a weighted graph (G, d) where $V(G) = \{1, 2, 3, 4, 5\}$,

$$\begin{aligned} d_{12} &= \sqrt{2} \quad & d_{13} &= 2 \\ d_{23} &= \sqrt{2} \quad & d_{24} &= 2 \\ d_{34} &= \sqrt{2} \quad & d_{35} &= 2.1, \end{aligned}$$

and d_{45} is obtained by the method of Sect. 4.2.

What is the set X of all realizations of G in $\mathbb{R}^2$, ignoring translations and rotations? Since G is a minimal contiguous 1-laterative graph, we can compute X by means of the RealizeDMDGP algorithm. All realizations in X are shown in Fig. 5.3. Let x^1, x^2, x^3, x^4 be the realizations in the first row, and x^5, x^6, x^7, x^8 be those in the second row.

Notice that no realization in $X = \{x^1, \ldots, x^8\}$ is a rotation of another: for example, the rotation of x^1 by 90 degrees clockwise just looks like x^5, but the edge $\{3, 5\}$ in the rotated x^1 is superposed to the edge $\{1, 3\}$ in x^5; moreover, $d_{35} = 2.1 \neq 2 = d_{13}$. If we consider reflections, the situation changes: rotating x^p by 90 degrees counterclockwise and then reflecting it through the vertical axis yields x^{p+4}, for all $p \leq 4$. This is consistent with the fact that X was only assumed to contain distinct realizations up to rotations and translations.

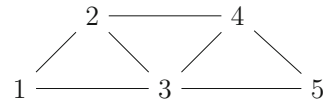


Fig. 5.2 A chain of triangles.

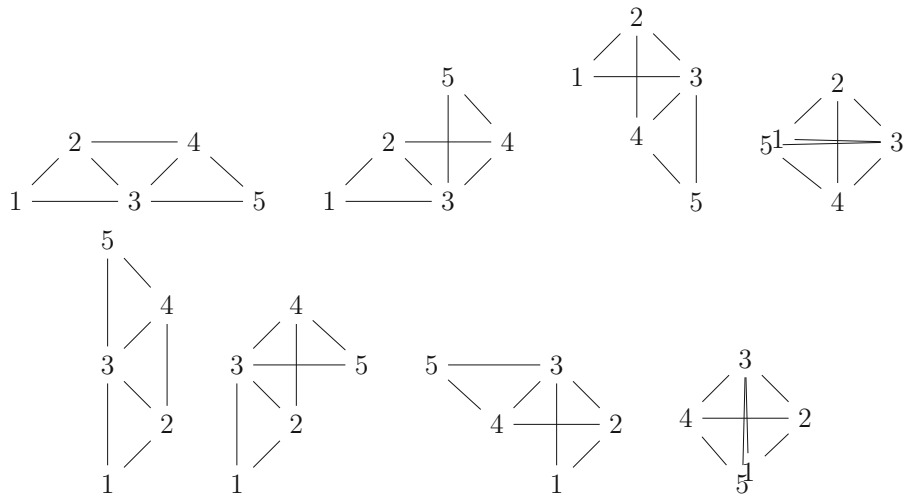


Fig. 5.3 All realizations (modulo rotations and translations) of the graph G shown in Fig. 5.2.

What if we consider partial reflections? That is, if we keep an initial set of points fixed, say from 1 to $v-1$, then we start reflecting all subsequent points with respect to the line defined by x_{v-1}, x_{v-2}. Let this operation be given by g_v. Looking at Fig. 5.3,

$$\begin{array}{llll} g_5(x^1) = x^2 & g_5(x^3) = x^4 & g_5(x^5) = x^6 & g_5(x^7) = x^8 \\ g_4(x^1) = x^3 & g_4(x^2) = x^4 & g_4(x^5) = x^7 & g_4(x^6) = x^8 \\ g_3(x^1) = x^5 & g_3(x^2) = x^6 & g_3(x^3) = x^7 & g_3(x^4) = x^8. \end{array}$$

So how can we map, say, x^1 to x^4? Simple: through g_5 first, and then the result through g_4. By the above table, $g_4(g_5(x^1)) = g_4(x^2) = x^4$. It is easy to show that for any $i, j \leq 8$, there is always a composition of g_3, g_4, g_5 taking x^i to x^j. In group theoretical terminology, the action of the group $\mathscr{G} = \langle g_3, g_4, g_5 \rangle$ on X is transitive.

5.2.1 Isometry and congruence

Let G be a weighted graph and ϕ be a function from $\mathbb{R}^K \to \mathbb{R}^K$. We call ϕ an *isometry* of G if, for any realization x of G in $\mathbb{R}^K$, $\phi(x)$ is also a realization of G. In other words, isometries preserve the distances corresponding to the edges of G. If ϕ preserves all pairwise distances between realized vertices of G, then ϕ is a *congruence*. Obviously, all congruences are also, by definition, isometries.

For example, translations, rotations, and reflections in $\mathbb{R}^K$ (see Appendix A.5.6) are congruences, since they preserve all pairwise distances when applied to any set of points in $\mathbb{R}^K$. Most partial reflections are isometries but not congruences: the distance between vertices 2 and 5 in the top left realization in Fig. 5.3 is evidently different from the corresponding distance in the second realization on the top row in Fig. 5.3.

5.2.2 The discretization group

We generalize the above discussion to general K. Consider a ^{K}DMDGP graph $G = (V, E)$ with as few edges as possible, and edge weight function d. The fact that E has minimum cardinality, while G still has a contiguous $(K-1)$-lateration order, implies that G is just a chain of $(K+1)$-cliques, with each consecutive pair sharing a K-clique, see Fig. 5.4. In fact, the edges in E are all discretization edges, according to the edge classification introduced at the end of Sect. 4.4.4.

Let X be the set of all realizations of G in $\mathbb{R}^K$ up to rotations and translations. For any $x \in X$ and $v \in V$ with $v > K$, we define the *partial reflection* operator $g_v : \mathbb{R}^{Kn} \to \mathbb{R}^{Kn}$ given by:

$$g_v(x_1, \ldots, x_n) = (x_1, \ldots, x_{v-1}, R_x^v(x_v), \ldots, R_x^v(x_n)), \tag{5.1}$$

where $R_x^v : \mathbb{R}^K \to \mathbb{R}^K$ is a reflection operator that maps each point in $\mathbb{R}^K$ to its reflection with respect to the hyperplane defined by $\{x_{v-1}, \ldots, x_{v-K}\}$.

We remark that for the R_x^v to be reflections in $\mathbb{R}^K$, we need the edge weight function d to be such that the assumption on the rank of A being full holds (Sect. 3.3.6). This assumption will stand for the rest of this chapter.

Obviously, if you apply any reflection operator twice, you reflect points back to their original positions. Since g_v is simply a composition of $v-1$ identities and $n-v+1$ reflections, it enjoys the same property: thus, every g_v is idempotent.

Also, for $x \in X$ and $K < u < v$, we have:

$$\begin{aligned}
g_u g_v(x) &= g_u(x_1, \ldots, x_{v-1}, R_x^v(x_v), \ldots, R_x^v(x_n)) \\
&= (x_1 \ldots, x_{u-1}, R_{g_v(x)}^u(x_u), \ldots, R_{g_v(x)}^u R_x^v(x_v), \ldots, R_{g_v(x)}^u R_x^v(x_n)) \\
&= (x_1 \ldots, x_{u-1}, R_x^u(x_u), \ldots, R_{g_u(x)}^v R_x^u(x_v), \ldots, R_{g_u(x)}^v R_x^u(x_n)) \\
&= g_v(x_1, \ldots, x_{u-1}, R_x^u(x_u), \ldots, R_x^u(x_n)) \\
&= g_v g_u(x).
\end{aligned}$$

To infer the third line from the second, we used the fact that $R_{g_v(x)}^u R_x^v(x_w) = R_{g_u(x)}^v R_x^u(x_w)$ for each $w \geq v$. A proof of this is given in [85, Lemma 1]; we only provide here a graphical sketch of the proof in 2D (Fig. 5.5), where, to simplify notation, we let $R_x^v = \rho^y$, where y is the normal to the line through x_{v-1}, x_{v-2}.

Let $\mathscr{G}_D(G) = \langle g_v \mid v > K \rangle$, i.e., the symmetry group generated by the action of all g_v's on the realizations of G, be the *discretization group* of G. Since its elements commute, as shown above, $\mathscr{G}_D$ is an abelian group. Moreover, all of the elements are idempotent. Thus, $\mathscr{G}_D$ is isomorphic to C_2^{n-K}, and the Cartesian product of $n-K$ copies of the cyclic group C_2.

Notice that if $v \in V$ such that $v > K$ and H is the subgraph of G induced by $\{1, \ldots, v\}$, then the discretization group of H is isomorphic to a subgroup of the discretization group of G: this follows

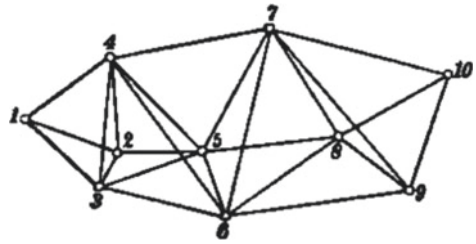


Fig. 5.4 A chain of 4-cliques in $\mathbb{R}^3$ [58].

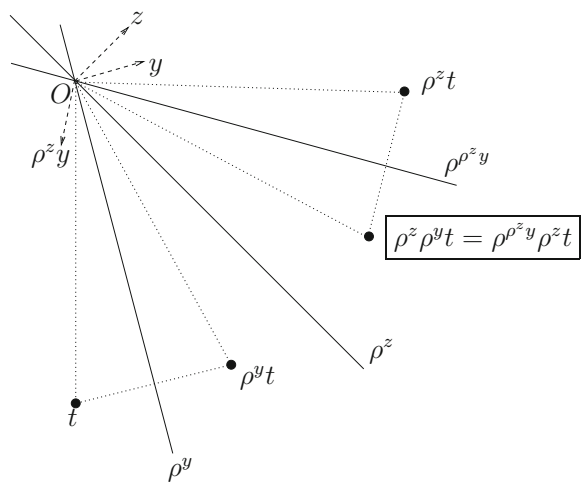


Fig. 5.5 Reflecting through ρ^y first and ρ^z second is equivalent to reflecting through ρ^z first and (the reflection of ρ^y through ρ^z) second.

by mapping $g_u \in \mathscr{G}_D(H)$ to $\bar{g}_u \in \mathscr{G}_D(G)$ (for each $u \leq v$) such that for $\bar{x}$ a realization of G and $x = (\bar{x}_1, \ldots, \bar{x}_v)$ the induced realization of H,

$$\bar{g}_u(\bar{x}) = (g_u(x), R^u_x(x_{v+1}), \ldots, R^u_x(x_n)).$$

The point of this remark is to emphasize that we can apply partial reflections in $\mathscr{G}_D(H)$ to realizations of G.

The most interesting feature of the discretization group is that its action on X is transitive: given a single realization $x^1 \in X$, for every realization $x \in X$ there is $g \in \mathscr{G}_D(G)$ such that $g(x^1) = x$. In other words, $\mathscr{G}_D(G)x^1 = X$. This can be shown by induction on n.

The induction starts because when $n = K + 1$ we have $X = \{x^1, x^2\}$, where x^2 is a reflection of x^1 with respect to $\{x_1, \ldots, x_K\}$ (see Sect. 3.3.4): so $g_{K+1}(x^2) = x^1$ and vice versa.

Now let $H = G[\{1, \ldots, n-1\}]$, $X_H = \mathsf{RealizeDMDGP}(H, d, K)$, and assume that the action of $\mathscr{G}_D(H)$ on X_H is transitive. Pick any two realizations x^1, x^2 of G in X, and consider the partial realizations consisting of their first $n-1$ vectors: let $x^1 = (\bar{x}^1, y)$ and $x^2 = (\bar{x}^2, z)$, where $\bar{x}^1, \bar{x}^2 \in \mathbb{R}^{K(n-1)}$ and $y, z \in \mathbb{R}^K$. Note that RealizeDMDGP computes X by extending partial realizations in X_H using Next, so $\bar{x}^1, \bar{x}^2$ must be in X_H by definition of Next.

By the induction hypothesis, since $\bar{x}^1, \bar{x}^2 \in X_H$, and $\mathscr{G}_D(H)$ is assumed to be transitive, there is $\bar{g} \in \mathscr{G}_D(H)$ such that $\bar{g}(\bar{x}^2) = \bar{x}^1$. Since every hyperplane W, given e.g. as K points in $\mathbb{R}^K$, splits $\mathbb{R}^K$ into two (open) half-spaces, which we can label as positive and negative, we can assign $+, -$ labels to both y and z, according to which side of the reflecting hyperplane W they are on (this labelling is known as an *orientation*). Since a partial reflection g_v will swap the orientation of every realization vector from the v-th onwards, either $\bar{g}$ or $g_n\bar{g}$ maps x^2 to x^1 and vice versa, which concludes the proof.

Limited to graphs with no pruning edges, this suggests an alternative implementation of RealizeDMDGP: whenever $S = \mathsf{Next}^K_d(W)$ is computed (see Alg. 5), arbitrarily dismiss one of the two elements, but record g_v at each level. At level n, when a realization x is found, simply compute X by applying all possible compositions of the group elements $g_{K+1}, \ldots, g_n$ to x. In this setting, x can be found in polytime, since there is no branching, but it still takes exponential time to list all possible subsets of the generators of $\mathscr{G}_D(G)$, see Fig. 5.6.

Lastly, remark that, since $\mathscr{G}_D(G)x = X$ for all $x \in X$, then $\mathscr{G}_D(G)X = \{gx \mid g \in \mathscr{G}_D(G) \wedge x \in X\} = X$: so X is *invariant* with respect to $\mathscr{G}_D(G)$, i.e., X does not change, as a set, under the action of the discretization group.

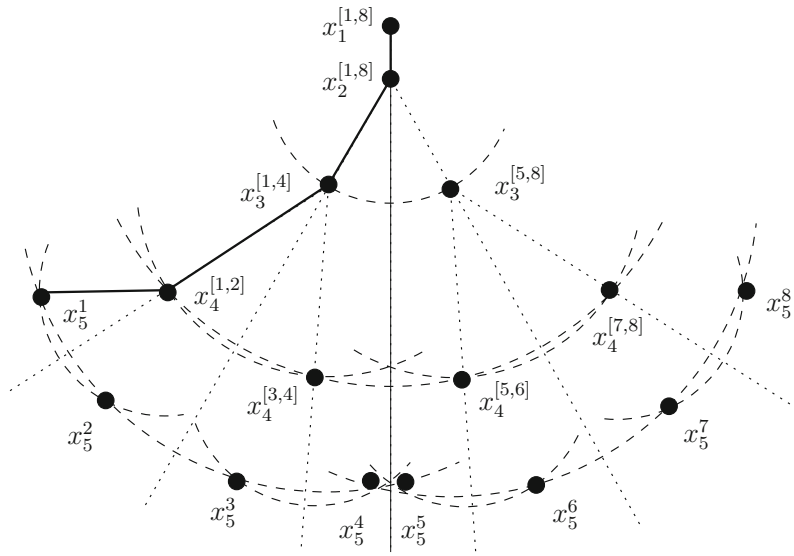


Fig. 5.6 Action of the discretization group $\mathscr{G}_D(G) = \langle g_3, g_4, g_5 \rangle$ on a realization $x^1 = (x_1^1, \ldots, x_5^1)$ is transitive and commutative: we have $g_5(x^1) = x^2, g_4(x^1) = x^4, g_5g_4(x^1) = g_4g_5(x^1) = x^3, g_3(x^1) = x^8, g_5g_3(x^1) = g_3g_5(x^1) = x^7$, $g_4g_3(x^1) = g_3g_4(x^1) = x^5$, $g_5g_4g_3(x^1) = g_5g_3g_4(x^1) = g_4g_5g_3(x^1) = g_4g_3g_5(x^1) = g_3g_5g_4(x^1) = g_3g_4g_5(x^1) = x^6$. The notation $x_j^{[i,\ell]}$ indicates that the neighboring dot represents the position of the (equal) point vectors $x_j^i, x_j^{i+1}, \ldots, x_j^\ell \in \mathbb{R}^K$, for $j \in \{1, \ldots, 5\}$ and $x^i \in X$ for all $i \leq 8$.

5.2.3 The pruning group

The results of Sect. 5.2.2 rely on G being a chain of $(K+1)$-cliques. But our motivation for studying ^{K}DMDGP graphs is that they provide a good model for protein backbones. The issue is that protein graphs have more edges besides those necessary for the existence of their contiguous trilateration order: in other words, their pruning edge set is nonempty. These edges, and their associated distance values, give RealizeDMDGP the information that is necessary in order to prune realizations that are not consistent with the measured distances; therefore, it would be unwise to disregard them. Below, we study how partial reflection symmetries behave in the presence of pruning edges.

Let $G = (V, E)$ be a weighted ^{K}DMDGP graph and d an edge weight function. We partition E into discretization edges E_D and pruning edges E_P. The graph $G[E_D]$ induced by the discretization edges is a minimal ^{K}DMDGP graph, and hence, by the results in Sect. 5.2.2, its realization set $\bar{X}$ is invariant with respect to its discretization group. We define $\mathscr{G}_D(G) = \mathscr{G}_D(G[E_D])$.

Figure 5.7 shows the partial planar realizations computed by RealizeDMDGP in the first five BP tree levels of an instance where $K = 2$. The positions of vertices at level j arise as the intersection of two circles (dashed arcs) centered at vertex levels $j-1$ and $j-2$, with respective radii $d_{j-1,j}, d_{j-2,j}$. The partial reflection symmetry at each level is defined across the dotted lines through the circle centers at the preceding two levels. The solid concentric circles at levels $1, \ldots, 5$ are all centered at $x_1^{[1,8]}$, and their radii are the distance values on pruning edges which are compatible with the given discretization edges. The crucial observation is the symmetric placement of the points x_j^i on the solid circles, for $i \in \{1, \ldots, 8\}$ and $j \in \{3, 4, 5\}$. More precisely, pairs like $x_4^{[1,2]}, x_4^{[7,8]}$ or $x_4^{[3,4]}, x_4^{[5,6]}$, which are reflections along the line defined by x_1, x_2, are at the same distance from $x_1^{[1,8]}$. So, if $X \neq \varnothing$ and we had a pruning edge $\{1, 4\}$, the distance d_{14} would either prune $x_4^{[1,2]}, x_4^{[7,8]}$ or $x_4^{[3,4]}, x_4^{[5,6]}$; it could

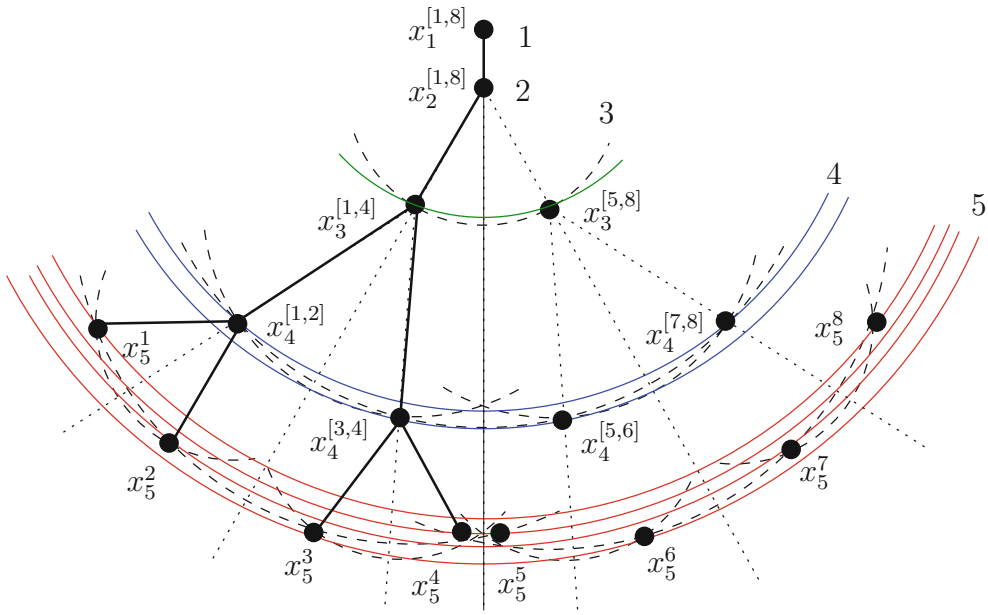


Fig. 5.7 Pruning edge $\{1, 4\}$ prunes either $x_4^{[1,2]}$, $x_4^{[7,8]}$ or $x_4^{[3,4]}$, $x_4^{[5,6]}$. The notation $x_j^{[i,\ell]}$ indicates that the neighboring dot represents the position of the (equal) point vectors $x_j^i, x_j^{i+1}, \ldots, x_j^\ell \in \mathbb{R}^K$, for $j \in \{1, \ldots, 5\}$ and $x^i \in X$ for all $i \leq 8$.

never prune different subsets of these four points. Similarly, a pruning edge $\{1, 5\}$ would only save two symmetric point pairs in the set $\{x_5^1, \ldots, x_5^8\}$, i.e., $\{x_5^1, x_5^8\}$, $\{x_5^2, x_5^7\}$ $\{x_5^3, x_5^6\}$, or $\{x_5^4, x_5^5\}$.

As a consequence, the presence of a single pruning edge $\{u, v\}$ with $u < v$ and $v > K$ means that at level v, we are left with only 2^u out of the 2^{v-K} points that are feasible with the discretization distances. In general, a pruning edge $\{u, v\}$ where there are 2^{u-K-h} feasible points at level u (the factor 2^{-h} arising because of other pruning edges incident on u) will yield 2^{u-h} feasible points at level v. In other words, if $f(u)$ is the number of feasible points at level u and $\{u, v\} \in E_P$, then $f(v) = 2^K f(u)$.

These properties can be generalized to arbitrary K and to ^{K}DMDGP graphs of any size [87]. The property, which is intimately linked to the K adjacent predecessors being immediate, allows us to compute the invariant group of the realization set X of G as a subgroup of $\mathscr{G}_D(G)$. We call this invariant group the *pruning group* $\mathscr{G}_P(G)$ of G.

We are going to work on the intuitive idea that a pruning edge $\{u, v\}$ prevents branching to feasible points between levels $u + K$ and v. We, therefore, have 2^u instead of 2^{v-K} feasible points at level v. Accordingly, we should remove from $\mathscr{G}_D(G)$ all generators g_w where $w \in \{u + K + 1, \ldots, v\}$. It turns out that this intuition gives the correct point of view (see Fig. 5.8).

We let:

$$\Gamma_P = \{g_w \mid w > K \wedge \forall \{u, v\} \in E_P \ (w \notin \{u + K + 1, \ldots, v\})\}, \tag{5.2}$$

and $\mathscr{G}_P(G) = \langle \Gamma_P \rangle$. The fact that $\mathscr{G}_P(G) \leq \mathscr{G}_D(G)$ holds because Γ_P is a subset of the generators of $\mathscr{G}_D(G)$ and commutativity of $\mathscr{G}_P(G)$ trivially follows because it is a subgroup of a commutative group. The fact that X is invariant with respect to $\mathscr{G}_P(G)$ follows by construction of Γ_P. The action of $\mathscr{G}_P(G)$ on X again turns out to be transitive, as in the case of the discretization group. Unfortunately, however, no elementary proof is known for this fact yet. A rigorous (but complicated) proof is given in [87, Theorem 6.4]. A shorter and somewhat more informal proof is given in [85, Theorem 3.3].

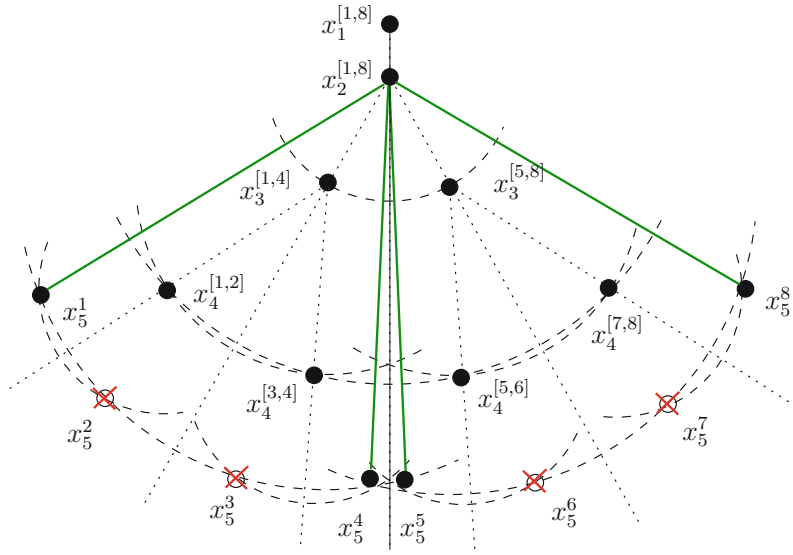


Fig. 5.8 Influence of pruning edge $\{2, 5\}$: removing group generators g_w for $w \in \{2+K+1, 5\}$, i.e., removing g_5 from $\mathscr{G}_D(G) = \langle g_3, g_4, g_5 \rangle$ yields the pruning group $\mathscr{G}_P(G) = \langle g_3, g_4 \rangle$. The notation $x_j^{[i,\ell]}$ indicates that the neighboring dot represents the position of the (equal) point vectors $x_j^i, x_j^{i+1}, \ldots, x_j^\ell \in \mathbb{R}^K$, for $j \in \{1, \ldots, 5\}$ and $x^i \in X$ for all $i \leq 8$.

5.2.4 A symmetry-aware BP

The alternative implementation to RealizeDMDGP of Sect. 5.2.2 can be generalized to this setting: it suffices to find Γ_P (Alg. 7). Notice we represent each generator $g_w \in \Gamma_P$ by its index w.

Algorithm 7 $\Gamma_P = \mathsf{PruningGroupGenerators}(G, E_P)$

```
// start with all generator indices
Γ_P = {K + 1, ..., n}
for w ∈ {K + 1, ..., n} do
  for {u, v} ∈ E_P do
    if u + K + 1 ≤ w ≤ v then
      // if w is between u + K and v, and {u, v} is a pruning edge, remove w
      Γ_P = Γ_P ∖ {w}
      break
    end if
  end for
end for
return Γ_P
```

Notice that if G is a general ^{K}DMDGP graph, unlike for the case where $E_P = \varnothing$ (Sect. 5.2.2), finding the first realization can still take exponential time, depending on the structure of E_P.

On the other hand, the symmetry-aware ^{K}DMDGP realization algorithm (let us call it RealizeDMDGPSymm) is better than RealizeDMDGP whenever finding the first realization is reasonably efficient. Thereafter, no more branching is required to find the other realizations: it suffices to run Alg. 7 (which takes $O(mn)$ at worst, but is practically fairly efficient), and then generates all other realizations from the first one [101].

5.2.5 Number of realizations of ^{K}DMDGP graphs

Since ^{K}DMDGP graphs are also $(K-1)$-laterative graphs, the bound $|X| \leq 2^{n-K}$ given in Sect. 4.4.5 holds. It turns out that in this case we can be much more precise: $|X| = 2^{|\Gamma_P|}$ [82] (recall that Γ_P, defined in Eq. (5.2), is the set of generators of the pruning group).
Here is the proof.

- Since $\mathscr{G}_D(G)$ is isomorphic to C_2^{n-K}, $|\mathscr{G}_D(G)| = 2^{n-K}$.
- Since $\mathscr{G}_P(G)$ is a subgroup of $\mathscr{G}_D(G)$, $\exists \ell \in \mathbb{N}$ such that $\mathscr{G}_P(G) \cong C_2^{\ell}$ (this is called "Lagrange's little theorem").
- Since each C_2 in the Cartesian product C_2^{ℓ} corresponds to as generator g_v of the group, $\ell = |\Gamma_P|$.
- By transitivity, $\mathscr{G}_P(G)x = X$ for each $x \in X$.
- By idempotency, $g^2 = e$ for each $g \in \mathscr{G}_P(G)$.
- By the above, $g^2 = e \Rightarrow g^{-1}gg = g^{-1} \Rightarrow g^{-1} = g$ for each $g \in \mathscr{G}_P(G)$.
- If $gx = hx$ for $g, h \in \mathscr{G}_P(G)$ and $x \in X$, then $h^{-1}gx = x \Rightarrow hgx = x \Rightarrow hg = e \Rightarrow h = g^{-1} = g$.
- By the above, the mapping $\mathscr{G}_P(G)x \to \mathscr{G}_P(G)$ given by $gx \to g$ is injective.
- As long as the assumption (Sect. 3.3.6) on the rank of A holds, if $g, h \in \mathscr{G}_P(G)$ are distinct and $x \in X$, then gx and hx are also distinct.
- By the above, the mapping $\mathscr{G}_P(G)x \to \mathscr{G}_P(G)$ given by $gx \to g$ is a bijection.
- By the above, $|\mathscr{G}_P(G)x| = |\mathscr{G}_P(G)|$.
- Thus, for any $x \in X$, $|X| = |\mathscr{G}_P(G)x| = |\mathscr{G}_P(G)| = 2^{|\Gamma_P|}$.

5.3 Fixed-parameter tractability

In this section, we show that under certain conditions, RealizeDMDGP runs in time $O(2^h p(n))$ in the worst case, where $p(n)$ is a polynomial. Such algorithms are called *fixed-parameter tractable* (FPT), because fixing one parameter (in this case, h) yields a polytime algorithm.

Bear in mind that any BP tree has a height bounded by n, the number of vertices of the graph being realized. So the exponential growth must be due to the tree width.

5.3.1 BP tree width

Consider an array listing the possible number of partial realizations in function of the BP tree level, as shown in Fig. 5.9. Now, from vertex K we draw a path on this array, which will pass through exactly one value per column. This path depends on E_P; if there are no pruning edges, we know that the number of partial realizations just doubles at each level, so the path will be wholly contained in the first row (shown in Fig. 5.9). At the opposite end, if G is the complete graph, there will be a pruning edge $\{1, v\}$ incident to every column $v > K+1$, the effect of which will be to prune all but $2^1 = 2$ out of the possible 2^{v-K} points at level v. So the path will be down the diagonal (shown in Fig. 5.9): the tree width being bounded by 2, the whole algorithm will take time $O(2^K n)$ if we seek all possible realizations of the initial clique, or $O(n)$ otherwise (this is consistent with the polynomial running time of RealizeComplete). For different types of graphs and different pruning edge sets E_P, the path will be different (again, see Fig. 5.9).

Now consider a general graph, assume $X \neq \varnothing$, and suppose we are at column v. If there are two pruning edges $\{u, v\}$ and $\{z, v\}$ incident on v, with $u < z$, then, because a realization must exist, the points x_v at level v feasible with d_{uv} must be a strict subset of the points that are feasible with d_{zv}

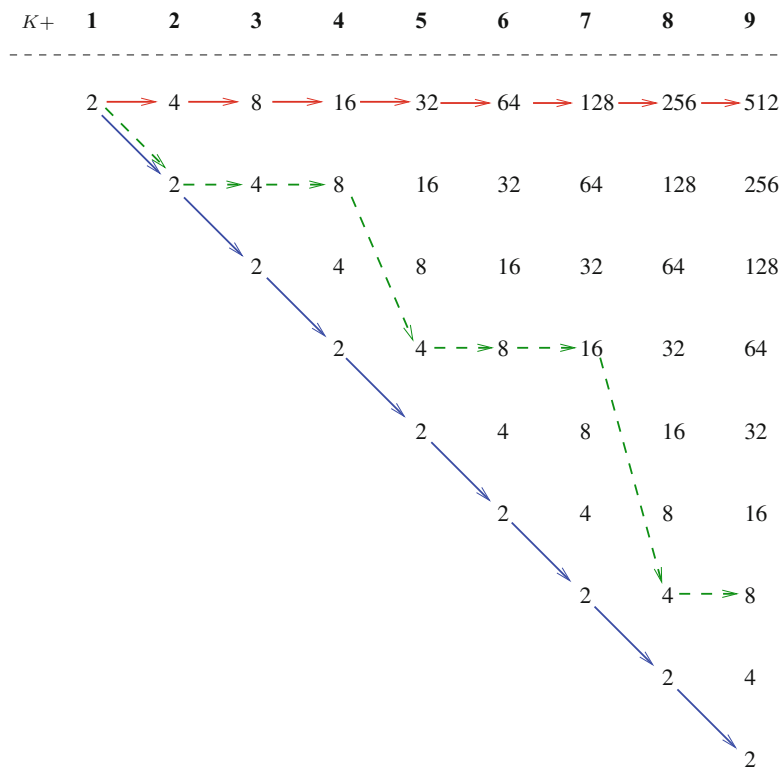


Fig. 5.9 Possible values for $|X|$ in function of the BP tree level $K + v$, and BP behavior for three graph types.

(see Fig. 5.7, and consider the case where $u = 1$ and $z = 2$). So it suffices to look at vertex u having minimum rank $\rho(u)$ such that $\exists\{u, v\} \in E_P$: this is a certificate that there are 2^u feasible points at level v.

For any $v > K + 1$, let $f(v)$ be the number of points that are feasible with all distances d_{uw} (with $u < w \leq v$) at level v. We compute f using Alg. 8.

Algorithm 8 $f = \mathsf{FeasiblePointsTreeLevel}(G, E_P)$

$\forall v \leq K \ f(v) = 1$
$f(K + 1) = 2$
for $v \in \{K + 2, \ldots, n\}$ **do**
  // find the "longest" pruning edge $\{u, v\}$ with $u < v$
  $u = \arg\min\{\rho(w) \mid \{w, v\} \in E_P\}$
  $f(v) = 2^K f(u)$
end for

In order to show that the BP algorithm is FPT, it suffices to prove that the tree width is bounded by a quantity $O(2^h p(n))$ where h is a constant and $p(n)$ a polynomial. Consider the case of pruning edges in a pattern $\{1, \ell\}, \{2, \ell + 1\}, \ldots$, i.e., suppose that there is an integer $\ell \in \{K + 1, \ldots, n\}$ such that $\{u, v\} \in E_P$ if and only if $v - u = \ell - 1$ (for convenience, we call pruning edges in this pattern *periodic*). Then, branching will occur up to level ℓ (since the first pruning edge can only be $\{1, \ell + 1\}$), and then, at level ℓ, pruning will only leave $2^1 = 2$ feasible points. Since every $v > \ell$ will also be incident to a pruning edge, the tree width will be exponential until level ℓ, and constant

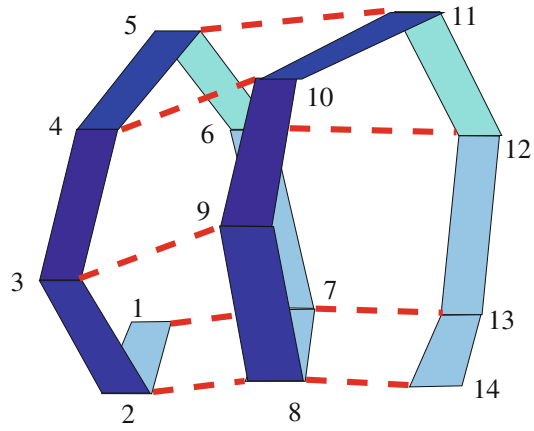


Fig. 5.10 Periodic pruning edges (dashed) with period $|u - v| - 1 = 5$ on a folded backbone.

$(= 2)$ afterwards. This yields a FPT worst-case running time of $O(2^\ell n)$. More FPT cases of the BP algorithm are given in [85].

5.3.2 The BP seems polynomial on proteins

Since protein backbones fold in space into helical shapes, they are likely to give rise to situations where atoms are close in space whenever they are at fixed rank difference on the backbone (see Fig. 5.10). All the protein backbone graphs we tested in the Protein Data Bank (PDB) [11] had subgraphs with periodic pruning edges $\{u, v\}$ having constant period $\ell = |u - v| - 1$. For most of them, $\ell = 4$; for a few, $\ell = 7$ [85]. So we can fix $\ell = 7$ and claim that the BP runs polynomially on our tested instances. This led us to formulate the thesis that the BP algorithm is polynomial on proteins. For more details about how proteins fold, see [40, 108].

5.4 Exercises

5.4.1 Exercise
Prove that whenever realizing a K-clique, infeasibility detection can never occur because of the call to Next.

5.4.2 Exercise
Draw all the possible realizations in the plane of a graph consisting of a chain of p triangles (sharing edges, as in Fig. 5.2, with vertices ordered in a similar way, from the start to the end of the chain) with unit edge weights, for $p \in \{4, 5, 6\}$: how many are there, modulo translations and rotations, in function of p? How does your answer change if you add an edge from the first to the last vertex?

5.4.3 Exercise
Generalize Exercise 5.4.2 to the three-dimensional setting: replace "plane" by "3D space," "triangles" by "3-simplices," and "sharing an edge" by "sharing a triangular face" (see Fig. 5.4). Answer the same two questions as in Exercise 5.4.2. What about a generalization to a general K-dimensional space: would your answers change?

5.4.4 Exercise

Consider the weighted graph on five vertices in Fig. 5.2 having edge weights $d_{12} = d_{23} = d_{34} = d_{45} = \sqrt{2}$, $d_{13} = d_{24} = d_{35} = 2$ and drawn in $\mathbb{R}^2$. Using the formulæ given in Appendix A.5.6 (specifically the material about reflections on page 133), write down explicitly (as matrices) all of the operators R_x^v in the closed-form equation of the partial reflection g_v (Eq. (5.1)), for $v \in \{3, 4, 5\}$. Can you also do this for $v \in \{1, 2\}$? Now suppose you ignore the weight d_{45}, but you know that $d_{15} = 4$. As before, write down all of the operators R_x^v as matrices for $v \in \{3, 4, 5\}$, and determine whether you can do it also for $v \in \{1, 2\}$ (if yes, how?).

5.4.5 Exercise

How many realizations (modulo translations and rotations) does the graph G in Fig. 5.4 have? Now consider G'_q having $E(G'_q) = E(G) \cup \{\{1, q\}$ for $q \in \{5, 6, 7, 8, 9\}$: how many realizations? Now consider G'' having $E(G'') = E(G) \cup \{\{1, 5\}, \{5, 10\}\}$: how many realizations?

5.4.6 Exercise

Define a class of ^{K}DMDGP instances on which the BP algorithm is FPT, and which has as few pruning edges as possible. [Hint: see [85, Prop. 3]]

Chapter 6
Vertex orders

The message so far is that, if the input graph G has a K-lateration order, then we can realize it in $\mathbb{R}^K$ in polytime; if it has a $(K-1)$-lateration order, we can realize it in $\mathbb{R}^K$ in exponential time, or still in polytime if we are lucky with the adjacent predecessors, the pruning edges, and the possibility of fixing some parameter over a whole class of instances. The trouble here is that in practice the input often just consists of the graph and the edge weight function: no order is explicitly given.

In this chapter, we address the question: given a graph, does it have a trilateration order? And if so, is it contiguous? Unlike the DGP, which is not known to be in **NP**, these order existence problems are both in **NP**: if a graph is a YES instance, a suitable vertex order can be verified to be correct in polytime, by simply checking that it has enough adjacent predecessors.

6.1 Existence of trilateration orders

We address the former question first: how do we know whether a graph is a DDGP instance graph?

TRILATERATION ORDERING PROBLEM[1] (TOP). Given a simple undirected graph $G = (V, E)$ and a positive integer K, does G have a $(K-1)$-lateration order?

Any $(K-1)$-lateration order requires the first K vertices to be a clique, and all subsequent vertices to have at least K adjacent predecessors. Since finding an initial K-clique is necessary to start the order off, it would intuitively seem that the TOP is at least as hard as:

K- CLIQUE. Given a graph G, does it have a K-clique as a subgraph?

It turns out that this intuition is correct, although for a formal **NP**-completeness proof we need a little more work.

6.1.1 Problem hardness

In this section, we prove that TOP is **NP**-complete. The strategy is the same as for **NP**-hardness of the DGP (see Sect. 2.4.2): we pick another **NP**-hard problem P, and we show that there is a polynomial transformation (also known as *reduction*, see Sect. A.9.6) from P to TOP such that P is a YES instance if and only if the reduced TOP instance is YES. This implies that if we could solve TOP, then P could be solved by means of a polynomial numbers of calls to a solution algorithm for TOP, which is another

[1] Formally, this is the same as the order called DVOP in [76, 84].

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_6

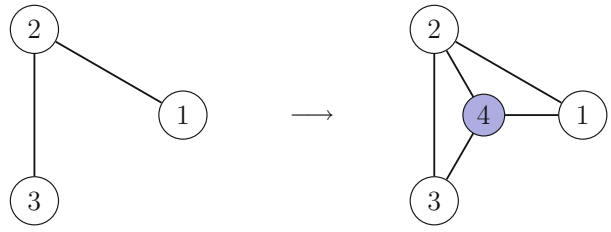


Fig. 6.1 Polynomial reduction from K- CLIQUE to TOP with $K = 2$, with G on the *left* and G' on the *right*, where $U = \{4\}$.

way of saying that TOP is at least as hard as P modulo a polynomial amount of computational effort. Since P is assumed to be **NP**-hard, then TOP must also be in the same complexity class. Moreover, since TOP is also in **NP**, it is **NP**-complete.

Let $G = (V, E)$ (with $V = \{1, \ldots, n\}$) be a connected graph and K be a positive integer, so that (G, K) is an instance of K- CLIQUE. We transform G into a graph $G' = (V', E')$ as follows: we add a set U of $K-1$ vertices to V to obtain $V' = V \cup U$ (say, $U = \{n+1, \ldots, n+K-1\}$). We make every new vertex in U adjacent to every vertex in V, so that $E' = E \cup \{\{u, v\} \mid u \leq n \wedge n < v < n+K\}$ (see Fig. 6.1). Next, we prove that (G, K) is a YES instance of K- CLIQUE if and only if (G', K) is a YES instance of TOP.

6.1.1.1 If G is YES, then G' is YES

The main intuition behind this proof is shown in Fig. 6.2; refer to this when reading the proof.

Assume G is a YES instance of K- CLIQUE: this means it has a K-clique, say $\mathbf{K} = \{w_1, \ldots, w_K\}$. We start defining a $(K-1)$-lateration order in G' starting with $\alpha = (w_1, \ldots, w_K, n+1, \ldots, n+K-1)$:

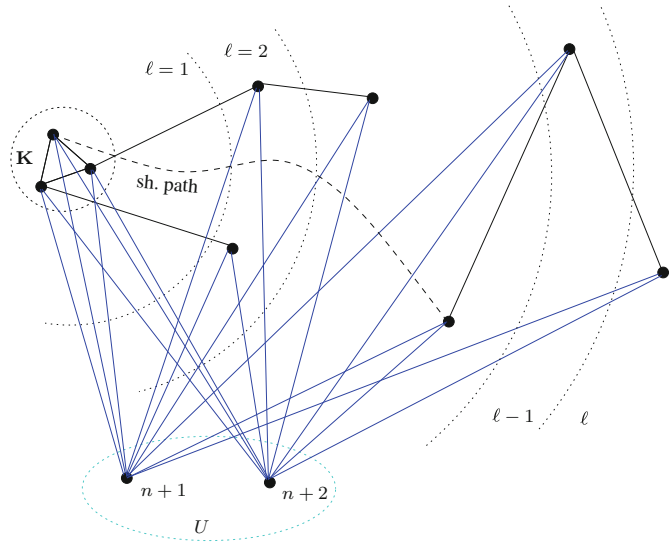


Fig. 6.2 Intuition behind the proof that TOP is **NP**-complete, with $K = 3$. The order α starts with $\mathbf{K}$ then U.

since $\mathbf{K}$ is a clique, and every vertex in U is adjacent to all of V by construction (and hence to the vertices $w_1, \ldots, w_K$ in particular), α is a $(K-1)$-lateration order for $\mathbf{K} \cup U$.

We have to extend α to the yet unlisted vertices of $V \setminus \mathbf{K}$. Since G is connected, G' is connected, which means that there is a shortest path from any vertex to any other. For any $U' \subseteq V'$ and positive integer ℓ, let $N^\ell(U')$ be the set of vertices $v \in V'$ at graph distance[2] at most ℓ from U' (we define $N^0(U') = U'$). In other words, if $v \in N^\ell(U')$, then there must be a shortest path having ℓ edges from some vertex of U' to v. We shall prove by induction on ℓ that we can list the yet unlisted vertices in $N^\ell(\mathbf{K})$ and obtain a $(K-1)$-lateration order on G'.

The induction starts with $\ell = 1$: if an unlisted $v \in V'$ is in $N^1(\mathbf{K})$, then it means it is adjacent to some vertex in $\mathbf{K}$. But then, since by construction every vertex in U is adjacent to every vertex in V, it means that v is adjacent to every one of the $K-1$ vertices in U. So v is adjacent to one vertex in $\mathbf{K}$ and the $K-1$ vertices in U, all of which are already listed in α. This means that we can list v directly after α: this way, v will have K adjacent predecessors. Any other unlisted vertex in $N^1(\mathbf{K})$ can be listed after v for the same reasons. Let β be the order obtained by listing $N^1(\mathbf{K})$ after α.

The induction hypothesis is that β is a partial $(K-1)$-lateration order for α followed by vertices in $N^{\ell-1}(\mathbf{K})$. We aim to conclude that listing the unlisted vertices in $N^\ell(\mathbf{K})$ after β yields a $(K-1)$-lateration order. The reason is simply that $N^\ell(\mathbf{K}) = N^1(N^{\ell-1}(\mathbf{K}))$: if v is at shortest distance ℓ from $\mathbf{K}$, then it is adjacent to some vertex at shortest distance $\ell - 1$ from $\mathbf{K}$. So the same argument used to start the induction applies here with $\mathbf{K}$ replaced by $N^{\ell-1}(\mathbf{K})$. But since listing the latter after α yields a $(K-1)$-lateration order by the induction hypothesis, the same must hold for listing the unlisted vertices in $N^1(N^{\ell-1}(\mathbf{K}))$ after β, which is the same as listing $N^\ell(\mathbf{K})$ after α.

By this induction argument, the claim must hold for every ℓ including the maximum possible ℓ in G', which means that there is a $(K-1)$-lateration order in G'.

6.1.1.2 If G is NO, then G' is NO

Conversely, let G be a NO instance of K-CLIQUE. We show by contradiction that G' cannot have a $(K-1)$-lateration order. So suppose it has one, and call it $\alpha = (v'_1, \ldots, v'_{n+K-1})$. Since this is a $(K-1)$-lateration order, its first K vertices are a K-clique: let us call it $C = \{v'_1, \ldots, v'_K\}$. If $C \subseteq V$, then the vertex set of G has a K-clique, which implies that G is a YES instance of K-CLIQUE against the assumption: so at least one vertex in C must be in U. Moreover, since no edge in G' is adjacent to two vertices in U, it means that U is a stable set: hence, since C is a clique, at most one vertex in C can be in U. We conclude that C consists of exactly one vertex in U and $K-1$ vertices in V, which implies that G has a $(K-1)$-clique C'.

Now consider v'_{K+1}: can this vertex be in U? Notice that v'_{K+1} is the first vertex in the $(K-1)$-lateration order α after the initial clique C, so it must be adjacent to all vertices in C: but one of these is in U, and we already remarked that no two vertices in U are pairwise adjacent. So v'_{K+1} cannot be in U, and must necessarily be in V. But then there is a vertex in V which is adjacent to all of the vertices in C, $K-1$ of which (i.e., those in C') are also in V. By this, we conclude that $C' \cup \{v'_{K+1}\} \subset V$ is a K-clique in G, which is a contradiction since G was assumed to be a NO instance of K-CLIQUE. Hence, G' is a NO instance of TOP.

[2]This distance is given by the length of the path.

6.1.2 A Fixed-Parameter Tractable algorithm

Since TOP is in **NP** and **NP**-hard (by the reduction in Sect. 6.1.1), it is **NP**-complete . Notwithstanding, its hardness mostly stems from finding the initial K-clique $\mathbf{K}$. If it is given, then Alg. 9 can complete $\mathbf{K}$ to a $(K-1)$-lateration order, or conclude that none exists, in polytime. We look at the worst-case complexity of Alg. 9 in function of $n = |V|$: the loop at lines 5–7 takes $O(n)$, Line 11 takes $O(n)$, the loop at lines 19–21 takes $O(n)$, and the outer loop at lines 9–24 takes $O(n)$. So we end up with $O(n + n(n + n)) = O(n + n^2) = O(n^2)$.

Algorithm 9 $\alpha = \mathsf{FindTOFromClique}(G, K-1, \mathbf{K})$

```
1: // initialize the order with the clique
2: let α = (u_1, ..., u_K)
3: // initialize number of adjacent predecessors
4: let W = V ∖ K
5: for v ∈ W do
6:    a_v = |N(v) ∩ K|
7: end for
8: // rank vertices in W
9: while |W| > 0 do
10:   // pick vertex with most adj. pred. in W
11:   v = arg max{a_u | u ∈ W}
12:   if a_v < K then
13:      // v has fewer than K adj. pred., no order
14:      return ∅
15:   end if
16:   // extend the order to v
17:   α = (α, v)
18:   // increase adj. pred. of adj. successors of v
19:   for u ∈ N(v) ∩ W do
20:      a_u = a_u + 1
21:   end for
22:   // v has a rank now, disregard it
23:   W = W ∖ {v}
24: end while
25: return α
```

Let us show that Alg. 9 is correct. If (G, K) is a YES instance of TOP and $\mathbf{K}$ is the initial clique of a $(K-1)$-lateration order α in G, then each $v > K$ must be adjacent to at least K predecessors. When $v = K+1$, all the predecessors are in $\mathbf{K}$: the loop at lines 5–7 will set a_v at the maximum value K, so Line 11 will find v. Now let $v > K+1$ and suppose Alg. 9 recovers α correctly for all vertices $u < v$: by the loop at lines 19–21, for all $w \geq v$, a_w will be set to the number of adjacent predecessors in $V \setminus W$ of w. We can suppose without loss of generality that v has maximum a_v among all such ws, since two vertices with $> K$ adjacent predecessors can always be swapped without invalidating the $(K-1)$-lateration order: then Alg. 9 will choose v at Line 11, and the result follows by induction. If (G, K) is a NO instance, then at some point all vertices in $v \in W$ will have $a_v < K$, which means that the order cannot be completed: and no v could have been chosen earlier since a_v increases monotonically; so, if it is $< K$ at the current iteration, it must have been $< K$ at previous iterations too.

If the initial clique is not given, we employ a brute-force approach: we simply call Alg. 9 with any possible initial K-clique, as shown in Alg. 10 (we denote by $\mathcal{P}_K(V)$ the set of all subsets of V with cardinality K). There are $\binom{n}{K}$ subsets of vertices of cardinality K which might be cliques. Since

$\binom{n}{K} = \frac{1}{K!} n(n-1)\cdots(n-K+1)$ is a polynomial $p_K(n)$ for any *fixed* constant K, if the parameter K is fixed, then TOP can be solved in polytime. On the other hand, if K is not fixed (i.e., it is part of the problem input), then n appears in $p_K(n)$ with highest degree $O(K)$, so that the worst-case running time is the exponential function $O(n^{K+2})$. Thus, we conclude that Alg. 10 is FPT.

Let us now discuss why Alg. 10 is correct. If G has a $(K-1)$-lateration order starting with a given clique $\mathbf{K}$, then every $v > K$ will have at least K adjacent predecessors, which will not change by reordering $V \setminus \mathbf{K}$. Since Alg. 10 tries to construct the order by starting from every initial K-clique, it will eventually find it.

Algorithm 10 $\alpha = \mathsf{FindTrilaterationOrder}(G, K-1)$

```
1: for C ∈ P_K(V) do
2:   // cycle over all initial cliques
3:   if C is a clique then
4:     // attempt to find a trilateration order from C
5:     α = FindTOFromClique(G, K − 1, C)
6:     if α ≠ ∅ then
7:       // found, return it
8:       return α
9:     end if
10:  end if
11: end for
12: return ∅
```

Suppose now that G does not have a $(K-1)$-lateration order. Then, for each initial clique, any ordering of the subsequent vertices will end up with some v having fewer than K adjacent predecessors. Conversely, if, for each initial clique $\mathbf{K}$, no decreasing adjacent predecessor cardinality vertex order is a $(K-1)$-lateration order (let us call this assumption $(\star)$), then no other order starting with $\mathbf{K}$ will. Indeed, if one did exist, then reordering it according to decreasing number of adjacent predecessors would yield another $(K-1)$-lateration order, which is a contradiction with $(\star)$. So Alg. 10 correctly returns the infeasibility flag $\emptyset$.

Figure 6.3 shows *Mathematica* finding $(K-1)$-lateration orders.

6.2 Existence of contiguous trilateration orders

We now tackle the second problem, i.e., establishing whether a graph is a ^{K}DMDGP graph.

CONTIGUOUS TRILATERATION ORDERING PROBLEM[3] (CTOP). Given a simple undirected graph $G = (V, E)$ and a positive integer K, does G have a contiguous $(K-1)$-lateration order?

The fact that the trilateration order is required to be contiguous makes things more complicated: CTOP turns out to be **NP**-complete for each fixed K.

[3] In [22], the TOP is called DDGPO and the CTOP is called ^{K}DMDGPO.

```
G = SpiralGraph [0.4,2.2];
Map[(Timing[TrilaterationOrder[G, #]])&, Range[1,5]]

{ { 0.000922, {1, 2, 3, 4, 5, 6, 8, 7, 9, 11, 10} },
  { 0.000554, {1, 2, 3, 4, 5, 6, 8, 7, 9, 11, 10} },
  { 0.000481, {1, 2, 3, 4, 5, 6, 8, 7, 9, 11, 10} },
  { 0.073968, {4, 5, 6, 10,7, 8, 9,11, 2,  3,  1} },
  { 0.086920, {} } }

GraphPlot[G, VertexCoordinateRules->None, VertexLabeling->True]
```

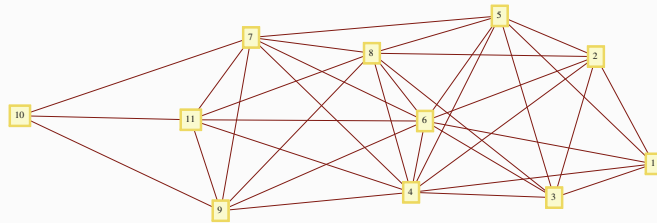


Fig. 6.3 Construction of a random "spiral graph" (see Fig. 1.2), search for 1-lateration, 2-lateration, 3-lateration orders, and proof that it has no 4-lateration orders. The CPU times (in seconds) are shown at the beginning of each output line, and the last instruction displays the graph in 2D

6.2.1 Problem hardness

We show that CTOP is **NP**-hard with $K = 1$ by reduction from a problem called HAMILTONIAN PATH (HP). Recall that a path in a graph G is *Hamiltonian* if it is simple and spans $V(G)$ (see Appendix A.8.4). The HP problem is as follows: given an undirected graph, does it contain a Hamiltonian path?

The reduction from HP to CTOP with $K = 1$ is trivial: a given HP instance G is mapped to G itself as part of the instance $(G, 1)$ of CTOP. We prove that a Hamiltonian path in G induces a contiguous 0-lateration order on G and vice versa.

Since any path consists of an alternating sequence of incident vertices and edges, starting and ending with vertices, paths induce vertex orders where each vertex $v > 1$ is adjacent to $v - 1$; moreover, since a 1-clique only contains a single vertex, the trivial reduction maps Hamiltonian paths to contiguous 0-lateration orders, proving that if G is YES for HP, then it is also YES for CTOP with $K = 1$.

Conversely, suppose there is no Hamiltonian path but there is a contiguous 0-lateration order α in G. Since every $v > 1$ is adjacent to $v - 1$, α induces an alternating sequence $\ldots, v-2, \{v-2, v-1\}, v-1, \{v-1, v\}, v, \{v, v+1\}, v+1, \ldots$ of vertices and edges, starting and ending with a vertex, that lists all vertices in V exactly once: in other words, a Hamiltonian path. But this contradicts G being a NO instance of HP, which means that if G is NO in HP, then it must also be NO in CTOP with $K = 1$.

Since HP is known to be **NP**-complete [104], CTOP with $K = 1$ must be **NP**-hard. Moreover, since CTOP is in **NP**, it is also **NP**-complete.

Extending this proof to a general (fixed) K involves a nontrivial reduction, where each vertex in the HP instance G becomes a K-clique in the CTOP instance G', and each edge in $E(G)$ becomes a chain of two bicliques in G', as shown in Fig. 6.4. The proof is really a generalization of the case $K = 1$: a Hamiltonian path in G yields a contiguous $(K-1)$-lateration order in G', and vice versa. We omit the details of the proof, but conclude that CTOP is **NP**-complete for any fixed K [22].

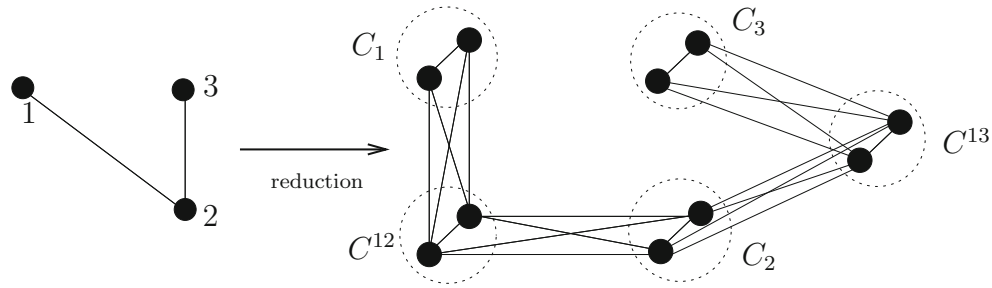


Fig. 6.4 The reduction from HP to CTOP, for $K = 2$.

6.2.2 A mathematical programming formulation

One possible way to find contiguous $(K-1)$-lateration orders for a graph G is to "model" the problem using the Mathematical Programming (MP) language.

An MP formulation consists of a list p of *parameter* symbols (which encode the input instance), a list x of *decision variable* symbols (which will encode a YES certificate at the end of the optimization process, if the input instance is YES), an *objective function* $f_p(x)$ to be optimized, and a set of *constraints* $\boldsymbol{g}_p(x) \leq 0$, where $x \in \mathbb{R}^n$, $f_p : \mathbb{R}^n \to \mathbb{R}$, and $\boldsymbol{g}_p : \mathbb{R}^n \to \mathbb{R}^m$.

Let (G, K) be an instance for CTOP, with $G = (V, E)$, $n = |V|$, and K a positive integer. We present an Integer Linear Programming (ILP) formulation for finding a $(K-1)$-lateration order in G if it exists. For any $v \in V$ and $i \in \bar{n} = \{1, \ldots, n\}$, let x_{vi} be a binary variable, which will take value 1 if v is the $\rho(v) = i$, or 0 otherwise. This is a pure feasibility problem, so it has no objective function:

1. Each vertex has a unique order rank:

$$\forall v \in V \quad \sum_{i \in \bar{n}} x_{vi} = 1;$$

2. Each rank value is assigned a unique vertex:

$$\forall i \in \bar{n} \quad \sum_{v \in V} x_{vi} = 1;$$

3. There must be an initial K-clique:

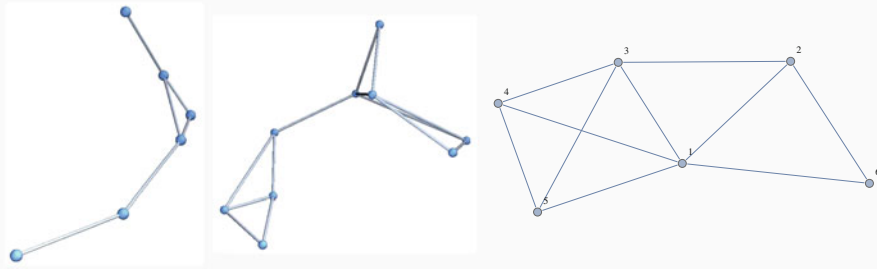
$$\forall v \in V, i \in \{2, \ldots, K\} \quad \sum_{u \in N(v)} \sum_{j < i} x_{uj} \geq (i-1) x_{vi};$$

4. Each vertex with rank $> K$ must have at least K contiguous adjacent predecessors

$$\forall v \in V, i > K \quad \sum_{u \in N(v)} \sum_{i-K \leq j < i} x_{uj} \geq K x_{vi}.$$

This ILP formulation can be solved with an off-the-shelf MP solver such as CPLEX [60]. This solution approach is based on a worst-case exponential algorithm (called Branch-and-Bound), but at least little coding and no creativity is necessary to deploy the solution process. Some open-source codes

```
G = SpiralGraph[0.7, 1.4]; G2 = SpiralGraph[0.5, 1.8]
G3 = Graph[{1<->2,1<->3,1<->4,1<->5,1<->6,2<->3,2<->6,3<->4,3<->5,4<->5}]
```




```
ContiguousTrilaterationOrder [G,2]
  {}
ContiguousTrilaterationOrder [G2,2]
  {}
ContiguousTrilaterationOrder [G3,2]
  {6, 2, 1, 3, 5, 4}
```

Fig. 6.5 Some graphs with and without contiguous 2-lateration orders. The time taken to compute orders (or prove they do not exist) is exponential in the input size, so be prepared to wait for a long time, and try this on graphs with more than 10 vertices at your own risk!

similar to CPLEX exist in the literature (e.g., GLPK [89] and SCIP [12]). CPLEX can be downloaded free of charge for academic work. Although *Mathematica*'s abilities to solve MPs are not at the state of the art, Fig. 6.5 shows what can be done.

6.3 Exercises

6.3.1 Exercise

In Sect. 6.1.1, page 57, we state that we have to "extend α to the yet unlisted vertices of $V \setminus \mathbf{K}$": which vertices are these?

6.3.2 Exercise

Consider the operator $N^\ell(U)$ defined in Sect. 6.1.1 on page 57, and the graph G in Fig. 5.4. Let $U = \{1, 2, 3\}$ and compute $N^\ell(U)$ for $\ell \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

6.3.3 Exercise

Consider again the operator $N^\ell(U)$ defined above. Prove formally that for any $\ell > 1$, we have $N^\ell(U) = N^1(N^{\ell-1}(U)) = N^{\ell-1}(N^1(U))$.

6.3.4 Exercise

Show formally that, if K is fixed to a constant, $\binom{n}{K}$ is a polynomial in function of n.

6.3.5 Exercise
Compute trilateration orders (by hand first, then implementing Alg. 9–10 using *Mathematica*) for the graphs in Figs. 4.1, 4.5, 5.2, 5.4, 5.10 for $K \in \{1, 2, 3, 4\}$.

6.3.6 Exercise
A path is *Eulerian* if it traverses each edge of the graph exactly once. Is a Hamiltonian path necessarily Eulerian? Is a Eulerian path necessarily Hamiltonian? Provide classes of graphs where each path is Eulerian if and only if it is Hamiltonian.

6.3.7 Exercise
Compute *contiguous* trilateration orders (using the Mathematical Programming formulation in Sect. 6.2.2) for the graphs in Figs. 4.1, 4.5, 5.2, 5.4, 5.10 for $K \in \{1, 2, 3\}$.

Chapter 7
Flexibility and rigidity

This chapter is optional.

Most of the methods in this book have to do with realizing graphs consisting of K-cliques or $(K+1)$-cliques (or quasi-cliques) in $\mathbb{R}^K$. The reason is that such graphs have a finite number of realizations in $\mathbb{R}^K$ up to rotations and translations, which means that realizations can be obtained by searching discrete subsets of $\mathbb{R}^K$, rather than the whole of $\mathbb{R}^K$. In turn, this often implies that a theoretically or practically efficient search method can be used.

In fact, if you were to "construct" mechanical realizations of DDGP and ^{K}DMDGP YES instance graphs in 2D or 3D using rigid bars and movable joints, you would obtain frameworks whose only continuous movements are translations and rotations. By fixing a realization of an initial clique, we are essentially preventing translations and rotations, so the bar-and-joint frameworks we obtain from these graphs are rigid (see Fig. 7.1, right). By contrast, structures having continuous movements excluding translations and rotations are flexible (see Fig. 7.1, left).

7.1 Some preliminary notions

There are many definitions of flexibility and rigidity, and they all differ over special cases: collinear/coplanar points, matrices not having full rank, and so on. The definitions below are often very technical, because it is difficult to tell these special cases apart. Once the special cases have been dealt with, however, we end up with a notion of rigidity that applies to graphs independently of their realizations.

Let $G = (V, E)$ be a simple undirected connected graph with $|V| = n$ and d be an edge weight function. Assume that (G, d, K) is a YES instance of the DGP for some positive integer K, so that there exists at least one element x in the set of realizations of G in $\mathbb{R}^K$ satisfying Eq. (2.2). In this chapter, the set of realizations of G will be denoted X_G instead of X, to emphasize the dependency on G.

The *completion* of (G, d, K, x) is a clique $\mathbf{K}(G)$ on V with an edge weight function $\bar{d}$ that extends d using information from x (see Fig. 7.2). More precisely,

$$\forall u \neq v \in V \quad \bar{d}_{uv} = \begin{cases} d_{uv} & \text{if} \quad \{u, v\} \in E(G) \\ \|x_u - x_v\| & \text{if} \quad \{u, v\} \notin E(G). \end{cases}$$

The pair (G, x) is also known as a *framework*. Any mapping $\phi : \mathbb{R}^{Kn} \to \mathbb{R}^{Kn}$ such that $\phi(x)$ belongs to X_G is an *isometry* of the realization x (also see Sect. 5.2.1). Translations, rotations, and reflections are isometries (see Appendix A.5.6) of sets of points. Partial reflections in the pruning group $\mathscr{G}_P$ (see Eq. (5.1)) are also isometries. An isometry of the completion of (G, d, K, x) is a *congruence*

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_7

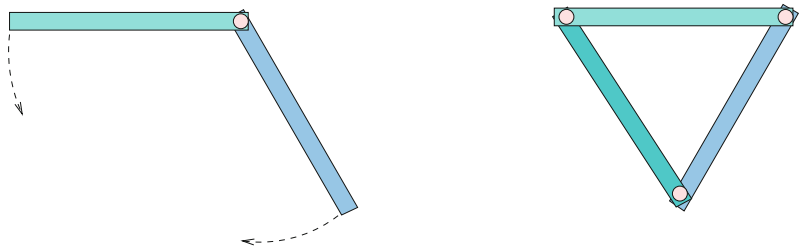


Fig. 7.1 Flexible bar-and-joint framework (left), and a rigid one (right).

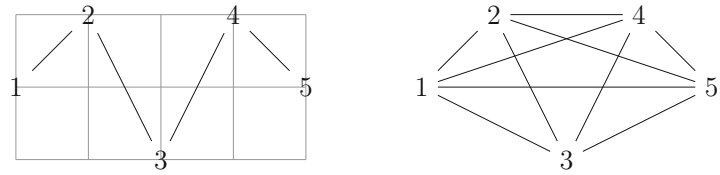


Fig. 7.2 Realization of a graph in the plane (left) and its completion (right).

of a set of points (it follows trivially that an isometry of any clique realization is also congruence). Note that isometry and congruence have been defined and used before, in slightly different settings: all these definitions are compatible, and they indicate the same mathematical entity.

Translations, rotations, and reflections that are applied to whole sets of points are congruences. The converse also holds: any congruence of a set of points is a composition of translations, rotations, and reflections applied to every point in the set.

Consider a framework (G, x) where $x \in \mathbb{R}^{Kn}$, and a neighborhood χ of x (see Appendix A.6) in the topology induced by the ℓ_2 metric. A set Φ of isometries of (G, x) is *local* w.r.t. χ if $\forall \phi \in \Phi$ $(\phi(x) \in \chi)$. Local isometries provide a convenient way to define rigidity, as we shall soon see below (Sect. 7.2). As χ becomes small enough, if Φ only contains the identity (modulo congruences), the framework is rigid. Think of local isometries as moving the realization x to other realizations that are arbitrarily close to x.

7.2 Rigidity of frameworks

A framework is *rigid* if all of its local isometries are congruences. Any framework that is not rigid is *flexible*.

This definition can be written in mathematical symbols as follows: for a graph $G = (V, E)$ weighted by d and $x \in X_G$, let $f_G : \mathbb{R}^{Kn} \to \mathbb{R}^{|E|}$ be given by

$$f_G(x) = (\|x_u - x_v\|^2 \mid \{u, v\} \in E).$$

Then, (G, x) is rigid [3] if there is a neighborhood χ of x such that:

$$f_G^{-1}(f_G(x)) \cap \chi = f_{\mathbf{K}(G)}^{-1}(f_{\mathbf{K}(G)}(x)) \cap \chi. \tag{7.1}$$

Let us parse this expression: $f_G(x)$ is the vector of distance values in the framework x, ordered by the corresponding edges in E. Applying f_G^{-1} to this vector yields all realizations that are feasible with

respect to those same distances, i.e., X_G. Intersecting this set with χ only yields those realizations in X_G which can be made to be arbitrarily close to x, i.e., the result of applying local isometries to x. On the right-hand side, we have the same, but applied to the completion of G. We showed in Chap. 3 that $|X_{\mathbf{K}(G)}| = 1$ if we do not allow congruences, so Eq. (7.1) is the same as saying that, close to x, G "behaves like" its completion, i.e., it has no local isometries aside from congruences. In other words, (G, x) is rigid.

If, on the other hand, (G, x) is flexible, it was shown in [3] that there is a continuous path starting at x and such that every point in the path is a realization in $f_G^{-1}(f_G(x)) \setminus f_{\mathbf{K}(G)}^{-1}(f_{\mathbf{K}(G)}(x))$.

This definition means precisely what is suggested in Fig. 7.1: the only way we can continuously move a vertex in a rigid framework is if all vertices move in such a way as to preserve all pairwise distances, and not just those given by the graph edges.

7.3 The rigidity matrix

Since flexibility has to do with movement, which is a function of time, we take $p(t)$ to model the continuous movement of $x = p(0)$ to a different realization $x' = p(1)$, as t varies in $[0, 1]$. Since $p : [0, 1] \to \mathbb{R}^{Kn}$, we can write $p(t)$ as a vector $(p_1(t), \ldots, p_n(t))$ for any $t \in [0, 1]$. We then rewrite Eq. (2.4) as a function of time:

$$\forall \{u, v\} \in E, t \in [0, 1] \quad \|p_u(t) - p_v(t)\|^2 = d_{uv}^2. \tag{7.2}$$

Since d_{uv}^2 is a constant which does not depend on t, its derivative with respect to t is zero. Hence,

$$\forall \{u, v\} \in E, t \in [0, 1] \quad \frac{\mathsf{d}}{\mathsf{d}t}\|p_u(t) - p_v(t)\|^2 = 0,$$

which is sometimes also written $\mathsf{d}f_G(p(t)) = 0$. We can assume without loss of generality that $p(t)$ varies linearly with t (i.e., its speed vectors have constant modulus), so:

$$\forall \{u, v\} \in E, \quad (p_u - p_v) \cdot (\dot{p}_u - \dot{p}_v) = 0, \tag{7.3}$$

where $\dot{p}_v$ is the velocity of v. Notice that Eq. (7.3) is a linear system $R\dot{p} = 0$. R is known as the *rigidity matrix* of the framework (G, x), and denoted by $\mathbf{R}(G, x)$.

7.3.1 The rank of the rigidity matrix

The rank of the rigidity matrix $\mathbf{R}(G, x)$ is an indicator of the existence of local isometries moving (G, x) aside from congruences. As such, it can tell rigid frameworks apart from flexible ones [3]. Let us see two examples relating to Fig. 7.1 (left) and (right).

7.3.1 Example
If (G, p) is the framework in Fig. 7.1 (left), then $G = (\{1, 2, 3\}, \{\{1, 2\}, \{1, 3\}\})$, with $d_{12} = d_{13} = 1$ and $K = 2$. The system in Eq. (7.3) is as follows:

$$\begin{pmatrix} p_{11} - p_{21} & p_{12} - p_{22} & p_{21} - p_{11} & p_{22} - p_{12} & 0 & 0 \\ p_{11} - p_{31} & p_{12} - p_{32} & 0 & 0 & p_{31} - p_{11} & p_{32} - p_{12} \end{pmatrix} \begin{pmatrix} \dot{p}_{11} \\ \dot{p}_{12} \\ \dot{p}_{21} \\ \dot{p}_{22} \\ \dot{p}_{31} \\ \dot{p}_{32} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \tag{7.4}$$

which is usually written in the shorthand form:

$$\begin{pmatrix} p_1 - p_2 & p_2 - p_1 & 0 \\ p_1 - p_3 & 0 & p_3 - p_1 \end{pmatrix} \dot{p} = 0.$$

We can pick any feasible realization, such as $x_1 = (0, 0)$, $x_2 = (1, 0)$, and $x_3 = (0, 1)$ (see Fig. 7.3, left), so that Eq. (7.4) is $\mathbf{R}(G, p)\dot{p} = 0$, where the rigidity matrix:

$$\mathbf{R}(G, p) = \begin{pmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \end{pmatrix} \tag{7.5}$$

has rank 2. Notice that translations in the plane have $\tau = 2$ degrees of freedom (vertical and horizontal) and rotations have $\rho = 1$ (the angle). Remark that $\tau + \rho - \mathsf{rk}\,\mathbf{R}(G, p) = 1$: intuitively, this means that there is one degree of freedom for choosing some combination of rotations and translations yielding a local isometry. More precisely, we can rotate the two bars in Fig. 7.1 (left) about the common joint by an arbitrary angle. So (G, x) is flexible. □

7.3.2 Example
We can carry out the same analysis as in Example 7.3.1 with the graph G given by Fig. 7.1 (right), i.e., E also has the edge $\{1, 3\}$, with $d_{13} = 1$. Accordingly, x must change: we take $x_1 = (0, 0)$, $x_2 = (1, 0)$, $x_3 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$. Now, the rigidity matrix

$$\mathbf{R}(G, x) = \begin{pmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 & 0 & \frac{1}{2} & \frac{\sqrt{3}}{2} \\ 0 & 0 & \frac{1}{2} & -\frac{\sqrt{3}}{2} & -\frac{1}{2} & \frac{\sqrt{3}}{2} \end{pmatrix} \tag{7.6}$$

has rank 3, and $\tau + \rho - \mathsf{rk}\,\mathbf{R}(G, x) = 0$. Intuitively, this means that there are no degrees of freedom for choosing local isometries modulo congruences of the whole set of points x. So this framework is rigid. □

7.3.2 Regular and singular realizations

We now introduce some theoretical notions about the relationship between the rank of the rigidity matrix and the corresponding realization.

A realization x of a graph $G = (V, E)$ is *regular* if the rigidity matrix of the corresponding framework (G, x) (where x is $K \times n$) has maximum rank over all edge weight functions d of G and all possible realizations of the graph (G, d) in $\mathbb{R}^K$. Otherwise, x is a *singular* realization.

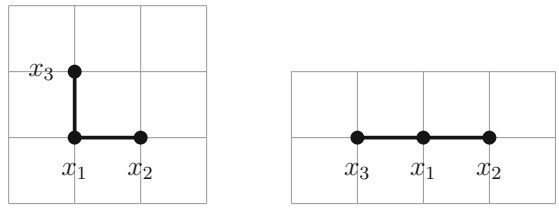


Fig. 7.3 Two realizations of $G = (\{1, 2\}, \{\{1, 2\}, \{2, 3\}\})$ weighted by $d_{12} = d_{13} = 1$.

7.3.3 Example
Consider the framework (G, x) given in Example 7.3.1, and another framework $y = (y_1, y_2, y_3)$ for G where $y_1 = x_1$, $y_2 = x_2$, and $y_3 = (-1, 0)$ (see Fig. 7.3). Since the rigidity matrix $\mathbf{R}(G, x)$ with respect to x (see Eq. (7.5)) has two rows and rank 2, its rank is maximum, which means that x is a regular realization. On the other hand, the rigidity matrix $\mathbf{R}(G, y)$ with respect to y is as follows:

$$\begin{pmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 \end{pmatrix}.$$

Obviously, the second row is just the first multiplied by -1, which means that the rows are not linearly independent, which implies that the rank of this matrix is 1. Since $\mathsf{rk}\,\mathbf{R}(G, x) > \mathsf{rk}\,\mathbf{R}(G, y)$, y is a singular realization. □

7.3.3 The nullity of the rigidity matrix: infinitesimal rigidity

In differential geometry, there is the notion of *tangent space to a manifold*. We are only going to introduce this concept informally (see [114] or the Wikipedia entry for a definition): think of a manifold M as a smooth (hyper-)curve or (hyper-)surface in $\mathbb{R}^K$, and of the tangent space at a point $x \in M$ as the vector space T_x spanned by all vectors tangent to M at x.

In our setting, M is the "surface" in $\mathbb{R}^{|E|}$ defined by $f_G(p)$ as p varies over $\mathbb{R}^{Kn}$ (more precisely, M is the range of f_G, while p belongs to its domain). The tangent space of M at p is denoted by $T(G, p)$. By definition of the rigidity matrix $\mathbf{R}(G, p)$, any tangent z to M at p will satisfy $\mathbf{R}(G, p)z = 0$, and hence will belong to the kernel of $\mathbf{R}(G, p)$, denoted $\ker \mathbf{R}(G, p)$.

Consider $T(\mathbf{K}(G), p)$: this is the tangent space to the manifold (given by $f_{\mathbf{K}(G)}$) of all congruences of p. Intuitively, you may be able to see that having to preserve more distances will make the manifold "smaller" (i.e., fewer degrees of freedom). This also makes the tangent space smaller dimensional (this is not an intuitive consequence, though). In particular, $T(\mathbf{K}(G), p) \subseteq T(G, p)$. Hence, $T(\mathbf{K}(G), p) \subseteq \ker \mathbf{R}(G, p)$.

7.3.4 Example
What does the manifold M defined by $f_G(x)$ look like? Let us see an example: G is the graph of Fig. 7.3, i.e., $V(G) = \{1, 2, 3\}$ and $E(G) = \{\{1, 2\}, \{1, 3\}\}$, weighted by $d_{12} = d_{13} = 1$. As d_{12} and d_{13} range over all of their possible nonnegative values, $f_G(x_1, x_2, x_3) = (\|x_1 - x_2\|^2, \|x_1 - x_3\|^2)$ achieves all of the possible vectors (d_{12}^2, d_{13}^2) in $\mathbb{R}^2$. So M in this case is simply the nonnegative quadrant $\mathbb{R}^2_+$.

We look now at a more interesting set, namely,

$$M' = \{x \in \mathbb{R}^{Kn} \mid f_G(x) = (d_{ij}^2 \mid \{i, j\} \in E)\},$$

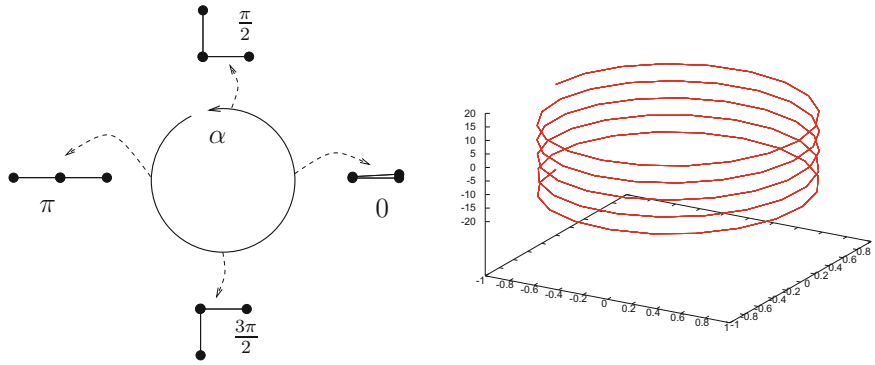


Fig. 7.4 Manifold of realizations of G weighted by d given by the rotation by α around x_1: left, as α varies on a circle; right, as α varies in $\mathbb{R}$ (vertical axis).

which, for the graph of Fig. 7.3, is $M' = \{x \in \mathbb{R}^{2\times 3} \mid f_G(x) = (1, 1)\}$. If we consider congruences, every triplet (x_1, x_2, x_3) respecting the given unit distances will be in M'; in order to represent M' in a limited number of dimensions, we analyze M' "modulo congruences." In general, congruences in the plane can be described using five degrees of freedom: rotations have three (an angle and a 2-vector for the center of rotation) and translations have two (a 2-vector). So we can represent the action of any congruence on x in a five-dimensional space. For the configuration at hand, we also have to keep in mind that x_2 and x_3 can rotate around x_1. If we fix x_1, x_2 to, say, $(0, 0)$ and $(1, 0)$, the framework can no longer translate: the only possible vertex movement compatible with the given distances is the rotation of x_3 around x_1. The angle α of this rotation can be chosen as an independent variable. Two dependent variables that well describe the movement of x_3 are its components x_{31} and x_{32}. As α varies in $\mathbb{R}$, it traces a circle in the projection of the realization space $\mathbb{R}^6$ to $\mathbb{R}^2$. M' is thus given by the spiral $(\cos\alpha, \sin\alpha, \alpha)$ (see Fig. 7.4), which is itself a manifold (since it is a smooth curve). □

We can now define infinitesimal rigidity as follows: (G, p) is *infinitesimally rigid* if $T(\mathbf{K}(G), p) = \ker \mathbf{R}(G, p)$ and *infinitesimally flexible* if $T(\mathbf{K}(G), p) \subsetneq \ker \mathbf{R}(G, p)$.

Let us see an example in 1D.

7.3.5 Example

Consider the empty graph $H = (V, \varnothing)$ with $V = \{1, 2\}$ and the graph $G = (V, E)$ with $E = \{\{1, 2\}\}$, weighted by the distance $d_{12} = 1$, and let $K = 1$. Notice that $\mathbf{K}(H) = G$. Consider any realization x in X_G, say $x_1 = 0$ and $x_2 = 1$; obviously, we also have $x \in X_H$ (see Fig. 7.5). Since H has no edges, f_H maps x to the empty sequence. This is a pathological case which corresponds to there being no relation whatsoever between components of x. This means that in a 1D realization of an empty

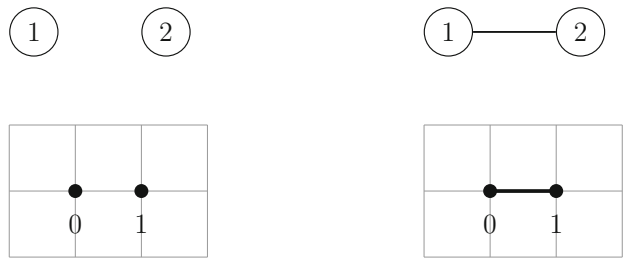


Fig. 7.5 Empty graph (left) and its completion (right) with $K = 1$, and some corresponding realizations in $\mathbb{R}$.

graph, any point can move freely in $\mathbb{R}$. For G, we have $f_G = (\|x_1 - x_2\|^2)$, which yields the 1×2 rigidity matrix $\mathbf{R}(G, x) = (-1\ 1)$ having rank 1. $\mathbf{R}(G, x)\dot{x} = 0$ yields $\dot{x}_1 = \dot{x}_2$, which means that the only possible local isometry for x is a translation, i.e., a congruence. Thus, G is rigid. The set of tangents $T(G, x)$ turns out to be the same as $\ker \mathbf{R}(G, x)$, which is a one-dimensional subspace of $\mathbb{R}^2$. Since $G = \mathbf{K}(H)$, and $T(G, x) \subsetneq \ker \mathbf{R}(H, x)$, H is infinitesimally flexible. On the other hand, since $T(G, x) = \ker \mathbf{R}(G, x)$, G is infinitesimally rigid. □

7.3.4 Asimow and Roth's theorems

Examples 7.3.1–7.3.2 are just special cases of Asimow and Roth's first theorem [3]. This theorem states that if (G, x) is a framework such that x is regular and[1] $\dim \operatorname{aff} x = K' \leq K$, then (G, x) is rigid in $\mathbb{R}^K$ if and only if:

$$\operatorname{rk} \mathbf{R}(G, x) = Kn - \frac{(K' + 1)(2K - K')}{2} \tag{7.7}$$

and flexible if and only if:

$$\operatorname{rk} \mathbf{R}(G, x) < Kn - \frac{(K' + 1)(2K - K')}{2}. \tag{7.8}$$

In Examples 7.3.1–7.3.2, $K' = K = 2$ and $n = 3$. In Example 7.3.2, we have $\operatorname{rk} \mathbf{R}(G, p) = 3$, $Kn = 6$, and $\frac{(K'+1)(2K-K')}{2} = 3$, which satisfy Eq. (7.7). In Example 7.3.1, $\operatorname{rk} \mathbf{R}(G, p) = 2$, which satisfies Eq. (7.8).

Asimow and Roth's second theorem [4] states that a framework (G, x) is infinitesimally rigid if and only if it is rigid and x is regular. This means that all singular realizations are infinitesimally flexible.

7.3.6 Example

Consider the completion $\mathbf{K}(G)$ of the graph G of Example 7.3.3, weighted by $d_{12} = d_{13} = 1$ and $d_{23} = 2$, with the (collinear) realization y given in Example 7.3.3 (see Fig. 7.3, right). The rigidity matrix is

$$R = \begin{pmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & -2 & 0 \end{pmatrix},$$

which has rank 2. But we remarked in Example 7.3.2 that there is an edge weight function for $\mathbf{K}(G)$ and a realization that yields a rigidity matrix for $\mathbf{K}(G)$ having rank 3 (see Eq. (7.6)). So R does not have maximum rank, and hence, y is singular. Now consider the vector $\dot{y} = (0, 1, 0, 0, 0, 0)$: obviously, $R\dot{y} = 0$, so $\dot{y} \in \ker R$. However, $\dot{y}$ is not a tangent vector to the manifold defined by $f_{\mathbf{K}(G)}$ at y, since any translation of vertex 1 by one vertical unit should correspond (as G is a clique) to the same translation applied to vertices 2, 3 (in other words, any tangent having 0, 1 in the first two component must also have some other nonzero component, since it must be a combination of the translation (0, 1, 0, 1, 0, 1) with some other translations and rotations). Thus, $(\mathbf{K}(G), y)$ is rigid but infinitesimally flexible. □

By Asimow and Roth's second theorem, if (G, x) is infinitesimally rigid, then it is also rigid. Moreover, if (G, x) is flexible and x is regular, by Asimow and Roth's first theorem, $\operatorname{rk} R < Kn - \frac{(K'+1)(2K-K')}{2}$. By the rank and nullity theorem, we have $\operatorname{rk} R + \dim \ker R = Kn$, so $\ell = \frac{(K'+1)(2K-K')}{2}$

[1] The operator dim aff returns the dimensionality of the affine hull of the argument, see Appendix A.4.3.

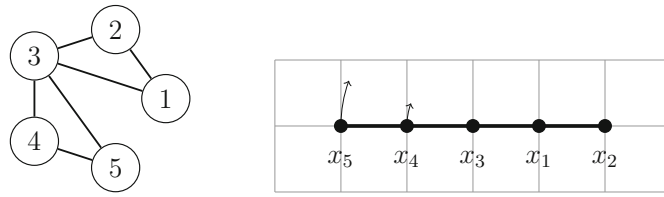


Fig. 7.6 Two 3-cliques joined at vertex 3, and a collinear realization in 2D where x_4, x_5 can rotate around x_3 with x_1, x_2, x_3 fixed.

is strictly smaller than the nullity of R (i.e., the dimension of its kernel). Since ℓ turns out to be the dimension of the tangent space $T(\mathbf{K}(G), x)$, we obtain $T(\mathbf{K}(G), x) \subsetneq \ker R$, which implies that (G, x) is infinitesimally flexible.

The upshot of this is that, for regular realizations, rigidity is equivalent to infinitesimal rigidity and flexibility is equivalent to infinitesimal flexibility. All singular realizations are infinitesimally flexible, but some of them are rigid (see Example 7.3.6) while others are flexible (see Example 7.3.7).

7.3.7 Example
Let $G = (V, E)$ consist of two 3-cliques joined at a vertex (see Fig. 7.6):

$$V = \{1, 2, 3, 4, 5\}, E = \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}, \{3, 5\}, \{4, 5\}\},$$

weighted by $d_{12} = d_{13} = d_{34} = d_{45} = 1$ and $d_{23} = d_{35} = 2$. Let x be a collinear realization in $\mathbb{R}^2$, with $x_1, \ldots, x_5$ all on the horizontal axis, such that $z = (x_1, x_2, x_3)$ is the same as the realization y given in Example 7.3.3 for $G[\{1, 2, 3\}]$, and such that $z' = (x_4, x_3, x_5)$ is the same as the realization y for $G[\{4, 3, 5\}]$ under the vertex relabeling $1 \to 4$, $2 \to 3$, $3 \to 5$. Since z' can rotate by any angle around x_3 with z being fixed, the framework (G, x) is flexible in $\mathbb{R}^2$. Moreover, both $(G[\{1, 2, 3\}], z)$ and $(G[\{4, 3, 5\}], z')$ are infinitesimally flexible frameworks in $\mathbb{R}^2$, so their rigidity matrices R^1, R^2 do not have maximum ranks. But the rigidity matrix of (G, x) simply consists of the rows of R^1 and R^2, so they cannot be all independent, for otherwise both R^1 and R^2 would have maximum rank. Hence, (G, x) is flexible. □

In conclusion, infinitesimal rigidity implies rigidity, and flexibility implies infinitesimal flexibility.

7.3.5 Generic rigidity

The crucial idea behind genericity is that almost no quadruplet of points in 3D space lies in the same plane. In other words, those sets whose vectors all lie on a hyperplane of smaller dimension than the ambient space are rare: a line takes zero area in the plane, and a plane takes zero volume in space. A generalization to arbitrary dimensions requires the concept of Lebesgue measure [64].

Gluck's theorem [52] states that almost all frameworks of a given graph $G = (V, E)$ are rigid or almost all of them are flexible in $\mathbb{R}^K$; in other words, there is no graph for which a fraction $\phi > 0$ of its frameworks are rigid while the remaining $1-\phi$ of them are flexible. This can be seen as a consequence[2] of Asimow and Roth's first theorem when $K' = K$, with the additional observation that the set $\mathcal{X}$ of regular realizations x of G weighted by d, as d varies over all edge weight functions leading to YES instances of the DGP, is a dense open subset of $\mathbb{R}^{Kn}$ (a subset X is *dense* in Y if, for any $y \in Y$, there is $x \in X$ arbitrarily close to y).

[2]Gluck's theorem actually predates, and indeed inspired, Asimow and Roth's.

The reason why $\mathcal{X}$ is dense in $\mathbb{R}^{Kn}$ is that, in order to be regular, a realization x must yield a rank rigidity matrix having maximum rank. If you uniformly sample all the components of a matrix from a given interval, you will get a matrix having maximum rank with[3] probability 1; or, in other words, almost all matrices have maximum rank.

Another way of seeing this is by noticing that smaller ranks correspond to linearly dependent rows (and/or columns) of the linear system (7.3). Linear dependence is a very specific linear relationship between the rows r_i of the matrix, given by $\sum_i c_i r_i = 0$ (for some scalars c_i). It suffices to perturb the rows even slightly, and the equation no longer holds. Intuitively, this suggests that there is always a linearly independent set of rows r_i' arbitrarily close to r_i. Geometrically, linearly dependent rows turn out to refer to sets of points which lie on hyperplanes of the ambient space $\mathbb{R}^{Kn}$, and which are therefore "special": any perturbation of the points leads to them no longer being contained in any hyperplane.

These observations generalize the discussion on the rank of A in Sect. 3.3.6, and lead to the same conclusion: in general, we can assume that the rank of the rigidity matrix is full without worrying about those "few" special cases where it is not. Of course, if we known *a priori* that our framework of interest is special,[4] then this statement no longer holds.

Consequently, we can assume that rigidity and flexibility are properties of the graph G rather than a specific framework (G, x) or a specific edge weight function. We can now plausibly speak of *rigid graphs* and *flexible graphs* with respect to a given positive integer K. Another consequence of Gluck's theorem is that if G is rigid with respect to K, then almost all its frameworks (G, x) are rigid in $\mathbb{R}^K$. We call[5] any of these rigid frameworks (G, x) *generically rigid* (in $\mathbb{R}^K$).

We stress that "almost all" is not the same as "all." Rigid graphs may have edge weight functions and realizations which fail to be rigid or infinitesimally rigid, as shown in Fig. 7.7, right.

7.4 Graph rigidity on the line and in the plane

A condition for verifying graph rigidity is purely combinatorial if it only looks at information given in a graph G, rather than a framework. In this section, we discuss purely combinatorial conditions for establishing graph rigidity on the line and in the plane.

[3] Also see the footnote on page 28.

[4] In the world of human manufacturing, because humans appears to like symmetry and special relationships, realizations may be more likely to be singular than one would believe reasonable.

[5] In most of the existing literature (with the notable exception of [57]), generic rigidity is defined differently: a framework (G, x) is generically rigid in $\mathbb{R}^K$ if it is rigid and there is no single rational polynomial having all the components x_{ij} as roots, for all $i \leq K$, $j \leq n = |V(G)|$; or, in other words, x is *algebraically independent*. If this is the case, then of course the rows of the rigidity matrix could never be linearly dependent, which makes x regular. But since the converse is not true (i.e., there are algebraically dependent but regular realizations), and regularity is really all that is required, we feel, along with [57], that this traditional genericity notion is too strong. *By contrast, with our definition a generically rigid framework is simply a rigid framework with a regular realization, so any infinitesimally rigid framework is generic* (this is false using the definition based on algebraic independence). This definition is similar to the one given by Graver in [56]: the framework (G, x) is generic if all the nontrivial minors of the rigidity matrix have nonzero value (a minor depending on some symbol x is *trivial* if it is identically zero independently of the value assigned to x); and generically rigid if it is generic and rigid. Graver's definition is actually stricter than ours, in the sense that *all* nontrivial minors of the rigidity matrix are required to take nonzero value at x. The framework at $x_1 = (0, 0), x_2 = (1, 0), x_3 = (0, 1)$ of a 3-clique provides an example of a generic framework which is not generic in the sense of Graver (its rigidity matrix has maximum rank 3 but the 3×3 minor given by columns 1, 3, 6 is nontrivial yet has value 0).

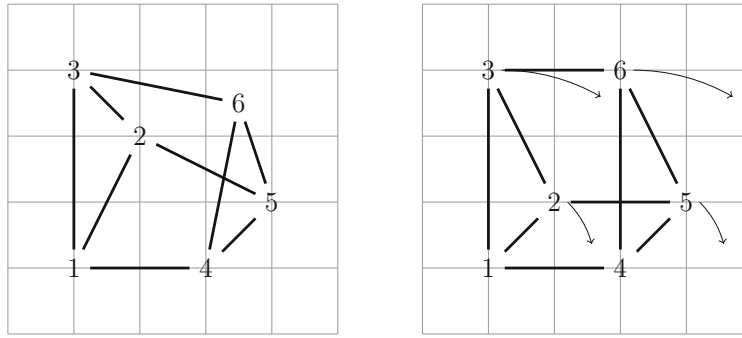


Fig. 7.7 Rigid graph with a generically rigid framework (left) and a flexible one (right) in the plane.

7.4.1 Graph rigidity on a line

Imagine a 2-clique (i.e., an edge) realized on a line: the only possible movements are translations, left or right; and the edge can only move left if both its adjacent vertices undergo the same translation: in short, any isometry is also a congruence. If a graph consists of two disconnected edges $\{1, 2\}$, $\{3, 4\}$, then each edge can be translated independently; since the distance between vertex 2 and 3 changes if the two translations are different, this is an isometry which is not a congruence. This should be sufficient intuitive evidence that graphs are rigid on a line if and only if they are connected.

Obtaining a purely combinatorial characterization of graph rigidity in the plane is more difficult; we shall require yet one more notion of rigidity.

7.4.2 General position

One further notion of genericity is that of general position. A set Z of points of $\mathbb{R}^K$ are in *general position* if, for $1 < K' \leq K$, no $K' + 1$ points in Z lie in an affine subspace having dimension strictly smaller than K', e.g., no three points on a line, no four points on a plane. Accordingly, a realization $x = (x_1, \ldots, x_n)$ is *general* if $x_1, \ldots, x_n \in \mathbb{R}^K$ are in general position.

The generality requirement is stronger[6] than genericity. The graph realization shown in Fig. 7.8 (where x_4 is in the origin and grid squares have unit side) is generic but not general: the rank of its rigidity matrix is 5, the maximum possible value for 5 edges, but x_1, x_2, x_4 are collinear in $\mathbb{R}^2$.

7.4.3 Abstract rigidity

For $K \in \{1, 2\}$, it is known how to establish graph rigidity as an inherent property of the graph $G = (V, E)$ [88]: we need not concern ourselves with the rigidity matrix and its rank. It is currently not known whether this is also possible for larger values of K.

The notion of *abstract rigidity* is intuitively based on modeling flexibility by means of a "hinge." Take $K = 3$ as an example: if we can partition E into sets E_1, E_2 that are incident to at most two common vertices u, v, then the subgraphs $G[E_1]$, $G[E_2]$ can rotate in 3D around the line (or "hinge")

[6] By contrast, any realization which is generic in the sense of Graver is also general [56].

defined by x_u, x_v (see Fig. 7.9, right). Conversely, if E_1, E_2 induce rigid subgraphs and are incident to at least three common vertices affinely spanning a plane, this is no longer possible (see Fig. 7.9, left).

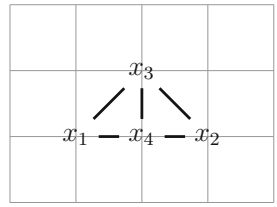


Fig. 7.8 Generic realization which is not general.

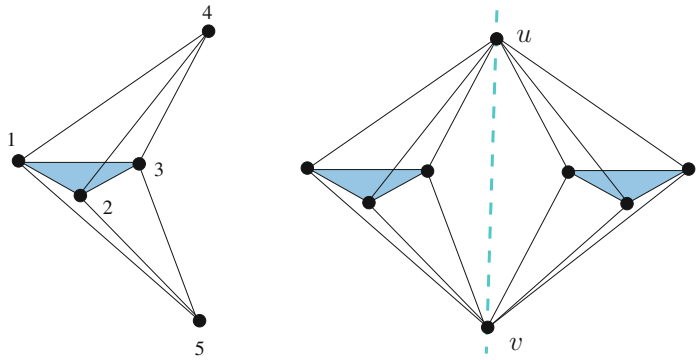


Fig. 7.9 Rigid single banana (left) and a flexible double banana (right).

At this point, it should be clear that rigidity is a property related to the rank of the rigidity matrix. In this section, we relate this to the rank of a matroid [57]. *Matroids* are abstractions of the row (or column) space of matrices, and they encode the concept of span, linear independence, bases, and rank from a purely combinatorial point of view. We shall define matroids formally by means of the closure, which plays the role of span in linear algebra.

Given a set E, a *closure* is a mapping $\langle\cdot\rangle : \mathcal{P}(E) \to \mathcal{P}(E)$, where $\mathcal{P}(E)$ is the set of subsets of E. $(E, \langle\cdot\rangle)$ is a *matroid* if the closure satisfies the following axioms:

1. the closure of a set is at least as large as the set:
   $\forall F \subseteq E\ (F \subseteq \langle F\rangle)$ (containment);
2. applying the closure twice is the same as applying it once:
   $\forall F \subseteq E\ (\langle F\rangle = \langle\langle F\rangle\rangle)$ (stability);
3. the closure preserves the relative containment of sets:
   $\forall F, G \subseteq E\ (F \subseteq G \to \langle F\rangle \subseteq \langle G\rangle)$ (monotonicity);
4. for any set and two elements not in the set, if the former is in the closure of the union of the set with the latter element, the latter element is in the closure of the set and the former element:
   $\forall F \subseteq E\ \forall e, f \in E \setminus F\ (e \in \langle F \cup \{f\}\rangle \to f \in \langle F \cup \{e\}\rangle)$ (exchange).

If $(E, \langle\cdot\rangle)$ is a matroid and $F \subseteq E$, F is *independent* if, for every $e \in F$, $e \notin \langle F \setminus \{e\}\rangle$. Maximal independent subsets of E are called *bases*. The *rank* of $F \subseteq E$ is the cardinality of any basis in F; it can be shown by the closure axioms (Axiom 1-4 above) that all bases have the same cardinality, as is the case in linear algebra. Note the similarity with linear algebra: a set of vectors F is independent if,

for any $e \in F$, the span of $F \setminus \{e\}$ does not contain e; a basis is a maximal linearly independent set of vectors and the rank is the cardinality of a basis.

We are now going to look at graphs $G = (V, E)$ as subgraphs of a clique on $V = \{1, \dots, n\}$. Since cliques are rigid graphs, this will afford us the opportunity of looking at any graph within a rigid framework. Let $G = (V, E)$ be a graph, d be an edge weight function on E, x be a regular realization of G in $\mathbb{R}^K$, and $\mathbf{K}(G) = (V, \bar{E})$ be the completion of (G, d, K, x). For $F \subseteq \bar{E}$, we shall denote by $\bar{F}$ the edge set of the completion of $G[F]$. Moreover, for $F \subseteq E$, let $V(F)$ be the set of vertices of V adjacent to the edges in F.

The *K-infinitesimal rigidity matroid* $\mathscr{M}(K, n)$ is the matroid of the rows of the rigidity matrix $\mathbf{R}(\mathbf{K}(G), x)$ of $\mathbf{K}(G)$. Notice that any realization x of G is also, by definition, a realization of its completion; but since the completion has more edges than G, its rigidity matrix has more rows than that of G. Since x is regular, it is generic, so $\mathscr{M}(K, n)$ only depends on K and n. Since $\mathscr{M}(K, n)$ is given by a matrix, it carries essentially the same information as its linear algebraic structure (linear independence, span, rank, etc.). The closure $\langle E \rangle$ of a subset of edges $E \subseteq \bar{E}$ is given by the indices $\{u, v\}$ of those rows of $\mathbf{R}(G, x)$ which are linearly dependent on the rows corresponding to the edges in E.

Let $F \subseteq E$ and x be a regular realization of $G[F]$ in $\mathbb{R}^K$, so that rigidity and flexibility are the same notions as their infinitesimal counterparts, and x is generic. Let R be the rigidity matrix of $(G[F], x)$; if $(G[F], x)$ is rigid, then the matrix R' obtained by adding any row

$$0, \dots, 0, x_u - x_v, 0, \dots, 0, x_v - x_u, 0, \dots, 0$$

to R, where $u, v \in V(F)$ and $\{u, v\} \notin F$, has the same rank as R. This is because $(G[F], x)$ can only move isometrically with the same degrees of freedom as its completion (which is rigid because it is a complete graph). Thus, if F induces a rigid graph, $\langle F \rangle = \bar{F}$. Conversely, for the same reason, if F induces a flexible graph, $\langle F \rangle \subsetneq \bar{F}$. Graver [56] used this closure property to prove that infinitesimal rigidity matroids of general realizations have two "hinge" properties with respect to any two edge sets $F, H \subseteq \bar{E}$: if F, H meet in fewer than K vertices, the closure of their union is contained in the union of their completions, and if F, H meet in at least K vertices and $\bar{F}, \bar{H}$ induce rigid graphs, their union induces a rigid graph. These properties can be used as two further matroid axioms.

If E is the edge set of a graph $G = (V, E)$ with $|V| = n$, K is an integer, and the matroid $(\bar{E}, \langle\cdot\rangle)$ obeys the following axioms:

5. $\forall F, H \subseteq \bar{E}\ (|V(F) \cap V(H)| < K \to \langle F \cup H \rangle \subseteq \bar{F} \cup \bar{H})$ (hinge);
6. $\forall F, H \subseteq \bar{E}\ (|V(F) \cap V(H)| \geq K \wedge \langle F \rangle = \bar{F} \wedge \langle H \rangle = \bar{H} \to \langle F \cup H \rangle = \overline{F \cup H})$ (rigidity),

then $(\bar{E}, \langle\cdot\rangle)$ is called the *abstract K-rigidity matroid*, and denoted $\mathscr{A}(K, n)$. Bear in mind that there exist abstract rigidity matroids that are not infinitesimal rigidity matroids [57, Cor. 4.6.1].

7.4.4 Laman's theorem

Laman's theorem [68] states that $G = (V, E)$ is rigid in the plane if and only if there is a spanning subset of $2|V| - 3$ edges $E' \subseteq E$, i.e., a set of edges adjacent to all vertices in V, such that following holds for $K = 2$:

$$\forall F \subseteq E' \quad |V(F)| \geq K \to |F| \leq K|V(F)| - \frac{K(K+1)}{2}, \tag{7.9}$$

or, in other words,

$$\forall F \subseteq E' \quad |F| \leq 2|V(F)| - 3. \tag{7.10}$$

It is known since Maxwell [92] that the independent sets of an abstract rigidity matroid satisfy Eq. (7.9) for $K \in \{2, 3\}$, and we know that this holds for any K [56]. Laman was able to prove the converse for $K = 2$, but a counterexample to this converse for $K = 3$ is given by the double banana shown in Fig. 7.9 (right). A result of Lovasz and Yemini also provides a polytime algorithm for testing graph rigidity in the plane [88].

Notwithstanding many attempts, no one has ever established a purely combinatorial criterion for establishing graph rigidity for values of K greater than 2. As such, this remains one of the great open challenges in DG.

7.5 Exercises

7.5.1 Exercise
Prove that the partial reflections in the pruning group $\mathscr{G}_P(G)$ are isometries.

7.5.2 Exercise
Prove that all congruences are isometries.

7.5.3 Exercise
With reference to Example 7.3.2, prove that no reflection can be a local isometry.

7.5.4 Exercise
Prove that cliques, quasi-cliques, $(K-1)$-trilaterative graphs, and ^{K}DMDGP graphs are rigid graphs in Euclidean spaces of appropriate dimensions.

7.5.5 Exercise
Show some algebraically dependent and regular realization of a 3-clique in $\mathbb{R}^2$.

7.5.6 Exercise
Use Eq. (7.9) to prove that connectedness is a combinatorial characterization of rigidity on a line.

Chapter 8
Approximate realizations

So far, we have always assumed that the input graph G is rigid, and that its structure consists essentially of a chain of $(K+1)$-cliques for some positive integer K. What if this is not the case? Small-sized graphs can be realized using the direct methods in Sect. 2.5.

In this chapter, we propose some methods for realizing large graphs. These methods do not explicitly rely on the adjacency structure of the graph, and are mostly based on linear algebra.

8.1 The weighted adjacency matrix

As a first step, we transform the given nonnegatively edge-weighted simple undirected graph $G = (V, E, d)$ into its *weighted adjacency matrix* $A_G = (a_{ij})$, where

$$\forall i, j \leq n = |V| \quad a_{ij} = \begin{cases} d_{ij} & \text{if} \quad \{i, j\} \in E \\ 0 & \text{otherwise.} \end{cases}$$

Notice that A_G is a $n \times n$ square matrix whose (i, j)-entry is nonzero if and only if i, j are adjacent vertices. Because G has no loops, A_G has zeroes along its diagonal; because G is undirected, A_G is a *symmetric matrix* (i.e., $a_{ij} = a_{ji}$ for each $i, j \leq n$).

8.2 Matrix completion

Notice that the off-diagonal zero elements of A_G correspond to unknown distances between pairs of vertices. If there is a valid realization x for G in $\mathbb{R}^K$, then all pairwise distances for G can be computed, and the off-diagonal (i, j)-th zero elements of A_G can be replaced by $\|x_i - x_j\|$ to obtain a full Euclidean *distance matrix*[1] (EDM) $D_x = (\|x_i - x_j\|)$ for G. In this sense, A_G can be seen as a *partial distance matrix*, and D_x as its *completion*.

The problem of completing a partial matrix so that it becomes an EDM can be cast in terms of weighted graphs as follows:

EUCLIDEAN DISTANCE MATRIX COMPLETION PROBLEM (EDMCP). Given a simple, connected, undirected weighted graph $G = (V, E, d)$, is there a positive integer K and a realization $x : V \to \mathbb{R}^K$ such that Eq. 2.2 holds?

[1] Some texts define distance matrices to contain the squared distances $\|x_i - x_j\|^2$—we would call this a *squared distance matrix* instead.

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_8

Note the similarity of the EDMCP and the DGP: the only difference is whether the integer K is given as part of the input or not. The DGP asks whether a given graph can be realized in a given dimension; the EDMCP asks whether a given graph can be realized in any dimension, or not at all. Of course, if G is a YES instance of the DGP, then it is also YES in the EDMCP, but the converse does not necessarily hold.

Although DGP and EDMCP look deceptively similar, the DGP is **NP**-hard (see Sect. 2.4.2), whereas the complexity status of the EDMCP is not known[2] [73].

8.3 Overall method structure

The general structure of the methods in this chapter in order to construct a realization in $\mathbb{R}^K$ for a given K is as follows:

1. guess an approximate completion $\tilde{D}$ of the weighted adjacency matrix A_G;
2. find an approximate realization $\tilde{x}$ of $\tilde{D}$ in some ambient space $\mathbb{R}^H$;
3. if $H > K$, project $\tilde{x}$ onto the desired space $\mathbb{R}^K$, so as to distort the given distances d as little as possible.

We name these steps *complete*, *realize*, and *project*. The last step is optional, and required only if $H > K$.

All three steps are carried out approximately, rather than precisely, and therefore yield errors. The quality of the end realization can be assessed by computing the discrepancy between its distance matrix and the given weighted graph. An average relative error measure on x with respect to A_G is given by:

$$\rho_G(x) = \frac{1}{|E|} \sum_{\{i,j\} \in E} \frac{|d_{ij} - \|x_i - x_j\|\,|}{d_{ij}}. \tag{8.1}$$

The smaller $\rho_G(x)$ is, the better x approximates a realization of G.

Approximate methods may yield realizations even for NO instances, and therefore cannot be used to prove that an instance is NO.

8.4 Approximate Completion Methods

We discuss methods for guessing a completion $\tilde{D} = (\tilde{d}_{ij})$ of A_G.

8.4.1 Constant completion

The simplest approximation is to simply consider that all distances which are not given are zero, i.e., $\tilde{D} = A_G$.

A slightly more accurate method is to set a missing distance at a value which is smaller than the sum over all pairs of edge weights that form a triangle with the missing pair: for each $\{i, j\} \notin E$, let $\tilde{d}_{ij} < \min\{d_{ik} + d_{kj} \mid \{i, k\}, \{k, j\} \in E\}$. This ensures that the chosen values satisfy the triangle inequality.

[2] Establishing the worst-case complexity class of EDMCP is another great open challenge in DG.

8.4.2 Shortest paths

A more accurate approximation is to set $\tilde{d}_{ij}$ to the length of the shortest weighted path from i to j in G. We employ the Floyd–Warshall algorithm [49] to compute all shortest paths in G (see Alg. 11).

Algorithm 11 $\tilde{D} = \mathsf{FloydWarshall}(G, d)$

```
// n × n array d̃_ij to store distances
d̃ = 0
for {i, j} ∈ E do
  d̃_ij = d_ij
end for
for k ∈ V do
  for j ∈ V do
    for i ∈ V do
      if d̃_ik + d̃_kj < d̃_ij then
        // d̃_ij fails to satisfy triangle inequality, update
        d̃_ij = d̃_ik + d̃_kj
      end if
    end for
  end for
end for
```

8.5 Approximate realization methods

Realizing a complete graph G with n vertices in $\mathbb{R}^{n-1}$ can be done using Alg. 2, but the weighted adjacency matrix of G must be a very good approximation of a Euclidean distance matrix. By contrast, the EDM approximations proposed in Sect. 8.4 can be very poor. Under these conditions, Alg. 2 would simply conclude that the instance is infeasible.

8.5.1 Classic Multidimensional Scaling

In this section, we discuss an error-tolerant method for realizing weighted cliques, called *classic multidimensional scaling* (or classic MDS) [30], which is often used in statistics and data mining.

8.5.1.1 The Gram matrix in function of the distance matrix

Given a realization $x \in \mathbb{R}^{Kn}$, its *Gram matrix* is the square matrix $B = x^\top \cdot x$ having $x_i \cdot x_j$ as its (i, j)-th component. We show that a squared EDM $D^2 = (d_{ij}^2)$ can be efficiently transformed into the Gram matrix of the corresponding realization.

Let $G = (V, E, d)$ be a nonnegatively weighted complete graph with $n = |V|$, and let L be the smallest positive integer such that G has a valid realization x in $\mathbb{R}^L$. We first translate x so that its barycenter is at the origin:

$$\sum_{i \le n} x_i = 0. \tag{8.2}$$

Now we remark that, for each $i, j \leq n$, we have

$$d_{ij}^2 = \|x_i - x_j\|^2 = (x_i - x_j) \cdot (x_i - x_j) = x_i \cdot x_i + x_j \cdot x_j - 2x_i \cdot x_j. \tag{8.3}$$

Next, we "invert" Eq. (8.3) to express $x_i \cdot x_j$ in function of d_{ij}^2. We sum Eq. (8.3) over all values of $i \in \{1, \ldots, n\}$, obtaining:

$$\sum_{i \leq n} d_{ij}^2 = \sum_{i \leq n} (x_i \cdot x_i) + n(x_j \cdot x_j) - 2 \left(\sum_{i \leq n} x_i \right) \cdot x_j. \tag{8.4}$$

By Eq. (8.2), the rightmost term in the right-hand side of Eq. (8.4) is zero. On dividing through by n, we have

$$\frac{1}{n} \sum_{i \leq n} d_{ij}^2 = \frac{1}{n} \sum_{i \leq n} (x_i \cdot x_i) + x_j \cdot x_j. \tag{8.5}$$

Similarly for $j \in \{1, \ldots, n\}$, we obtain:

$$\frac{1}{n} \sum_{j \leq n} d_{ij}^2 = x_i \cdot x_i + \frac{1}{n} \sum_{j \leq n} (x_j \cdot x_j). \tag{8.6}$$

We now sum Eq. (8.5) over all j, getting:

$$\frac{1}{n} \sum_{\substack{i \leq n \\ j \leq n}} d_{ij}^2 = n \frac{1}{n} \sum_{i \leq n} (x_i \cdot x_i) + \sum_{j \leq n} (x_j \cdot x_j) = 2 \sum_{i \leq n} (x_i \cdot x_i) \tag{8.7}$$

(the last equality in Eq. (8.7) holds because the same quantity $f(k) = x_k \cdot x_k$ is being summed over the same range $\{1, \ldots, n\}$, with the symbol k replaced by the symbol i first and j next). We then divide through by n to get:

$$\frac{1}{n^2} \sum_{\substack{i \leq n \\ j \leq n}} d_{ij}^2 = \frac{2}{n} \sum_{i \leq n} (x_i \cdot x_i). \tag{8.8}$$

We now rearrange Eqs. (8.3), (8.5), (8.6) as follows:

$$2x_i \cdot x_j = x_i \cdot x_i + x_j \cdot x_j - d_{ij}^2 \tag{8.9}$$

$$x_i \cdot x_i = \frac{1}{n} \sum_{j \leq n} d_{ij}^2 - \frac{1}{n} \sum_{j \leq n} (x_j \cdot x_j) \tag{8.10}$$

$$x_j \cdot x_j = \frac{1}{n} \sum_{i \leq n} d_{ij}^2 - \frac{1}{n} \sum_{i \leq n} (x_i \cdot x_i), \tag{8.11}$$

and replace the left-hand side terms of Eqs. (8.10)–(8.11) into Eq. (8.9) to obtain:

$$2x_i \cdot x_j = \frac{1}{n} \sum_{k \leq n} d_{ik}^2 + \frac{1}{n} \sum_{k \leq n} d_{kj}^2 - d_{ij}^2 - \frac{2}{n} \sum_{k \leq n} (x_k \cdot x_k), \tag{8.12}$$

whence, on substituting the last term using Eq. (8.8), we have

$$2x_i \cdot x_j = \frac{1}{n}\sum_{k\leq n}(d_{ik}^2 + d_{kj}^2) - d_{ij}^2 - \frac{1}{n^2}\sum_{\substack{h\leq n\\ k\leq n}} d_{hk}^2. \tag{8.13}$$

It turns out that Eq. (8.13) can be written in matrix form as follows:

$$B = -\frac{1}{2} J \cdot D^2 \cdot J, \tag{8.14}$$

where $J = I_n - \frac{1}{n}\mathbf{1}\cdot\mathbf{1}^\top$ and $\mathbf{1} = \underbrace{(1,\ldots,1)}_{n}$.

The Gram matrix B has three important properties: (i) it is a symmetric matrix; (ii) $\mathsf{rk}\, B = \mathsf{rk}\, x$ (recall that x is a $K \times n$ matrix); and (iii) B is PSD, i.e., $y^\top \cdot B \cdot y \geq 0$ for all $y \in \mathbb{R}^n$. Both properties are very easy to establish in the case when $x = (x_1, \ldots, x_n)$ is a $1 \times n$ matrix, i.e., $x \in \mathbb{R}^n$, and x_i is a scalar for all $i \leq n$: (i) the i-th column of B is the vector x multiplied by the scalar x_i, which means that every column of B is a scalar multiple of a single column vector, and hence that $\mathsf{rk}\, B = 1$; (ii) $y^\top \cdot B \cdot y = y^\top \cdot (x \cdot x^\top) \cdot y = (y^\top \cdot x) \cdot (x^\top \cdot y) = (x^\top \cdot y)^2 \geq 0$.

In fact, B is a Gram matrix if and only if it is PSD; moreover, a matrix is PSD if and only if all its eigenvalues are nonnegative.

8.5.1.2 The positive eigenspace of the Gram matrix

Having computed the Gram matrix B from the EDM D using Eq. (8.13), we obtain the corresponding realization x as follows.

Let $\Lambda = \mathsf{diag}(\lambda_1, \ldots, \lambda_L)$ be the $L \times L$ matrix with the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_L$ of B along the diagonal and zeroes everywhere else, and let Y be the $n \times L$ matrix having the eigenvector corresponding the eigenvalue λ_j as its j-th column (for $j \leq L$), chosen so that Y consists of orthogonal columns. Then, $B = Y \cdot \Lambda \cdot Y^\top$ (see Sect. A.5.8). Since Λ is a diagonal matrix and all its diagonal entries are nonnegative (because B is PSD), we can write Λ as $\sqrt{\Lambda} \cdot \sqrt{\Lambda}$, where $\sqrt{\Lambda} = \mathsf{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_L})$. Now, since $B = x^\top \cdot x$, we have

$$x^\top \cdot x = B = (Y \cdot \sqrt{\Lambda}) \cdot (\sqrt{\Lambda} \cdot Y^\top),$$

which implies that

$$x = \sqrt{\Lambda} \cdot Y^\top \tag{8.15}$$

is a realization of G in $\mathbb{R}^L$. In practice, it might happen that some of the eigenvalues of B are very small floating-point numbers. Assume $H < L$ eigenvalues of B are positive and $L - H$ are considered zero. This will yield $L - H$ almost zero rows at the bottom of x: these can be removed, providing a realization x of G in $\mathbb{R}^H$.

8.5.1.3 Approximation and dimensionality

The most interesting feature of MDS is that it adapts well to approximate distance matrices $\tilde{d}$. We can still compute $\tilde{B} = -\frac{1}{2} J \cdot \tilde{D}^2 \cdot J$, where $\tilde{D}^2 = (\tilde{d}_{ij}^2)$ as above. In this case, $\tilde{B}$ will probably fail to be a Gram matrix, and as such might have negative eigenvalues. But it suffices to let Y be the eigenvectors

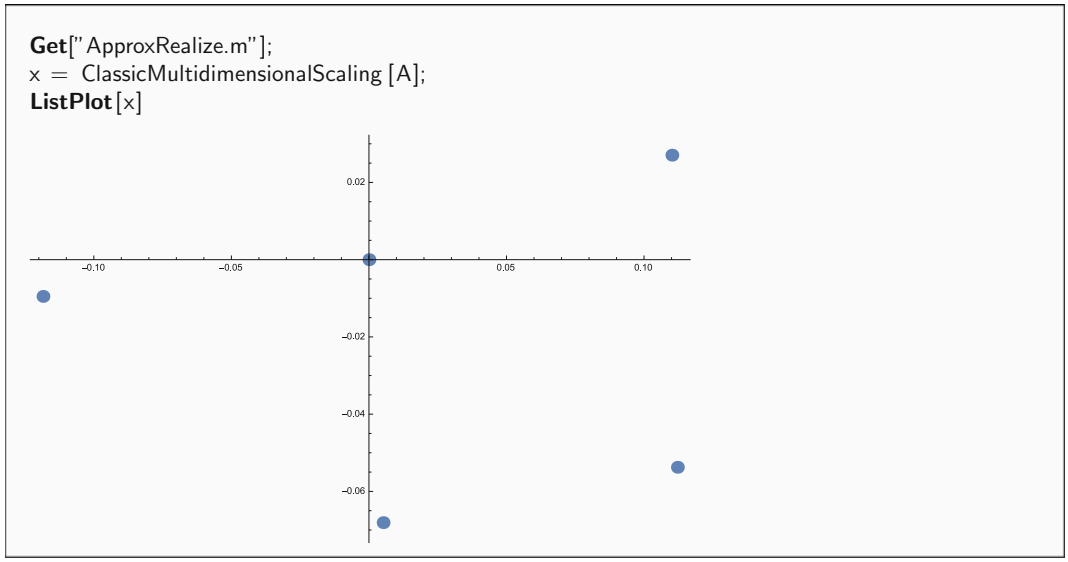


Fig. 8.1 Classic MDS, applied to the matrix A of genetic differences between populations in different continents, yields a realization of the closest distance matrix of low rank. In this case, the realization x has rank 2, and is shown in the *plot*.

corresponding to the H positive eigenvalues $\lambda_1, \ldots, \lambda_H$, to recover an approximate H-dimensional realization x of G.

Another interesting feature of MDS is that the dimensionality H of the ambient space of x is actually determined by d or $\tilde{d}$ rather than given as a problem input. In other words, MDS finds the "inherent dimensionality" of a set of pairwise distances. As such, MDS can be seen as an approximate method for solving the EDMCP.

8.5.1.4 Visualizing data

The classic application of classic MDS is in data visualization: MDS approximates a given matrix to a close distance matrix of low rank. We showcase this using a sample of genetic difference data, shown below, obtained on the `gendist` program Web page,[3] and a corresponding realization where $H = 2$.

	European	African	Chinese	American	Australian
European	0.000000	0.078002	0.080749	0.066805	0.103014
African	0.078002	0.000000	0.234698	0.104975	0.227281
Chinese	0.080749	0.234698	0.000000	0.053879	0.063275
American	0.066805	0.104975	0.053879	0.000000	0.134756
Australian	0.103014	0.227281	0.063275	0.134756	0.000000

The *Mathematica* instructions to obtain a realization of this difference data in $\mathbb{R}^2$ are given in Fig. 8.1.

[3] http://evolution.genetics.washington.edu/phylip/doc/gendist.html.

8.5.2 Proximity adjustment

Suppose some approximate realization x of G is known. Then, given some initial "learning rate" parameter $\lambda < 1$ and iteration bounds C (cycles) and S (steps), Alg. 12 below may improve its quality. This approximate realization improvement algorithm is used as part of the Stochastic Proximity

Algorithm 12 $x = \mathsf{ProximityAdjustment}(G, x, \lambda, C, S)$

```
i = 1, j = 1
while i ≤ C do
  while j ≤ S do
    let {u, v} be a random edge in E
    Δ = λ (|‖x_u − x_v‖ − d_uv| / ‖x_u − x_v‖) (x_u − x_v)
    x_u = x_u + Δ
    x_v = x_v − Δ
    j = j + 1
  end while
  update (decrease) the learning rate λ
  i = i + 1
end while
```

Embedding (SPE) method [123]. It works by using a scaled discrepancy Δ between $\|x_u - x_v\|$ and d_{uv} in order to either increase or decrease the distance between x_i and x_j. This adjustment step is carried out repeatedly (using the same learning rate λ) for S steps, after which the learning rate is updated. The algorithm terminates after C learning rate updates. Popular update strategies are $\lambda \leftarrow \lambda^2$ and $\lambda \leftarrow \lambda - \delta$, where $\delta \ll \lambda$ is a fixed prescribed value.

8.6 Approximate projection methods

So far, for a given input G, we showed how to complete the edge weights d to an approximate EDM $\tilde{d}$, and then how to compute an approximate realization x of the clique weighted by $\tilde{d}$ in $\mathbb{R}^H$, where H is the number of positive eigenvalues of the approximate Gram matrix of x, which is itself a function of $\tilde{d}$. In other words, we still lack the ability to find a realization in $\mathbb{R}^K$ for a *given* integer K. Since K is usually smaller than H, this can be addressed by projection methods.

8.6.1 Principal Components Analysis

The first method we discuss fits in very neatly with MDS. Given a set of n data points $x_1, \ldots, x_n$ in $\mathbb{R}^H$ represented as a $K \times n$ matrix $x = (x_{ij})$, the K *principal components* are the first K eigenvectors of the Gram matrix B of x, ordered by decreasing eigenvalues. This methodology is broadly known as Principal Component Analysis (PCA) [63]. In this sense, we can see PCA as classic MDS (see Sect. 8.5) applied to the first K eigenvectors of B.

8.6.2 Gaussian random projections

The second method may sound somewhat surprising. Let P be a $K \times H$ (with $K < H$) matrix where each component is sampled independently from a Gaussian distribution with zero mean and unit standard deviation. If y is a realization of G in $\mathbb{R}^H$, then it is likely that $x = \frac{1}{\sqrt{K}} P \cdot y$ is a good approximation of y in $\mathbb{R}^K$, in the sense that for any $i < j \leq n$, $\|x_i - x_j\| \approx \|y_i - y_j\|$.

Intuitively, the reason for this is best expressed in [32]:

> The squared length of a random vector is sharply concentrated around its mean when the vector is projected onto a random K-dimensional subspace.

When this principle is applied to the difference vectors $x_i - x_j$ (for $i, j \leq n$), then one obtains the preservation of Euclidean distances when projecting to a lower dimensional subspace.

8.6.3 The Johnson–Lindenstrauss lemma

The approximation quality of Gaussian random projections can only be guaranteed rigorously if more assumptions are made. The Johnson–Lindenstrauss (JL) lemma [62] states that, for a given $\varepsilon \in (0, \frac{1}{2})$, then $\frac{1}{\sqrt{K}} P$ projects y to x such that:

$$\forall i, j \leq n \quad (1 - \varepsilon)\|y_i - y_j\|^2 \leq \|x_i - x_j\|^2 \leq (1 + \varepsilon)\|y_i - y_j\|^2$$

if K is $O(\frac{1}{\varepsilon^2} \log n)$. Notice that K does not depend on H but only on n and ε. The logarithmic dependence on n is a desirable feature of JL, but if one also requires a small error tolerance ε, then $\frac{1}{\varepsilon^2}$ can be a very large number. In other words, JL tells us that Gaussian random projections are good for large sets of points in an arbitrarily huge number of dimensions, as long as one is willing to forego extreme accuracy.

8.6.3.1 An application of the JL lemma

The JL lemma can help us devise an efficient clustering method for the first n results of a *Google Images* query.

Let us first explain what we mean by this. Google tags images with words and sentences: some are found in the image meta-data (which could have been written either by a human or by a machine), and some are assigned to the images by many Machine Learning (ML) methods devised and/or implemented at Google. The user inputs a search string s, and a standard Google ranking [69] is performed over the image tags with respect to s. Because of the limited capacity that current ML algorithms have in the field of semantic context pinning, but also because tags assigned by humans could be entirely inappropriate with a certain probability, it is likely that among the first n ranked images there will be images of different types (see Fig. 8.2). Hence, the need for clustering with respect to the image bitmap data presenting 3–4 cluster representatives would provide a more synthetic and informative answer than the whole list of n images (note that *Google Images* already does this: see the top row of the image results array in Fig. 8.2).

Since we are showcasing an application of the JL lemma, which works for Euclidean norms, we propose to cluster the image data according to the Euclidean norm. Images are arrays of pixels, each pixel being defined by a number encoding its color. For simplicity, we assume the images all have the same size (an $\sqrt{H} \times \sqrt{H}$ pixel grid for some integer H which we assume to be a square, for example,

$\sqrt{H} = 300$) and color depth (say 24 bits, organized in three 8-bit sequences for the red, green, and blue components, each of which is a floating-point number in $[0, 1]$). In summary, we now have a set X of n vectors in $[0, 1]^{270000}$.

Let us now turn our attention to the clustering algorithm on the set X. To keep things simple, we consider what is possibly the most popular Euclidean clustering heuristic, called *k-means*, which assigns data points to k clusters and identifies their representatives:

1. randomly pick a subset $Y \subseteq X$ such that $|Y| = k$;
2. for each $x_i \in X$, assign x_i to the closest $y_j \in Y$ in the Euclidean norm
   let $z_{ij} = 1$ and $z_{i\ell} = 0$ for each $\ell \neq j$;
3. update vectors in Y as follows:

$$\forall y_j \in Y \quad y_j = \frac{\sum\limits_{i \leq n} z_{ij} x_i}{\sum\limits_{i \leq n} z_{ij}}$$

4. repeat from Step 2 until z and Y stop changing.

The vectors in Y are the *centroids* of the k clusters, and the binary variables z encode the assignment of data vectors in X to each cluster. The k-means algorithm repeatedly assigns existing data vectors to the cluster having the closest centroid, and then updates the centroids to reflect the change in the assignment. The algorithm stops when the clustering becomes stable, but it can also be stopped on the basis of CPU time or number of iterations. Since k-means does not guarantee the optimality of the solution, it is a heuristic algorithm, which is known to be practically very efficient. The Euclidean clustering problem is **NP**-hard, so any exact algorithm would be inefficient unless $\mathbf{P} = \mathbf{NP}$.

With the $n = 11$ images shown in Fig. 8.2, the *Mathematica* implementation of k-means with $k = 3$ performs as shown in Fig. 8.3. The images, contained in the array `Himg`, were downloaded from *Google Images*, rescaled to 300×300, and transformed into vectors having 270000 components in $[0, 1]$.

The computational per-iteration cost of the k-means algorithm is $O(knH)$. In this product, k is likely the smallest integer, followed in size by n and then H (this is because images are usually large arrays of pixels). We are going to use the JL lemma to obtain approximately the same results by considerably reducing H. To this end, we pick a reasonable approximation factor $\varepsilon \in (0, 0.5)$, say $\varepsilon = 0.1$. The low dimension K is described above to be $O(\varepsilon^{-2} \log n)$, but in [119], we find some indication that a multiplicative constant of $C = 1.8$ is appropriate. So we set $K = \lfloor C\varepsilon^{-2} \log n \rceil = 432$. Since $3H = 3 \times 300^2 = 270000$, the computational savings should be noticeable. We form the scaled

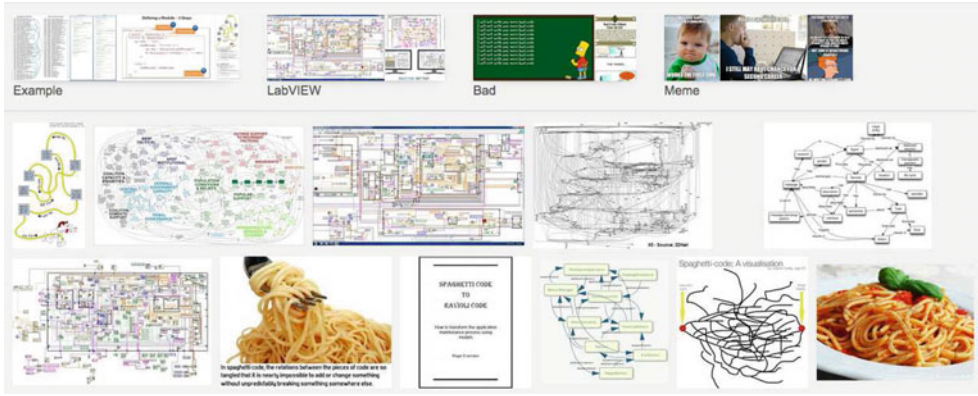


Fig. 8.2 *Google Images* search for the term "spaghetti code" with $n = 11$ (the first line lists four cluster representatives; the search matches are listed from the second line onward).

```
VHimg = Map[Flatten[ImageData[#]] &, Himg];
```



```
VHcl = Timing[ClusteringComponents[VHimg, 3, 1]]

{0.405908, {1, 2, 2, 2, 2, 2, 3, 2, 2, 2, 3}}
```

Fig. 8.3 The k-means algorithm is the default method of the `ClusteringComponents` function. By calling it as an argument to `Timing`, we obtain the CPU time spent to find the three clusters: most of the pictures are in cluster 2, with two small clusters: cluster 1 only contains the first picture, whereas cluster 3 contains the two spaghetti dishes.

```
Get["Projection.m"];
VKimg = JohnsonLindenstrauss[VHimg, 0.1];
VKcl = Timing[ClusteringComponents[VKimg, 3, 1]]

{0.002232, {1, 2, 2, 2, 2, 2, 3, 2, 2, 2, 3}}
```

Fig. 8.4 Running k-means on the lower dimensional random projections of the image vectors yields the same results in a fraction of the time.

Gaussian projection matrix $(1/\sqrt{K})\,P$ and apply it to the $n = 11$ image vectors, obtaining the same clustering as before in a fraction of the time (see Fig. 8.4).

8.7 Isomap

The Isomap algorithm was proposed in [117] to find realizations of weighted graphs: it completes A_G to an approximate EDM $\tilde{D}$ by means of weighted shortest paths (see Sect. 8.4.2), and then uses classic MDS to realize $\tilde{D}$ and PCA to project the realization to $\mathbb{R}^K$.

Isomap puts together three of the methods presented so far. Its efficacy in reconstructing a "spiral graph" (see Fig. 1.2) is shown in Fig. 8.5.

8.8 Stochastic Proximity Embedding

The SPE method, as described in [123], really only consists of a proximity improvement (Alg. 12) applied to a randomly generated realization. Several variants are possible to boost performance, such as applying a completion method, compute an initial approximate realization in $\mathbb{R}^K$ using some of the methods described above, and only then proceed to run Alg. 12 (see Fig. 8.6).

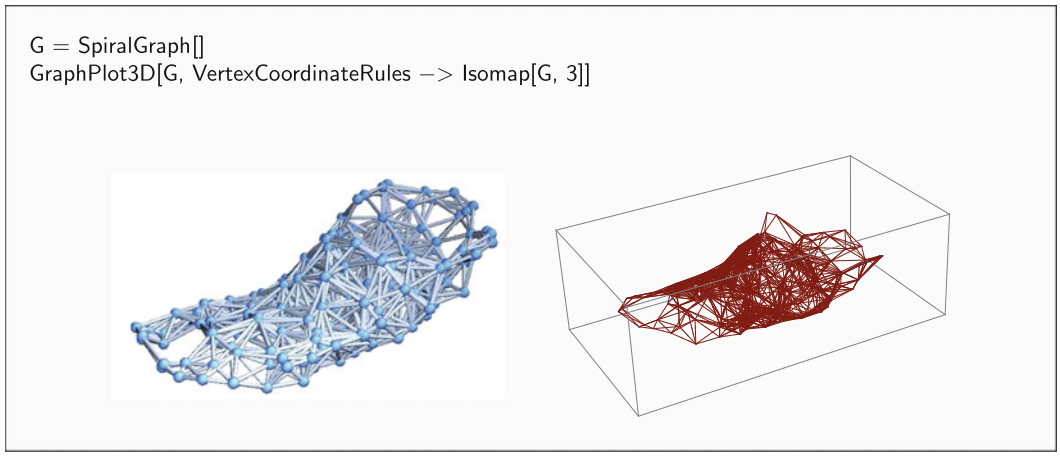


Fig. 8.5 *Mathematica*'s implementation of the Isomap algorithm. The "sock-shaped" triangulated surface on the right is a 3D realization of the weighted graph G on the left.

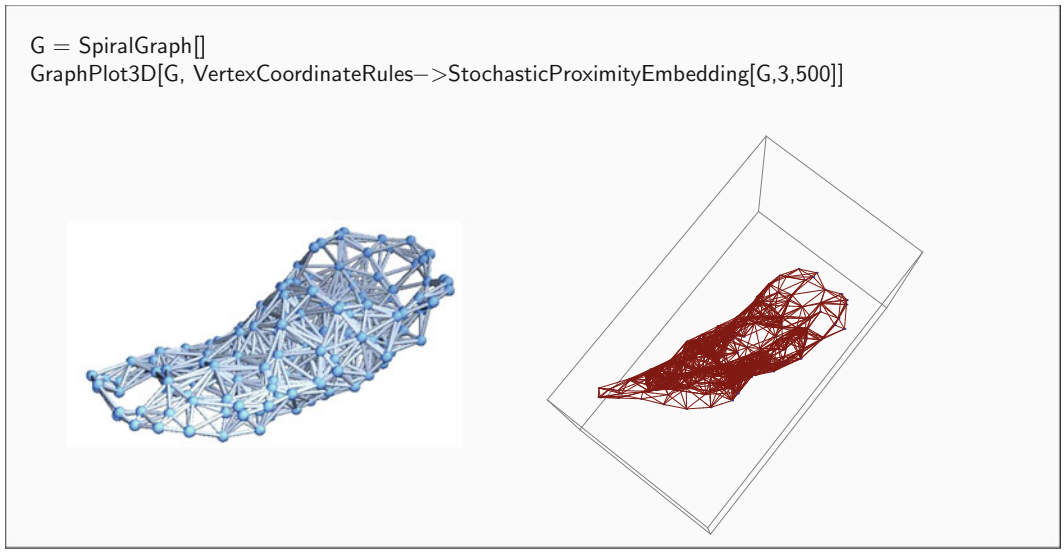


Fig. 8.6 *Mathematica*'s implementation of the SPE algorithm, acting on the same graph as in Fig. 8.5.

8.9 Exercises

8.9.1 Exercise
Prove that if G is a NO instance of the EDMCP, then (G, K) is a NO instance for the DGP for any K.

8.9.2 Exercise
Prove that Eq. (8.13) $\Leftrightarrow$ Eq. (8.14).

8.9.3 Exercise
Prove that for each $K \times n$ matrix $x = (x_{ij})$, the Gram matrix of x has rank $\mathsf{rk}\, x$.

8.9.4 Exercise
Prove that all Gram matrices are PSD.

8.9.5 Exercise
Prove that all PSD matrices are Gram matrices of some set of vectors.

8.9.6 Exercise
Prove that a symmetric matrix is PSD if and only if all its eigenvalues are nonnegative.

8.9.7 Exercise
Show that the worst-case complexity of the k-means algorithm over any iteration is $O(knH)$.

Chapter 9
Taking DG further

We wrote this last chapter in order to give our readers some sense of the direction that the DG field is taking today.[1]

The sketch we gave in Sect. 1.1 of the history of DG, up to when the use of computers became predominant, is rather fragmented over time: heron of Alexandria in Egypt around 100 AD, Euler's conjecture in 1766, Cauchy's early proof in 1813, and then the work of Cayley (1841) and Maxwell (1864), followed by that of Menger (1928) and Schoenberg (1935). We also mentioned Connelly's counterexample to Euler's conjecture (1978) and the lemma of Johnson and Lindenstrauss (1984). We might also split our historical sketch in two broad topics: results related to the DGP (Heron, Cayley, Menger, Schoenberg, Johnson–Lindenstrauss) and results related to rigidity (Euler, Cauchy, Maxwell, Connelly). This fragmentation reflects the current status of research in DG, i.e., it is fragmented by application.

We ascribe the turning point from "history of DG" to "contemporary research in DG" to a conference paper of Y. Yemini, published in 1978 with the extremely cautious title of *The positioning problem—A draft of an intermediate summary*. For the first time,[2] the crucial problem of DG went from "given *all* pairwise distances, find the positions of the points" to "given *some* of the pairwise distances, find the positions of the points." The issue is that the first problem can be solved efficiently, while the second cannot. So, while there was some hope of being able to solve the first problem essentially by hand, for the second problem this was impossible.

Yemini's paper was motivated by the application of DG to the localization of wireless sensor networks. A few years later, the chemical breakthrough which won Wüthrich the Nobel Prize [121, 122] introduced DG to structural biology. Rigidity notions are being used for statics [105], but also for robotics [106], controlled of unmanned underwater vehicles [5], and material science [13, 43, 118]. This fragmentation by application field meant that almost the same definitions and results were being rediscovered many times, using different notations and slightly different points of view, each tuned to the specific application. It also meant that the different research communities were mostly split "by application," as researchers attended conferences within their own application field.

We have tried, over the last few years, to "regroup" DG at its core, essentially via two actions: publications and workshops. We published [84, 86] and commissioned [13] cross-application surveys, and edited a cross-application book [102] as well as two special issues [75, 98]. We organized three

[1] To be totally honest, we wrote this chapter at the suggestion of our insightful editor at Springer, Dr. Christian Rauscher, who could not help noticing how abruptly our manuscript originally ended.

[2] We established this turning point to have happened as late as 1978 with surprise. Accordingly, we welcome any correction to this statement.

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4_9

cross-application workshops: one[3] in the Amazon forest in 2013, one[4] in Campinas, Brazil, and one[5] at the DIMACS applied mathematics center at Rutgers University in New Jersey, USA.

These efforts gave us a good (though certainly not exhaustive) view on the whole DG field today. We can establish four main cross-application branches of DG.

1. *Modeling*. The use of DG as a convenient modeling framework for many signal processing-type problems.
2. *Theory*. The use of rigidity techniques in order to establish (or deny) the uniqueness of solution of a given problem variant.
3. *Combinatorial methods*. Combinatorics plays two roles in DG: analysis and solution methods. There are combinatorial tools for counting or bounding the number of solutions, for example, as well as combinatorial methods for constructing realizations of given graphs.
4. *Optimization-based methods*. The use of optimization methods in order to compute positions from distances, particularly in the presence of noisy or inexact data.

In the rest of this chapter, we shall give an brief and nonexhaustive overview of each branch. We stress that our point of view stems from our above-cited experience in editing publications and organizing workshops. As such, not every aspect of current research in DG is represented.

9.1 Modeling signal processing problems

The DGP is a good example of an inverse problem. Its direct counterpart is: given n points in $\mathbb{R}^K$, find (some of) the pairwise distances. Most often, solving direct problems is much easier than solving inverse problems. Direct problems usually involve evaluating a given function, whereas inverse problems require the construction of some kind of inverse function. The DGP is also a good model for many *signal processing* problems. Signal processing is an inverse problem that consists in retrieving information from a physical signal. Its direct counterpart is: given the information, encode it in a signal that can be passed to a physical device for further processing or transmission. Since information is more important to us than the way it is encoded, and given that the encoding could be lossy and error-prone, it is most often the case that encoding (direct problem) is easier than decoding (inverse problem).

For example, one can use noise sources and microphones in order to estimate the shape of a room from the echos [39]: consider a noise source ν and a microphone μ placed in a room. A "ping" emitted from ν will reach μ using many different routes: the direct one, of course (which will have highest intensity), but also reflected (echoed) by the room walls. One can use the intensity and time lags of the direct and echoed pings in order to estimate the spatial positions where the noise reflection must have taken place. Placing sufficiently many microphones will guarantee that the correct signal (i.e., the shape of the room) can be reconstructed. This can be used in controlling a swarm of drones flying indoors, in an unknown environment, as well as for self-calibrating microphones when they are placed in different concert halls.

Another example of signal processing problem that is relevant to DG is the recovery of signals from the measurement of differences, or distances, whose value is known to be periodic. The simplest example is clock synchronization, whenever the time difference is taken modulo a second, minute, hour, or day [110]. Other applications in higher dimensions come from tomography, e.g., from images obtained with cryo-electron microscopy [7, 111].

[3] http://dga2013.icomp.ufam.edu.br/.

[4] http://www.ime.unicamp.br/workshop_distances/.

[5] http://dimacs.rutgers.edu/Workshops/Distance/.

9.2 Theory of solution uniqueness

Many DGP applications arise in situations when one would like to retrieve the configuration of entities, existing in the real world, after having measured some of the pairwise distances. If too few distances are measured, many incongruent configurations of points will satisfy the distances, and there will be ambiguity in choosing which configuration actually models reality. It is therefore desirable to establish whether a given DGP instance will give rise to many incongruent solutions or just one. If the solution is unique, we can simply consider it as a good model of reality.

Graphs yielding unique realizations in this sense are related to the notion of rigidity, explored in Chap. 7. Given that the notion of rigidity is used slightly differently in different DG applications, many definitions were introduced: some relating to mathematical analysis, some to mechanics, others to linear algebra, matroids, and so on. Verifying these definitions is sometimes easy and sometimes less so.

One of the foremost interests of the *graph rigidity* community is to devise definitions of solution uniqueness that are comprehensive, general, and yet easy to verify. See [1, 19, 28, 55, 57, 61, 113, 116] to find some of the people working in rigidity today.

9.3 Combinatorial methods

Since DGP is essentially a problem on graphs, it should not be surprising that combinatorics play a prominent role. From our (biased) point of view, the Branch-and-Prune algorithm ([83] and Sect. 4.4.3) is a by now well-studied mixed-combinatorial algorithm which gave rise to several different research directions: for example, finding and understanding vertex orders to improve solution quality and performance [23, 29, 54, 76, 97], studying the impact of partial reflection symmetry on the algorithm itself [87, 99], counting the number of solutions of the corresponding DGP instances without actually having to solve the instance [82], and more [80, 84].

Counting realizations of rigid graphs is also studied in more general settings [45]. Another type of combinatorial solution method for the DGP is geometric build-up, see [21, 34, 42, 112, 120]: this method is similar to Branch-and-Prune in that it uses K-lateration as a tool to find the position of the next vertex.

Within the theoretical computer science community, there is a lot of interest in the complexity and approximability of the DGP and its variants, see, e.g., the papers in the special issue [98].

9.4 Optimization-based solution methods

Although the DGP is a pure feasibility problem, many solution methods are based on optimization techniques. A first wave of methods (mostly proposed in the last century) were based on Eq. 2.6 [96]. More recently, the optimization method of choice for solving DGPs is based on semidefinite programming (SDP), through the connection between EDMs and PSD matrices established in Sect. 8.5.1.1. This link gave rise to many works in DG from the SDP community, see for example [2, 8, 14, 15, 37, 38, 66, 70–73, 90].

9.5 Debitum Gratitudinis (DG)

As our introduction to DG comes to an end, we would like to thank all of our coauthors who have been working with us on Distance Geometry: Agostinho Agra, Jorge Alencar, Rafael Alves, Benjamin Bardiaux, Nathanael Beeker, Simon Billinge, Tibérius Bonates, Guillaume Bouvier, Luiz Mariano Carvalho, Ricardo Carvalho, Andrea Cassioli, Virgínia Costa, Claudia D'Ambrosio, Gustavo Dias, Phillip Duxbury, Rosa Figueiredo, Weber Figueiredo, Stéphane Gaubert, Christian Glusa, Douglas Gonçalves, Warley Gramacho, Oktay Günlük, Vu Khac Ky, Jon Lee, Audrey Lee-St. John, Weldon Lodwick, Nelson Maculan, Thérèse Malliavin, Fabrizio Marinelli, Simone Martins, Benoît Masson, Tiago Mendonça da Costa, Antonio Mucherino, Albert Muritiba, Michael Nilges, Loana Nogueira, Pedro Nucci, António Pereira, Antonio Petraglia, Fábio Protti, Cristina Requejo, Germano Rezende, Grzegorz Swirszcz, Silas Sallaume, Luiz Satoru Ochi, Michael Souza, Maxim Sviridenko, Adilson Elias Xavier.

We like to single out Douglas Gonçalves for extra-special thanks for his patience in going over our manuscript and writing a solution manual for the exercises in this book.

We are grateful to the anonymous referees who have been helping us improve our papers. We are grateful to our editors at Springer: Elizabeth Loew, Razia Amzad, Christian Rauscher, for helping us making this book better. We are grateful to DIMACS at Rutgers University for cofunding and hosting one of the Distance Geometry workshops. Finally, we are grateful to the funding agencies who have been supporting us along the years: ANR (France); CNPq, CAPES, FAPERJ, FAPESP (Brazil), NSF (USA).

Appendix
Mathematical notions

This appendix is a review of notions the reader should already be familiar with: it is brief and to the point and mostly just provides formal definitions to be used in the book. We assume that the reader is acquainted with the concept of a *set*.[1]

A.1 Set cardinality

Sets can have finite or infinite cardinality; sets of the latter type can be countably or uncountably infinite. A set X is countably infinite if there is a bijection $X \leftrightarrow \mathbb{N}$. Among the sets of numbers, $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ are countably infinite. An infinite set is uncountably infinite if it is not countably infinite; $\mathbb{R}$ and $\mathbb{C}$ are examples of uncountably infinite sets. See [65, Chap. 1] for more information, and [67] for a formal treatment.

A.2 Some notation

We employ the following shorthand notations:

- $\forall$: for all, for each, for any;
- $\exists$: there is, there exists;
- $\exists!$: there is only one, there is a unique;
- $\wedge$: and;
- $\vee$: or (not the exclusive or—$A \vee B$ means either A or B or both);
- $\neg$: not;
- $\rightarrow, \Rightarrow$: implies;
- $\leftarrow, \Leftarrow$: is implied by;
- (): determine operation precedence.

If x, y are sets,

- $x \in y$: x is an element of y (sets can be elements of other sets);
- $x \cap y = \{z \mid z \in x \wedge z \in y\}$ is the intersection of x, y;
- $x \cup y = \{z \mid z \in x \vee z \in y\}$ is the union of x, y;
- $\bigcup x = \{z \mid \exists y \in x \ (z \in y)\}$ is the set of elements x;

[1] See [115] for a good technical introduction to set theory, and [67] for advanced notions.

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4

- $x \setminus y = \{z \mid z \in x \wedge z \notin y\}$ is the set difference of x and y;
- $x \subseteq y$: x is a subset of y, i.e., $\forall z\ (z \in x \to z \in y)$;
- $x \supseteq y$: x is a superset of y, i.e.,$\forall z\ (z \in y \to z \in x)$;
- $x = y$: x, y are equal, i.e., $x \subseteq y \wedge y \subseteq x$;
- $x \neq y$: x, y are different, i.e., $\neg(x = y)$;
- $x \subsetneq y$: x is a strict subset of y, i.e., $x \subseteq y \wedge x \neq y$;
- $x \supsetneq y$: x is a strict superset of y, i.e., $x \supseteq y \wedge x \neq y$;
- whenever x is a set, $|x|$ is the cardinality of x
  (if x is a finite set, $|x|$ is the number of elements in x).

A.3 Fields

A *field* is a set $\mathbb{F}$ with two binary operations (sum and product) acting on its elements, according to the rules (or *axioms*) below. Fields usually contain numbers, and the two operations correspond to the ordinary sum and product.

1. Closure of sum
   $\forall \alpha, \beta \in \mathbb{F}\ (\alpha + \beta \in \mathbb{F})$;
2. closure of product
   $\forall \alpha, \beta \in \mathbb{F}\ (\alpha\beta \in \mathbb{F})$;
3. associativity of sum
   $\forall \alpha, \beta, \gamma \in \mathbb{F}\ (\alpha + (\beta + \gamma) = (\alpha + \beta) + \gamma)$;
4. associativity of product
   $\forall \alpha, \beta, \gamma \in \mathbb{F}\ (\alpha(\beta\gamma) = (\alpha\beta)\gamma)$;
5. commutativity of sum
   $\forall \alpha, \beta \in \mathbb{F}\ (\alpha + \beta = \beta + \alpha)$;
6. commutativity of product
   $\forall \alpha, \beta \in \mathbb{F}\ (\alpha\beta = \beta\alpha)$;
7. distributivity of product over sum
   $\forall \alpha, \beta, \gamma \in \mathbb{F}\ (\alpha(\beta + \gamma) = \alpha\beta + \alpha\gamma)$;
8. $\mathbb{F}$ contains two elements called 0 and 1 which satisfy Axioms 9–13 (and no other element of $\mathbb{F}$ satisfies them);
9. sum is invariant w.r.t. 0
   $\forall \alpha \in \mathbb{F}\ (0 + \alpha = \alpha)$;
10. product is zero w.r.t. 0
    $\forall \alpha \in \mathbb{F}\ (0\alpha = 0)$;
11. product is invariant w.r.t. 1
    $\forall \alpha \in \mathbb{F}\ (\alpha \neq 0 \to 1\alpha = 1)$;
12. every $\alpha \in \mathbb{F}$ has an inverse $-\alpha$ with respect to sum
    $\forall \alpha \in \mathbb{F}\ \exists!(-\alpha) \in \mathbb{F}\ (\alpha + (-\alpha) = 0)$;
13. (every nonzero $\alpha \in \mathbb{F}$ has an inverse α^{-1} with respect to product)
    $\forall \alpha \in \mathbb{F}\ (\alpha \neq 0 \to \exists!\alpha^{-1} \in \mathbb{F}\ (\alpha\alpha^{-1} = 1))$.

We write $\alpha - \beta$ to mean $\alpha + (-\beta)$, for any $\alpha, \beta \in \mathbb{F}$. From the field axioms, we can prove elementary statements such that $\alpha + 0 = \alpha$ for all $\alpha \in \mathbb{F}$: this follows from Axioms 9 and 5.

The rational numbers, denoted by $\mathbb{Q}$, are a field. The real numbers $\mathbb{R}$ are also a field, and so are the complex numbers $\mathbb{C}$. The integers $\mathbb{Z}$ are not a field and neither are the nonnegative integers $\mathbb{N}$.

A.4 Vector spaces

A *vector space* over a field $\mathbb{F}$ is a set $\mathcal{V}$ satisfying the following axioms:

1. associativity
   $\forall x, y, z \in \mathcal{V}\ (x + (y + z) = (x + y) + z)$;
2. commutativity
   $\forall x, y \in \mathcal{V}\ (x + y = y + x)$;
3. $\mathcal{V}$ contains a unique element called 0 such that $\forall x \in \mathcal{V}\ (0 + x = x)$;
4. every $x \in \mathcal{V}$ has an inverse $-x$ with respect to sum
   $\forall x \in \mathcal{V}\ \exists!(-x) \in \mathbb{F}\ (x + (-x) = 0)$;
5. closure of scalar[2] product
   $\forall \alpha \in \mathbb{F}, x \in \mathcal{V}\ (\alpha x \in V)$;
6. associativity of the product by a scalar
   $\forall \alpha, \beta \in \mathbb{F}, x \in \mathcal{V}\ (\alpha(\beta x) = (\alpha\beta)x)$;
7. distributivity of $\mathbb{F}$ w.r.t. V
   $\forall \alpha \in \mathbb{F},\ x, y \in \mathcal{V}\ (\alpha(x + y) = \alpha x + \alpha y)$;
8. distributivity of $\mathcal{V}$ w.r.t. $\mathbb{F}$
   $\forall \alpha, \beta \in \mathbb{F}, x \in \mathcal{V}\ ((\alpha + \beta)x = \alpha x + \beta x)$;
9. invariance of product w.r.t. 1
   $\forall x \in \mathcal{V}\ (1x = x)$.

We write $-x$ for $-1x$, and $x - y$ to mean $x + (-y)$, for any $x, y \in \mathcal{V}$.

From these axioms, we can formally prove elementary statements such as $0x = 0$ for all $x \in \mathcal{V}$:

$$
\begin{aligned}
&\forall 0 \neq \alpha \in \mathbb{F}\ 0x = (\alpha + (-\alpha))x \text{ by field Axiom (12)} \\
&= \alpha x + (-\alpha)x \text{ by (8)} \\
&= \alpha x + (-1\alpha)x) \text{ by field Axiom (11)} \\
&= \alpha x + (-1)(\alpha x) \text{ by (6)} \\
&= (\alpha x) + -(\alpha x) \text{ by (9)} \\
&= 0 \text{ by (4)}.
\end{aligned}
$$

A.4.1 Linear independence and bases

Let $\mathcal{V}$ be a vector space over $\mathbb{F}$, and consider vectors $x_1, \ldots, x_n \in \mathcal{V}$ such that, for any choice of n scalars $\lambda_1, \ldots, \lambda_n \in \mathbb{F}$, the following holds:

$$\lambda_1 x_1 + \cdots + \lambda_n x_n = 0 \quad \Leftrightarrow \quad \lambda_1 = \ldots = \lambda_n = 0. \tag{A.1}$$

Then, we say that the set $B = \{x_1, \ldots, x_n\}$ is *linearly independent*. Otherwise, B is *linearly dependent*, i.e., there are scalars $\lambda_1, \ldots, \lambda_n$ not all zeroes, such that

$$\sum_{i=1}^{n} \lambda_i x_i = 0. \tag{A.2}$$

[2]When a vector space is defined over a field, the field elements are often called *scalars*.

If B is linearly dependent, then there must exist an index $j \leq n$ such that $\lambda_j \neq 0$ in Eq. (A.2), so:

$$\lambda_j x_j + \sum_{\substack{i \leq n \\ i \neq j}} \lambda_i x_i = 0$$
$$\Rightarrow \quad \sum_{\substack{i \leq n \\ i \neq j}} -\frac{\lambda_i}{\lambda_j} x_i = x_j,$$

which means that x_j can be expressed as a *linear combination* of $x_1, \ldots, x_{j-1}, x_{j+1}, \ldots, x_n$. A linearly independent set in V having maximum cardinality is called a *basis*.

A.4.2 Dimension

It turns out that all bases of a given vector space have the same cardinality, which is called the *dimension* of $\mathcal{V}$ and denoted by $\mathsf{dim}\mathcal{V}$. With respect to a given basis $B = \{x_1, \ldots, x_n\}$, every vector $y \in \mathcal{V}$ is a linear combination of vectors in B: if it were not so, then $B \cup \{y\}$ would be linearly independent and have a larger cardinality than B, which contradicts the fact that B is a basis. Hence, we can express any vector $v \in \mathcal{V}$ as $v = \sum_{i \leq n} v_i x_i$ for some set of n scalars $v_1, \ldots, v_n$ in $\mathbb{F}$. This justifies the following representation:

$$v = (v_1, \ldots, v_n). \tag{A.3}$$

Now, for any $i \leq n$ (where $n = \mathsf{dim}\mathcal{V}$), define e_i to be the vector $(0, \ldots, 0, 1, 0, \ldots, 0)$ which has a 1 in the ith component, and 0 elsewhere. The set $B = \{e_1, \ldots, e_n\}$ is a basis, called the *standard basis* of $\mathcal{V}$.

If $\mathcal{V}$ has dimension 1, then the standard basis consists of a single vector $e_1 = (1)$, and any vector $v \in \mathcal{V}$ can be written as (v_1), where $v_1 \in \mathbb{F}$. This makes it obvious that there is a bijection between $\mathbb{F}$ and a one-dimensional vector space over $\mathbb{F}$. If we take $\mathbb{F} = \mathbb{R}$, then a 1D vector space over $\mathbb{R}$ is essentially the same as the real line $\mathbb{R}$. For two dimensions, the standard basis is $B = \{e_1, e_2\}$, and 2D vectors can be written as (v_1, v_2), where v_1, v_2 are scalars in $\mathbb{R}$. This is the same representation used for the Cartesian plane, where each vector is given by a horizontal and a vertical coordinate. For 3D or general K-dimensional vector spaces, we just append components: from $v = (v_1, v_2, v_3)$ to $v = (v_1, \ldots, v_K)$.

A.4.3 Subspaces

If $\mathcal{U}, \mathcal{V}$ are vector spaces and $\mathcal{U} \subseteq \mathcal{V}$, then $\mathcal{U}$ is a *subspace* of $\mathcal{V}$. The intersection of two subspaces is a subspace.

Any (infinite) line containing the origin $(0, 0)$ is a subspace of $\mathbb{R}^2$; any line or plane containing the origin $(0, 0, 0)$ is a subspace of $\mathbb{R}^3$. The dimension of a line is 1; the dimension of a plane is 2.

The *span* of a set of vectors S in a vector space $\mathcal{V}$ is the set of vectors T that consists of vectors of the form $\sum_{x \in S} \lambda_x x$, where λ_x are scalars. The span of S is a subspace of $\mathcal{V}$.

An *affine space* is a subset $S = \{x + b \mid x \in \mathcal{V}', b \in \mathcal{V}\}$ of a vector space $\mathcal{V}$, where $\mathcal{V}'$ is a subspace of $\mathcal{V}$. The *dimension* of an affine space is the dimension of the subspace $\mathcal{V}'$. A set of vectors in $\mathcal{V}$ is *affinely dependent* if their translated pairwise differences all belong to an affine subspace S of $\mathcal{V}$, and *affinely independent* if the pairwise differences span the whole of $\mathcal{V}$. If X is a finite set of vectors in

$\mathcal{V}$, $\mathsf{aff}\, X$ is the affine space in $\mathcal{V}$ of smallest dimension which contains all vectors in X, also called the *affine hull* of X.

Geometrically, a subspace can be visualized in 2D as a line passing through the origin, or in 3D as a line or plane passing through the origin. An affine space is a translation of a subspace.

A.5 Matrices

A *matrix* is a rectangular array of scalars of a field $\mathbb{F}$, e.g., $(1\ 2)$ is a 1×2 matrix in $\mathbb{Q}$, $\begin{pmatrix} 1.1 & 2 \\ -1.5 & \sqrt{2} \end{pmatrix}$ is a 2×2 *square* matrix over $\mathbb{C}$ (and also $\mathbb{R}$), and so on. An $m \times n$ matrix A having component a_{ij} in row $i \leq m$ and column $j \leq n$ is denoted as $A = (a_{ij})$.

A.5.1 Transpose

The *transpose* of an $m \times n$ matrix $A = (a_{ij})$ is the $n \times m$ matrix $A^\top = (a_{ji})$. For example, the transpose of $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ is $\begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$. An $m \times 1$ matrix is a *column vector*, and a $1 \times n$ matrix is a *row vector*.

A.5.2 Sum of matrices

A sum is defined on $m \times n$ matrices, namely:

$$\begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix} + \begin{pmatrix} b_{11} & \dots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{m1} & \dots & b_{mn} \end{pmatrix} = \begin{pmatrix} a_{11} + b_{11} & \dots & a_{1n} + b_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & \dots & a_{mn} + b_{mn} \end{pmatrix}.$$

The invariant element of the matrix sum is the *zero* matrix (a matrix consisting of all zeroes).

A.5.3 Product by a scalar

A product by scalars is defined on $m \times n$ matrices, namely:

$$\lambda \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix} = \begin{pmatrix} \lambda a_{11} & \dots & \lambda a_{1n} \\ \vdots & \ddots & \vdots \\ \lambda a_{m1} & \dots & \lambda a_{mn} \end{pmatrix}.$$

A.5.4 Inner product

A product is defined between $m \times n$ and $n \times p$ matrices, namely:

$$\begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & \dots & b_{1p} \\ \vdots & \ddots & \vdots \\ b_{n1} & \dots & b_{np} \end{pmatrix} = \begin{pmatrix} \sum_{i \le n} a_{1i} b_{i1} & \dots & \sum_{i \le n} a_{1i} b_{ip} \\ \vdots & \ddots & \vdots \\ \sum_{i \le n} a_{mi} b_{i1} & \dots & \sum_{i \le n} a_{mi} b_{ip} \end{pmatrix},$$

or, in other words, the $m \times n$ matrix $A = (a_{ij})$ can be multiplied by an $n \times p$ matrix $B = (b_{ij})$ to obtain the $m \times p$ matrix $C = (c_{ij})$ where $c_{ij} = a_{i\cdot} \cdot b_{\cdot j}$, where $a_{i\cdot} = (a_{i1}, \dots, a_{in})$ and $b_{\cdot j} = (b_{1j}, \dots, b_{nj})$.

The invariant element of the matrix product is the *identity* matrix I: every diagonal element of I is 1, and every off-diagonal element is 0.

If $x, y \in \mathbb{R}^n$ are both column vectors, two possible products are defined: the inner product $x^\top \cdot y$ and the outer product $x \cdot y^\top$. The first yields the 1×1 matrix $(\sum_{i \le n} x_i y_i)$, which can be simply interpreted as the scalar $\sum_{i \le n} x_i y_i$, and the second yields the square $n \times n$ matrix having $x_i y_j$ as its (i, j)th component. By notational convention, $x \cdot y$ means $x^\top \cdot y$, whereas we explicitly write $x \cdot y^\top$ for the second case. If there is no ambiguity, we sometimes omit the $\cdot$ product symbol altogether.

The inner product $x \cdot y$ is often called *scalar product* of two vectors.

A.5.5 Linear transformations

Since a (column) vector $x \in \mathbb{R}^n$ is the same as an $n \times 1$ matrix, for a given $m \times n$ matrix A, we restrict the matrix product to multiply matrices and vectors: Ax is an $m \times 1$ column vector. We can therefore interpret matrices as mappings from $\mathbb{R}^n$ to $\mathbb{R}^m$. Square $n \times n$ matrices are mappings from $\mathbb{R}^n$ to itself.

This product is linear: $(\lambda A)x = \lambda(Ax)$ and $(A + B)x = Ax + Bx$ for any appropriately sized matrices A, B, vector x, scalar λ. Matrices interpreted as mappings defined on vector spaces are also known as *linear transformations*.

Consider a linear transformation $\mathbb{R}^n \to \mathbb{R}^m$ represented by a $m \times n$ matrix A. The *kernel* of A is $\mathsf{ker}\, A = \{x \in \mathbb{R}^n \mid Ax = 0\}$, and the *image* of A is $\mathsf{Im}\, A = \{y \in \mathbb{R}^m \mid \exists x \in \mathbb{R}^n \ (Ax = y)\}$. The kernel is a subspace of $\mathbb{R}^n$ and the image is a subspace of $\mathbb{R}^m$. Moreover, $\mathsf{dim\,Ker}\, A + \mathsf{dim\,Im}\, A = n$ (this is known as the *kernel and image* or *rank and nullity* theorem). We also call $\mathsf{dim\,Ker}\, A$ the *nullity* and $\mathsf{dim\,Im}\, A$ the *rank* of A and denote it $\mathsf{rk}\, A$. A has *full rank*, if $\mathsf{rk}\, A = \min(m, n)$. A square matrix with full rank is *nonsingular*; otherwise, it is *singular*. If A is square and nonsingular, then there exists another square $n \times n$ matrix A^{-1}, called the *inverse* of A, such that $A^{-1}A = AA^{-1} = I$.

A.5.6 Congruences

The transformation that adds a fixed vector to each vector of a set is called a *translation*. The transformation that rotates all the vectors in S by a given angle around a given center is called a *rotation* of the vector set. The transformation that reflects all the vectors in S with respect to a given hyperplane is called a *reflection* of the vector set.

Translations, rotations, and reflections of S are congruences of S, i.e., they preserve all distances (see Sect. A.6) between pairs of vectors in S. Less trivially, every congruence of S turns out to be a composition of translations, rotations, and reflections.

A.5.6.1 Translations

Algebraically, a translation τ acts on $S \subseteq \mathbb{R}^n$ as follows:

$$\forall x \in S \quad \tau(x) = x + b, \tag{A.4}$$

where $b \in \mathbb{R}^n$. This is also written in the more compact form $\tau(S) = S + b$.

A.5.6.2 Rotations

A rotation ρ_θ by an angle θ with respect to the origin acts on S as follows:

$$\forall x \in S \quad \rho_\theta(x) = A^\theta x, \tag{A.5}$$

where A^θ is an $n \times n$ matrix resulting from a product of scaled Givens matrices, defined below. With respect to a standard basis $e_1, \ldots, e_n$, where e_j is the zero vector with a 1 in the jth component, a *Givens matrix* $G^{\eta ij}$ rotates the unit projection of the unit vector $\hat{x} = \frac{x}{\sqrt{x \cdot x}}$ on the plane spanned by e_i, e_j by an angle η with respect to the origin. It embeds the 2D rotation matrix $r = \begin{pmatrix} \cos\eta & -\sin\eta \\ \sin\eta & \cos\eta \end{pmatrix}$ in an $n \times n$ identity matrix: $G^{\eta ij}_{ii} = r_{11}, G^{\eta ij}_{jj} = r_{22}, G^{\eta ij}_{ij} = r_{12}, G^{\eta ij}_{ji} = r_{21}$, and the rest of the components of $G^{\eta ij}$ equal to the components of the identity matrix:

$$G^{\eta ij} = \begin{pmatrix} & & \overset{\text{i}}{} & & \overset{\text{j}}{} & & \\ 1 & \ldots & 0 & \ldots & 0 & \ldots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \ldots & \cos\eta & \ldots & -\sin\eta & \ldots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \ldots & \sin\eta & \ldots & \cos\eta & \ldots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \ldots & 0 & \ldots & 0 & \ldots & 1 \end{pmatrix} \begin{matrix} \\ \\ \text{i} \\ \\ \text{j} \\ \\ \\ \end{matrix} \tag{A.6}$$

Thus, A^θ is a product of appropriately scaled Givens matrices $G^{\eta ij}$ for all $(i, j) \in H = \{(1, 1), \ldots, (1, n)\}$ (the reason why we fix i to 1 is that every swap (i, j) can be generated compositionwise as a product of the 2-cycle permutation swaps in H). In order to compute η from θ for each $(i, j) \in H$, one must have θ in terms of the angle between two vectors $x, y \in \mathbb{R}^n$, project x, y in the plane defined by each coordinate pair $(i, j) \in H$, then compute the angles η between the projections $\bar{x}, \bar{y}$, which must be scaled by $\|\bar{x}\|_2$ and $\|\bar{y}\|_2$, respectively (we are using the notation $\|\cdot\|_2$ before formally introducing it in Sect. A.6, but $\|z\|_2$ can be written as $z \cdot z$ for any vector z): this makes the projections have unit length, so that the Givens rotations apply. Accordingly, the product of the $G^{\eta ij}$ must be scaled back so that, when applied to x, it yields a vector with the same length as x.

Since this procedure only holds for rotations with respect to the origin, arbitrary rotations with respect to a vector $b \in \mathbb{R}^n$ must be prefixed by a translation to the origin and postfixed by its inverse, yielding $\rho_\theta(x) = A^\theta(x - b) + b$. Notationally, this is applied to each vector in S as $\rho_\theta(S) = A^\theta(S - b) + b$.

A.5.6.3 Reflections

A reflection $R_{a,0}$ acting on S, with respect to a hyperplane $a^\top x = 0$ (i.e., containing the origin and orthogonal to the unit vector $a \in \mathbb{R}^n$), is as follows:

$$\forall x \in S \quad R_{a,0}(x) = (I_n - 2aa^\top)x,$$

where $I_n - 2aa^\top$ is the matrix

$$\begin{pmatrix} 1-2a_1^2 & -2a_1a_2 & \dots & -2a_1a_n \\ -2a_2a_1 & 1-2a_2^2 & \dots & -2a_2a_n \\ \vdots & \vdots & \ddots & \vdots \\ -2a_na_1 & -2a_na_2 & \dots & 1-a_n^2 \end{pmatrix}.$$

For a reflection with respect to an affine subspace $a^\top x = b$, let $j \leq n$ be such that $a_j \neq 0$, and notice that the vector $\bar{b} = (0, \dots, \bar{b}_j, \dots, 0)$, having $\bar{b}_j = \frac{b}{a_j}$ as the only nonzero component, satisfies $a^\top \bar{b} = b$. So we simply translate x by $\bar{b}$ before reflecting it along $a^\top x = 0$, and by $-\bar{b}$ after. The resulting generalized reflection operator is:

$$\forall x \in S \quad R_{a,b}(x) = (I_n - 2aa^\top)(x - \bar{b}) + \bar{b}.$$

As before, we shorten the above as: $R_{a,b}(S) = (I_n - 2aa^\top)(S - \bar{b}) + \bar{b}$.

A.5.7 Determinants

Let A be an $n \times n$ square matrix $A = (a_{ij})$. For any given $i, j \leq n$, the $(n-1) \times (n-1)$ matrix A^{ij} given by deleting the ith row and jth column of A is the *submatrix* of A with respect to i, j.

For any $i, j \leq n$, consider the following recursive definitions of the functions ϕ_i, ψ_j mapping A to a scalar in $\mathbb{R}$:

$$\phi_i(A) = \sum_{j \leq n} (-1)^{i+j} a_{ij} \phi_i(A^{ij})$$
$$\psi_j(A) = \sum_{i \leq n} (-1)^{i+j} a_{ij} \psi_j(A^{ij}).$$

Notice that each ϕ_i applied to an $n \times n$ matrix is defined in terms of ϕ_i applied to an $(n-1) \times (n-1)$ matrix. The recursion starts by setting ϕ_i and ψ_j applied to 1×1 matrices (a_{11}) to be equal to their only component a_{11}.

It turns out that $\phi_i(A) = \psi_j(A)$ for all $i, j \leq n$, so we call this value the *determinant* of A and denote it by $|A|$; then, given any row index i or column index j, $|A| = \sum_{j \leq n} (-1)^{i+j} a_{ij} |A^{ij}| = \sum_{i \leq n} (-1)^{i+j} a_{ij} |A^{ij}|$.

If A is a square nonsingular $n \times n$ matrix, its inverse is given by:

$$A^{-1} = \frac{\mathsf{Adj}A}{|A|}$$

where $\mathsf{Adj}A$ is the *adjugate* matrix of A, having $(-1)^{i+j}|A^{ij}|$ as its (i, j)th component. We also have that $|A^{-1}| = \frac{1}{|A|}$ and that $|A \cdot B| = |A||B|$ for any square $n \times n$ matrix B.

A.5.8 Eigenvalues and eigenvectors

For a square $n \times n$ matrix B, consider the equation $By = \lambda y$, where λ is a scalar and $y \in \mathbb{C}^n$ is a nonzero vector: λ is an *eigenvalue* of B and y is an *eigenvector* of B associated to λ. By elementary linear algebra, B has at most $K = \mathsf{rk}\, B$ distinct eigenvalues. If B is a *symmetric matrix* (i.e., $B^\top = B$), all its eigenvalues and eigenvectors are real, and it is possible to choose K orthogonal eigenvectors of B which span the same K-dimensional subspace of $\mathbb{R}^n$ as the columns of B. Moreover, if one of the eigenvalues is zero, the corresponding eigenvectors span $\mathsf{Ker}\, B$, whereas the eigenvectors corresponding to nonzero eigenvalues span $\mathsf{Im}\, B$.

In general, if Λ is the $K \times K$ *diagonal matrix* having nonzero eigenvalues $\lambda_1, \ldots, \lambda_K$ (some of which may be equal) along the diagonal, and zeroes everywhere else, and if Y is the $n \times K$ matrix of corresponding eigenvectors of B, then $B = Y \cdot \Lambda \cdot Y^\top$.

A.6 Norms and metrics

Let $\mathcal{V}$ be a K-dimensional vector space over $\mathbb{R}$. We introduce a function of one argument, called *norm*, which maps each vector to a nonnegative real number. A function $\|\cdot\| : \mathcal{V} \to \mathbb{R}$ is a norm if it satisfies the following axioms:

1. $\forall x \in \mathcal{V}\ (\|x\| \geq 0)$;
2. $\forall x \in \mathcal{V}\ (\|x\| = 0 \leftrightarrow x = 0)$;
3. $\forall \lambda \in \mathbb{R}, x \in \mathcal{V}\ (\|\lambda x\| = \lambda \|x\|)$;
4. $\forall x, y \in \mathcal{V}\ (\|x + y\| \leq \|x\| + \|y\|)$.

A typical example of a norm is the *Euclidean norm*, defined as follows:

$$\forall x = (x_1, \ldots, x_n) \in \mathcal{V} \quad \|x\|_2 = \sqrt{x \cdot x} = \sqrt{\sum_{i \leq n} x_i^2}. \tag{A.7}$$

Now, we introduce a function of two arguments, called *metric*, which maps ordered pairs of vectors to a nonnegative real number. A function $d(\cdot, \cdot)$ is a metric if it satisfies the following axioms:

1. $\forall x, y \in \mathcal{V}\ (d(x, y) = 0 \leftrightarrow x = y)$;
2. $\forall x, y \in \mathcal{V}\ (d(x, y) = d(y, x))$;
3. $\forall x, y, z \in \mathcal{V}\ (d(x, z) \leq d(x, y) + d(y, z))$;
4. $\forall x, y \in \mathcal{V}\ (d(x, y) \geq 0)$.

A typical example of a metric is the *Euclidean metric*, defined in function of the Euclidean norm as:

$$\forall x, y \in \mathcal{V}\ (d(x, y) = \|x - y\|_2). \tag{A.8}$$

A vector space endowed with a metric is called a *metric space*.

Given a point x of a metric space $\mathcal{V}$, a neighborhood χ of x contains all elements y of $\mathcal{V}$ such that $d(x, y) \leq \epsilon$ for some (usually small) given constant $\epsilon > 0$. This concept extends to finite metric spaces (intended as finite sets, not necessarily vector spaces, endowed with a metric), see Sect. A.8.1.

A.6.1 The intuition behind vectorial geometry

A *Euclidean space* is a vector space $\mathcal{V}$ over $\mathbb{R}$ endowed with the Euclidean norm and distance. If $\mathsf{dim}\mathcal{V} = K$, we denote the Euclidean space by $\mathbb{R}^K$. $\mathbb{R}^1$ is a good mathematical model for a straight line, $\mathbb{R}^2$ for a plane, and $\mathbb{R}^3$ for the physical space that surrounds us locally. Vectors correspond to points in this space, with coordinates indicating width, depth and height. The norm of a vector x is its length (or the Euclidean distance from the point x to the origin), and the Euclidean metric between x and y is the Euclidean distance between the point x and the point y.

A.7 Groups

A *group* is a set G with a product operation that satisfies the following axioms:

1. closure
   $\forall g, h \in G\ (gh \in G)$;
2. associativity
   $\forall f, g, h \in G\ ((fg)h = f(gh))$;
3. G contains a unique identity element e
   $\exists! e \in G\ \forall g \in G\ (eg = ge = g)$;
4. every element has a unique inverse
   $\forall g \in G\ \exists! g^{-1} \in G\ (gg^{-1} = g^{-1}g = e)$.

From these axioms, we can easily prove elementary statements. For example, the uniqueness of the inverse actually follows from other axioms:

$$\begin{aligned} & \forall g \in G \exists h, h' \in G\ (gh = hg = gh' = h'g = e) \\ \Rightarrow\ & gh' = e \\ \Rightarrow\ & hgh' = he \\ \Rightarrow\ & eh' = he \\ \Rightarrow\ & h' = h. \end{aligned}$$

Note that the field axioms define a group over the sum and over the product—hence, the uniqueness of the inverses can also be reduced to other axioms in the same way.

A.7.1 A finite planar rotation group

For example, the set $R = \{0, \pi/2, \pi, 3\pi/2\}$ of planar rotations around the origin, under the operation given by $+$, is a group: all sums of elements of R are in R (because $2\pi = 0$); the sum is associative by definition, 0 is the unique identity, and given any rotation $g \in R$, there is always another rotation $g^{-1} \in R$ (denote it by $-g$) with $g + (-g) = 0$.

A.7.2 Abelian groups

A group for which commutativity holds, i.e., $\forall g, h \in G\ (gh = hg)$ is called *Abelian*. $(R, +)$ above is Abelian.

A.7.3 The group table

The *table* of a finite group G is a square $|G| \times |G|$ matrix whose (g, h)th entry is the product gh. For example, the table of $(R, +)$ above is:

	0	$\pi/2$	π	$3\pi/2$
0	0	$\pi/2$	π	$3\pi/2$
$\pi/2$	$\pi/2$	π	$3\pi/2$	0
π	π	$3\pi/2$	0	$\pi/2$
$3\pi/2$	$3\pi/2$	0	$\pi/2$	π

Let G be a group and H a subset of G. If H is itself a group under the same product as G, then H is a *subgroup* of G, denoted $H \leq G$. If $H \leq G$, then $|H|$ divides $|G|$. For example, if $S = \{0, \pi\}$, then $(S, +) \leq (R, +)$. Notice that 2 divides 4.

An element $g \in G$ such that $gg = g^2 = e$ is called *idempotent*. For example, since $(\pi)^2 = \pi + \pi = 0$, π is idempotent.

A.7.4 Group actions

Let V be any set, and G be a group of functions on V, where the product of two functions f, g is defined by their composition $f \circ g$. Then, G defines an *action* on V; we say that G *acts on* V.

For example, $(R, +)$ defines an action on $\mathbb{R}^2$: for any vector $x \in \mathbb{R}^2$ and any rotation $g \in G$, gx is the vector in $\mathbb{R}^2$ corresponding to x being rotated by g.

A.7.5 Generators

Let G be a group and $S \subseteq G$ a subset of its elements. If we can write any $g \in G$ as a product of some elements in S, then G is *generated* by S, which is a set of *generators* of G (written $G = \langle S \rangle$). We are often interested in finding *minimal* generator sets, i.e., sets of generators of minimum cardinality.

For example, $(R, +) = \langle \pi/2 \rangle$ and $(S, +) = \langle \pi \rangle$.

A.7.6 Orbits

Let G be a group acting on V. Define an equivalence relation $\sim$ on V given by $u \sim v$ if $\exists g \in G$ with $v = gu$. Then, $\sim$ partitions[3] V into equivalence classes called *orbits*.

[3] A *partition* of a set V is a set of subsets of V such that their union is V and their pairwise intersection is empty.

Another definition for orbits is as follows. For any $v \in V$, let $Gv = \{gv \mid g \in G\} \subseteq V$. So Gv is the orbit of v with respect to G. The two definitions are equivalent: if $u, w \in Gv$ for some $v \in V$, obviously $u \sim w$ by definition; and if u, w are part of an orbit, then there must exist $v \in V$ such that $u = gv$ and $w = hv$ for some $g, h \in G$: namely, take $w = v$ and $h = e$.

Let Ω be a partition of V into orbits $\omega_1, \ldots, \omega_\ell$. What this means, in practice, is that there is no $g \in G$ that can map an element in a given orbit to an element in a different orbit. If Ω contains only one orbit, i.e., $\ell = 1$ and $\Omega = \{\omega_1\}$, then obviously $\omega_1 = V$, and the action of G on V is said to be *transitive* (sometimes, we abuse terminology and simply say that G is transitive).

By the second definition of orbit, for each orbit $\omega \in \Omega$ we can single out an element $v \in \omega$ and call it an *orbit representative*.

Orbits come in useful when V is expensive to compute, but G is not: then, we might want to compute a small subset $S \subset V$ and then try to generate all the elements of V as products gv, where $g \in G$ and $v \in S$. In general, S must be a set of orbit representatives. Then, $\bigcup_{v \in S} Gv = V$. The computationally most convenient setting is when G is transitive, as S can be reduced to a single orbit representative.

The group $(R, +)$ defined above naturally acts on $\mathbb{R}^2$ by rotating vectors in the plane. The orbit of any vector y is a set ω_y, which contains the four vectors $\{y, A^{\pi/2}y, A^{\pi}y, A^{3\pi/2}y\}$, where A^θ is the rotation matrix of the angle θ in the plane. If $y = (\sqrt{2}/2, \sqrt{2}/2)$, ω_y consists of the four corners of the unit square centered at the origin. Since rotations do not change relative lengths, $(R, +)$ also naturally acts on the unit circle $S^1 = ((0, 0), 1)$ centered at the origin, and having unit radius. $(R, +)$ partitions the unit circles into a set Ω of uncountably many orbits: any $\omega \in \Omega$ consist of four points on the unit circle, and $\bigcup_{\omega \in \Omega} \omega$ *is* the unit circle.

A.7.7 Isomorphism

A *group homomorphism* from G to H is a mapping $\phi: G \to H$ such that, for any $f, g \in G$, $\phi(fg) = \phi(f)\phi(g)$. A *group isomorphism* between G and H is a bijection $\phi: G \to H$ such that ϕ, ϕ^{-1} are both homomorphisms. If ϕ is a group isomorphism between G and H, then we write $G \cong H$.

In practice, two groups are isomorphic if their group table looks the same, aside from the names of the elements. We tend to think of isomorphic groups as "essentially the same group."

For example, the *cyclic group* $C_2 = \{e, g\}$ is isomorphic to the group $(S, +)$ above (with $S = \{0, \pi\}$). Their group tables are:

	e	g
e	e	g
g	g	e

	0	π
0	0	π
π	π	0

It is obvious that it suffices to change the name g into π to recover the same group table. C_2 is the simplest nontrivial finite group; it has a single nonidentity element which is idempotent. Since $(S, +) \cong C_2$, $(S, +)$ and C_2 are essentially the same group.

A.8 Graphs

Graphs are a model of sets of pairs of elements in a set. Let V be any finite set, and let E a set of (unordered) pairs of elements in V. Thus, for example, we might take $V = \{A, B, C, S\}$ and $E = \{\{A, B\}, \{A, C\}, \{A, S\}, \{B, C\}\}$. Then, the couple $G = (V, E)$ is called a *graph*. If E consists of ordered pairs, then the graph is *directed*; otherwise, it is *undirected*. A directed graph is also sometimes called a *digraph*. Usually, if G is undirected, elements of V are called *vertices* and elements of E are

called *edges*. For *digraph*, the corresponding terminology is usually *nodes* and *arcs*. If G is a given graph, we also refer to its vertex set as $V(G)$ and to its edge set as $E(G)$. Two vertices u, v are *adjacent* if $\{u, v\} \in E$. An edge $e = \{u, v\}$ is *incident* to u and v, and vice versa u and v are *incident* to e. Two edges e, f are adjacent if $|e \cap f| = 1$, i.e., if they are both incident to the same vertex. Given a set of edges E, we let $V[E]$ be the set of vertices incident to (or induced by) all edges in E.

A.8.1 Neighborhoods, degrees and cutsets

For $G = (V, E)$ an undirected graph and $v \in V$, let $N_G(v) = \{u \in V \mid \{u, v\} \in E\}$ be the *neighborhood* of v, and $\delta_G(v) = \{\{u, v\} \in E\}$ be the *star* of v. Let $|N_G(v)|$ be the *degree* of v. Given $U \subseteq V$, we let $N_G(U) = \{v \in V \mid \exists u \in U \ \{u, v\} \in E\}$ be the *cut* or *neighborhood* of U, i.e., the set of vertices of $V \setminus U$ adjacent to a vertex in U (also see Sect. A.6). We let $\delta_G(U) = \{e \in E \mid |e \cap U| = 1\}$, the *cutset* of U, be the set of edges of E incident to exactly one vertex in U. We write $N(\cdot)$, $\delta(\cdot)$ whenever there is no ambiguity.

A.8.2 Simplicity and connectedness

A graph is *nonsimple* if some vertex is adjacent to itself (i.e., E contains some singleton sets $\{v\}$), or if E is a multiset, i.e., a list, or sequence, of some possibly repeated values. Singleton sets in E are called *loops*, and repeated occurrences of the same edge in E are called *parallel edges*. A graph is *simple* if it has no loops nor parallel edges. A graph $G = (V, E)$ is *connected* if no nontrivial cutset is empty (see Fig. A.1):

$$\forall U \subseteq V \quad (U \notin \{\varnothing, V\} \rightarrow \delta(U) \neq \varnothing). \tag{A.9}$$

A.8.3 Subgraphs

For $U \subseteq V$, we let $E[U] = \{\{u, v\} \in E \mid u, v, \in U\}$ be the set of edges *induced* by U, and let $G[U] = (U, E[U])$ be the *induced subgraph* of G w.r.t. U. Any graph (U, F) where $F \subseteq E[U]$ is a *subgraph* of G (Fig. A.2).

A subgraph of G is *spanning* in G if its vertex set is $V(G)$.



Fig. A.1 *Left* the graph is not connected, as the cutset defined by $\{1, 2, 3\}$ is nontrivial and empty. *Right* a connected graph.

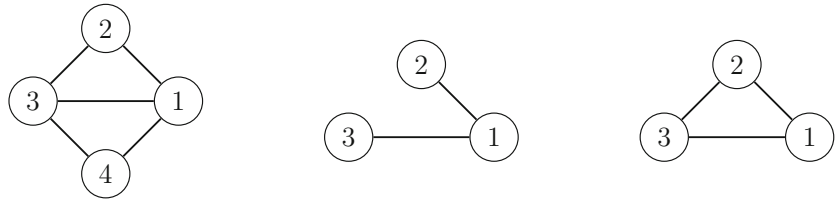


Fig. A.2 A graph, a (noninduced) subgraph and an induced subgraph.

A.8.4 Simple cycles and paths

Given a set of edges F, it induces the set $\bigcup F$ of vertices, and a graph $(U, \bigcup F)$. A *simple cycle* is a connected graph where each vertex has degree 2. If C is a simple cycle and the edge set F is such that $E(C) \setminus F$ induces a connected graph P with $E(P) \subseteq E(C)$, then P is called a *simple path* (see Fig. A.3):

$$\forall U \subseteq V \quad (U \notin \{\varnothing, V\} \to \delta(U) \neq \varnothing). \tag{A.10}$$

A simple cycle or path in a graph G is *Hamiltonian* if it is a spanning subgraph of G.

A.8.5 Edge weights

An *edge weight function* is a mapping $d : E \to \mathcal{S}$ for some set $\mathcal{S}$ of numbers. In the following, we often take $\mathcal{S} = \mathbb{R}_+$. If an edge weight function is defined on a graph, then the graph is *weighted*; otherwise, it is *unweighted*.

A.8.6 Some graph families

A graph is *complete* if it has all possible edges/arcs. A simple undirected complete graph on K vertices is called a K*-clique* (see Fig. A.4, left).

A graph $G = (V, E)$ is *bipartite* if its vertex set V can be partitioned into two subsets U, W such that for each edge $\{u, w\} \in E$, we have $u \in U$ and $w \in W$. A complete bipartite graph is also called a *biclique*.

The *complement* of a graph $G = (V, E)$ is a graph $\bar{G}$ on V where $\{u, v\}$ is an edge if and only if $\{u, v\} \notin E$. The *empty* graph is the complement of a clique. An induced subgraph S of G is *stable* (or an *independent set*) if $E[S]$ is empty (see Fig. A.4, right).



Fig. A.3 *Left* a simple cycle. *Right* a simple path yielded by $F = \{\{1, 6\}\}$.

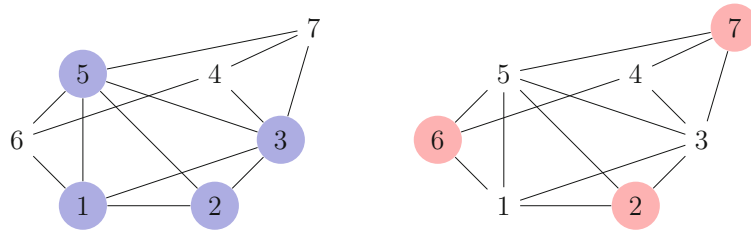


Fig. A.4 A 4-clique (*left*) and a stable set (*right*) in G.

A.9 Computational complexity

We discuss the very basic notions of computational complexity [104].

A.9.1 Worst-case complexity of an algorithm

The efficiency of algorithms is evaluated according to several criteria. The most common ones are the number of *elementary operations* (i.e., taking unit time to complete) executed before termination, or the amount of memory used during execution.

We focus on the former, which is an approximation for the execution time in practice. The *worst-case time complexity* $O(f(n))$ of an algorithm is the maximum number $f(n)$ of elementary operations executed by the algorithm over all the possible inputs encoded into n bits. This "big-oh" notation $O(f(n))$ follows asymptotic rules: if k is a constant, then $O(f(n)+k) = O(kf(n)) = O(f(n))$. If $\exists h, k, N \in \mathbb{N}\ \forall n > N\ g(n) \leq h + kf(n)$ (i.e., g is asymptotically smaller than f), then $O(g(n) + f(n)) = O(f(n))$.

There is a further convention on the big-oh notation: since these are worst-case bounds, rather than exact complexity counts, an algorithm which takes $O(n)$ at worst might also be said to take $O(n^2)$ at worst, or $O(f(n))$ for any f that is asymptotically larger than n. Formally, if g is asymptotically smaller than f, then $O(g(n)) = O(f(n))$. Of course, the tighter the function f providing the bound, the better the estimate will be.

A.9.1 Example

Let us find the worst-case time complexity of Alg. 13. We assume that $a_1, \ldots, a_n$ are integers having size bounded by 32 bits, so the size of the input is $32n$, which, asymptotically, behaves like n. The two statements inside the j loop are composed of 4 elementary operations: two sums and two assignments. These 4 operations are executed n times, for a total of $4n$ operations, and each j loop is executed n times, for a total of $4n^2$ operations. Each iteration of the k loop also requires 4 elementary operations: computing k mod 2, verifying whether it is zero, performing a difference $(c - k)$ and an assignment. These are executed $\lceil \frac{n}{2} \rceil$ times, for a total of $4\lceil \frac{n}{2} \rceil$ operations. Thus, the number of elementary operations over the whole algorithm is

$$f(n) = 4(n^2 + \left\lceil \frac{n}{2} \right\rceil).$$

Now, the asymptotical reduction is as follows: $O(\lceil \frac{n}{2} \rceil) = O(n)$ (because $\lceil \frac{n}{2} \rceil \leq n$ for each $n \in \mathbb{N}$), and $O(n^2 + n) = O(n^2)$ (because $n \leq n^2$ for each $n \in \mathbb{N}$). Also, $O(4n^2) = O(n^2)$ because 4 is a constant which does not depend on n. So the worst-case complexity of this algorithm is $O(n^2)$.

Algorithm 13 An $O(n^2)$ algorithm.

```
input (a_1, ..., a_n)
for i ≤ n do
  for j ≤ n do
    b = a_i + a_j
    c = c + b
  end for
end for
for k ≤ n such that k is odd do
  c = c − k
end for
return c
```

A.9.2 Decision and optimization problems

In computational complexity theory, by "problem" we usually mean a *decision problem*, where the output is just a YES or a NO. For example, "given a graph G and a positive integer K, does G contain a K-clique?" is a decision problem.

An *optimization problem* also provides a function from the set of possible inputs to some number field and asks for the input that maximizes or minimizes the function (which is called *objective function*). For example, "given a graph G, find the maximum K such that G contains a K-clique" is an *optimization problem*.

Every optimization problem has an associated decision problem. If $\phi(\cdot)$ is the objective function to be minimized, then the decision problem provides a lower bound K and asks whether there exists an input ι such that $\phi(\iota) \leq K$; if $\phi(\cdot)$ is to be maximized, the question is whether $\phi(\iota) \geq K$. The decision and optimization problems given above are associated in this sense.

The input of any problem is also known as an *instance*. In this sense, a problem P is just the set of all its instances.

A.9.3 Complexity of a problem

Let P be a problem, $A(P)$ be the set of algorithms that correctly solve P, and $O(a(n))$ be the worst-case complexity of the algorithm $a \in A(P)$. Then, the worst-case complexity of P is:

$$\min_{a \in A(P)} O(a(n)).$$

Because we write $O(g(n)) = O(f(n))$ whenever g is asymptotically smaller than f, we need not find the absolutely best algorithm a over the infinite set $A(P)$: the "best one found so far" will suffice and provide a worst-case complexity estimate for the problem P.

A.9.4 Easy problems

The Cobham–Edmonds thesis [26, 44] states that a problem is tractable whenever its worst-case complexity is $O(p(n))$, where p is a polynomial in n. If no polynomial-time (or *polytime*) algorithm

is known for P, then P is intractable—at least until a polytime algorithm is found, or is proven not to exist.

The class of all tractable problems is denoted by **P**. A problem in **P** is also informally called *easy*. Also, let $\bar{A}(P)$ the set of all polynomial-time algorithms for solving P.

A.9.5 Nondeterministic polynomially solvable problems

Hardness in worst-case complexity is not the converse of tractability or easiness: the notion of what it means for a decision problem to be hard is more complicated. We start with the class **NP** of decision problems P such that, for each YES instance ι (i.e., one which yields a YES answer from any algorithm in $A(P)$), there is a *certificate* $c(\iota)$ having polynomially bounded length that proves that the answer is correct.

A.9.2 Example
Let P be the HAMILTONIAN CYCLE (HC) problem: given a simple, undirected, and connected graph $G = (V, E)$, does it contain a Hamiltonian cycle? Setting aside the issue of actually finding such a cycle, it is easy to show that if G is a YES instance and $\gamma(G)$ is a Hamiltonian cycle in G, it only takes $O(n)$ to check that γ really is Hamiltonian, where $n = |V|$: it suffices to thread around the cycle from any vertex v back to v again and check that every vertex has been reached in the process. So $\gamma(G)$ is a polynomial certificate for G. Since this algorithm is valid for any instance $G \in P$, and P is a decision problem, then $P \in \mathbf{NP}$.

If $P \in \mathbf{NP}$, $G \in P$ and $a \in A(P)$, then running the algorithm a on the instance G returns a pair (r, γ) where $r \in \{\text{YES}, \text{NO}\}$, and γ is the certificate $\gamma(G)$ if $r = \text{YES}$ or $\varnothing$ if $r = \text{NO}$.

Any problem P in **P** must also be in **NP**: since $P \in \mathbf{P}$ implies the existence of a polynomial-time algorithm $a \in A(P)$, the whole execution trace of $a(G)$ provides a polynomial certificate for the instance $G \in P$.

The reason for the name "nondeterministic polynomially solvable" is technical and shall not be given here.

A.9.6 Polynomial reductions

The K-CLIQUE problem is: given a graph G and $K \in \mathbb{N}$, does G have a K-clique subgraph? The K-STABLE problem asks whether G has a stable subgraph on K vertices. Recall that cliques and stables are related by taking the complement operation $\bar{S}$ on a subgraph S of G (see Sect. A.8.6). Both problems are easily shown to be in **NP**.

Let $a \in A(K\text{-CLIQUE})$ be a solution algorithm for K-CLIQUE. A solution algorithm b for K-STABLE can be constructed using a as shown in Alg. 14. In Step 1, we *reduce* an instance G of K-STABLE to an instance $\bar{G}$ of K-CLIQUE. In Step 2, we run the K-CLIQUE algorithm a on the reduced input $\bar{G}$, and in Steps 3 and following, we apply the inverse reduction to the answer given by a so that it applies to G. The algorithm b (Alg. 14) is correct because a K-stable in G becomes a K-clique in the complement graph $\bar{G}$, and vice versa.

We generalize this to arbitrary problems P, Q (with P playing the role of K-STABLE and Q of K-CLIQUE): if we can solve Q and we can reduce an instance $\alpha \in P$ to an instance $\beta \in Q$ in such a way that an answer to β can be transformed back to an answer to α, then, if we can solve Q, then we can also solve P. If the inverse transformations between instances and between answers can be performed in polytime, then $P \to Q$ is a *polynomial reduction*.

Algorithm 14 An algorithm for solving K- STABLE.

compute the complement graph $\bar{G}$;
let $(r, \gamma) = a(\bar{G})$;
if $r =$ NO **then**
  return $(r, \varnothing)$;
else
  return $(r, \bar{\gamma})$.
end if

The reason why polynomial reductions are important is:

$$\forall P, Q \quad (Q \in \mathbf{NP} \wedge (P \to Q) \Rightarrow P \in \mathbf{NP}) \tag{A.11}$$

$$\forall P, Q \quad (Q \in \mathbf{P} \wedge (P \to Q) \Rightarrow P \in \mathbf{P}). \tag{A.12}$$

Notice that we do not need to actually know an algorithm for Q in order to postulate the existence of a polynomial reduction $P \to Q$: in Alg. 14, we run a as an *oracle*, i.e., without needing to know what actually happens to the computer when a is run. Polynomial reductions are more about proving that two problems are in the same complexity class than actually finding solution algorithms.

A.9.7 Completeness for a class

The discussion in Sect. A.9.6 can be generalized to an arbitrary problem class **C** as follows. Let **C** be a class of problems with worst-case complexities $\{O(f(n)) \mid f \in \mathcal{F}\}$ for some given family of functions $\mathcal{F}$. Let P be any problem, and ρ be a reduction from P taking time $O(g(n))$ with $g \in \mathcal{F}$. If $\rho(P) \in \mathbf{C}$, then $P \in \mathbf{C}$: for any $\iota \in P$, it suffices to compute $\rho(\iota)$, feed it as input to an algorithm $a \in A(\rho(P))$ having worst-case complexity in $\mathcal{F}$ (which must exist since $\rho(P) \in \mathbf{C}$), then inverse-reduce the answer from a to get an answer from ι.

The problem $Q \in \mathbf{C}$ is **C**-*complete* if, for any $P \in \mathbf{C}$, there is a reduction $\rho : P \to Q$.

For example, LINEAR PROGRAMMING (LP) is a **P**-complete problem: any problem in **P** can be reduced to a linear program (under reductions which require at worst a logarithmic amount of memory). HC and K- CLIQUE are **NP**-complete.

A.9.8 Hardness

A problem P is *hard* for a class **C** if every problem in **C** has a reduction to P. This is a weaker definition than **C**-completeness, since it does not require that $P \in \mathbf{C}$. For example, the DGP is **NP**-hard, but no one knows whether it is in **NP** (there is some evidence but not proof against).

Saying that every problem in **C** has a reduction to P means that P is as hard as any problem in **C**, and in particular that P is as hard as the hardest problem in C (since this observation only rests on reduction rather than membership in the class, it also applies to **C**-complete problems). This defines **C**-hard problems as a category of "equally hardest problems in **C**." So, if there is a reduction from a **C**-hard problem P to Q, it means that Q is also **C**-hard.

A.9.9 Hard problems

Informally, a decision problem is *hard* when it is **NP**-hard. Notice that this is not the converse of being easy. Notice also that, although $\mathbf{P} \subseteq \mathbf{NP}$, no one knows whether $\mathbf{P} = \mathbf{NP}$ or not (again, there is evidence but not proof against). So, in the unlikely event that $\mathbf{P} = \mathbf{NP}$, hard problems would also be easy!

Notions of hardness for optimization problems are either based on their associated decision problems, or else on the difficulty of finding provable good approximate solutions.

A.10 Exercises

A.10.1 Exercise
Prove that $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$ are fields and that $\mathbb{Z}$, $\mathbb{N}$, and $\mathbb{R}_+$ are not.

A.10.2 Exercise
Prove that, in a field, $0 + 1 = 1$, $01 = 0$, and for each $\alpha \in \mathbb{F}$, $\alpha \neq 0$ implies that $\alpha^{-1}\alpha = 1$.

A.10.3 Exercise
Prove that the algebraic numbers, i.e., those numbers which are roots of polynomials having rational coefficients, are a field.

A.10.4 Exercise
Prove that the standard basis is indeed a basis.

A.10.5 Exercise
Prove that the intersection of two or more subspaces of a vector space is a subspace. Prove that the union of two or more subspaces of a vector space may not be a subspace. Give an example of the union of two subspaces that *is* a subspace.

A.10.6 Exercise
Prove that the Euclidean norm is indeed a norm.

A.10.7 Exercise
A *path* in a graph is a sequence of vertices $(v_1, \ldots, v_n)$ such that, for all i with $1 < i \leq n$, $\{v_{i-1}, v_i\}$ is an edge of the graph. A *cycle* in a graph is a path such that $v_1 = v_n$. Prove that a simple cycle is a cycle and a simple path is a path, but the converse does not hold.

A.10.8 Exercise
Show that if P is a problem, then its set of solution algorithms $A(P)$ is (countably) infinite.

A.10.9 Exercise
Prove that K- CLIQUE and K- STABLE are in **NP**.

A.10.10 Exercise
Prove Eqs. (A.11)–(A.12).

A.10.11 Exercise
Prove that K- STABLE is **NP**-complete.

A.10.12 Exercise
Let HAMILTONIAN PATH (HP) be the problem of determining whether a given graph has a simple spanning path. Prove that HP is **NP**-complete.

References

1. Alfakih, A.: Universal rigidity of bar frameworks in general position: a Euclidean distance matrix approach. In: Mucherino et al. [102], pp. 3–22
2. Alfakih, A., Khandani, A., Wolkowicz, H.: Solving Euclidean distance matrix completion problems via semidefinite programming. Comput. Optim. Appl. **12**, 13–30 (1999)
3. Asimow, L., Roth, B.: The rigidity of graphs. Trans. Am. Math. Soc. **245**, 279–289 (1978)
4. Asimow, L., Roth, B.: The rigidity of graphs II. J. Math. Anal. Appl. **68**, 171–190 (1979)
5. Bahr, A., Leonard, J., Fallon, M.: Cooperative localization for autonomous underwater vehicles. Int. J. Robot. Res. **28**(6), 714–728 (2009)
6. Bajaj, C.: The algebraic degree of geometric optimization problems. Discret. Comput. Geom. **3**, 177–191 (1988)
7. Bandeira, A., Chen, Y., Singer, A.: Non-unique games over compact groups and orientation estimation in cryo-em. Technical report (2015). arXiv:1505.03840v1
8. Barvinok, A.: Problems of distance geometry and convex properties of quadratic maps. Discret. Comput. Geom. **13**, 189–202 (1995)
9. Beeker, N., Gaubert, S., Glusa, C., Liberti, L.: Is the distance geometry problem in NP? In: Mucherino et al. [102], pp. 85–94
10. Benedetti, R., Risler, J.-J.: Real Algebraic And Semi-algebraic Sets. Hermann, Paris (1990)
11. Berman, H., Westbrook, J., Feng, Z., Gilliland, G., Bhat, T., Weissig, H., Shindyalov, I.N., Bourne, P.: The protein data bank. Nucl. Acid Res. **28**, 235–242 (2000)
12. Berthold, T., Gamrath, G., Gleixner, A., Heinz, S., Koch, T., Shinano, Y.: Solving mixed integer linear and nonlinear problems using the SCIP Optimization Suite. ZIB (2012). http://scip.zib.de/
13. Billinge, S., Duxbury, P., Gonçalves, D., Lavor, C., Mucherino, A.: Assigned and unassigned distance geometry: applications to biological molecules and nanostructures. *4OR* **14**, 337–376 (2016)
14. Biswas, P., Toh, K.-C., Ye, Y.: A distributed SDP approach for large-scale noisy anchor-free graph realization with applications to molecular conformation. SIAM J. Scient. Comput. **30**(3), 1251–1277 (2008)
15. Biswas, P., Ye, Y.: Semidefinite programming for ad hoc wireless sensor network localization. In: Proceedings of the 3rd International Symposium on Information Processing in Sensor Networks (IPSN04), pp. 46–54. ACM, New York, NY, USA (2004)
16. Björner, A., Las Vergnas, M., Sturmfels, B., White, N., Ziegler, G.: Oriented Matroids. Cambridge University Press, Cambridge (1993)
17. Blumenthal, L.: Theory And Applications Of Distance Geometry. Oxford University Press, Oxford (1953)
18. Blumenthal, L.: A Modern View Of Geometry. Freeman & C, San Francisco (1961)
19. Borcea, C., Streinu, I.: On the number of embeddings of minimally rigid graphs. Discret. Comput. Geom. **31**(2), 287–303 (2004)
20. Bowers, J., Bowers, P.: A Menger redux: embedding metric spaces isometrically. Am. Math. Mon. (accepted)
21. Carvalho, R., Lavor, C., Protti, F.: Extending the geometric build-up algorithm for the molecular distance geometry problem. Inf. Process. Lett. **108**, 234–237 (2008)
22. Cassioli, A., Günlük, O., Lavor, C., Liberti, L.: Complexity of discretization vertex orders for distance geometry. Technical report, LIX, Ecole Polytechnique (2013)
23. Cassioli, A., Günlük, O., Lavor, C., Liberti, L.: Discretization vertex orders for distance geometry. Discret. Appl. Math. **197**, 27–41 (2015)
24. Cauchy, A.-L.: Sur les polygones et les polyèdres. J. de l'École Polytech. **16**(9), 87–99 (1813)
25. Cayley, A.: A theorem in the geometry of position. Camb. Math. J. **II**, 267–271 (1841)

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4

26. Cobham, A.: The intrinsic computational difficulty of functions. In: Bar-Hillel, Y. (ed.) Logic, Methodology and Philosophy of Science, pp. 24–30. North-Holland, Amsterdam (1965)
27. Connelly, R.: A counterexample to the rigidity conjecture for polyhedra. Pub. Mathématiques de l'IHES **47**, 333–338 (1978)
28. Connelly, R.: On generic global rigidity, applied geometry and discrete mathematics. In: DIMACS Series in Discrete Mathematics and Theoretical Computer Science, vol. 4. American Mathematical Society, Providence (1991)
29. Costa, V., Mucherino, A., Lavor, C., Cassioli, A., Carvalho, L., Maculan, N.: Discretization orders for protein side chains. J. Glob. Optim. **60**, 333–349 (2014)
30. Cox, T., Cox, M.: Multidimensional Scaling. Chapman & Hall, Boca Raton (2001)
31. Crippen, G., Havel, T.: Distance Geometry and Molecular Conformation. Wiley, New York (1988)
32. Dasgupta, S., Gupta, A.: An elementary proof of a theorem by Johnson and Lindenstrauss. Random Structures and Algorithms **22**, 60–65 (2002)
33. Dattorro, J.: Convex Optimization and Euclidean Distance Geometry. $\mathcal{M}\epsilon\beta oo$, Palo Alto (2015)
34. Davis, R., Ernst, C., Wu, D.: Protein structure determination via an efficient geometric build-up algorithm. BMC Struct. Biol. **10**(Suppl 1), S7 (2010)
35. Demmel, J.: Applied Numerical Linear Algebra. SIAM, Philadelphia (1997)
36. Descartes, R.: Discours de la Méthode. Ian Maire, Leiden (1637)
37. Dias, G., Liberti, L.: Diagonally dominant programming in distance geometry. In: Cerulli, R., Fujishige, S., Mahjoub, R. (eds.) International Symposium In Combinatorial Optimization. LNCS, vol. 9849, pp. 225–236. Springer, New York (2016)
38. Ding, Y., Krislock, N., Qian, J., Wolkowicz, H.: Sensor network localization, Euclidean distance matrix completions, and graph realization. Optim. Eng. **11**, 45–66 (2010)
39. Dokmanić, I., Parhizkar, R., Ranieri, J., Vetterli, M.: Euclidean distance matrices: Essential theory, algorithms and applications. IEEE Signal Process. Mag. **1053–5888**, 12–30 (2015)
40. Donald, B.: Algorithms in Structural Molecular Biology. MIT Press, Boston (2011)
41. Dong, Q., Wu, Z.: A linear-time algorithm for solving the molecular distance geometry problem with exact inter-atomic distances. J. Glob. Optim. **22**, 365–375 (2002)
42. Dong, Q., Wu, Z.: A geometric build-up algorithm for solving the molecular distance geometry problem with sparse distance data. J. Glob. Optim. **26**, 321–333 (2003)
43. Duxbury, P., Granlund, L., Juhas, P., Billinge, S.: The unassigned distance geometry problem. Discret. Appl. Math. **204**, 117–132 (2016)
44. Edmonds, J.: Paths, trees and flowers. Can. J. Math. **17**, 449–467 (1965)
45. Emiris, I., Tsigaridas, E., Varvitsiotis, A.: Mixed volume and distance geometry techniques for counting Euclidean embeddings of rigid graphs. In: Mucherino et al. [102], pp. 23–46
46. Eren, T., Goldenberg, D., Whiteley, W., Yang, Y., Morse, A., Anderson, B., Belhumeur, P.: Rigidity, computation, and randomization in network localization. IEEE, 2673–2684 (2004)
47. Euclid. Elements. Alexandria, ∼300BC
48. Euler, L.: Continuatio fragmentorum ex adversariis mathematicis depromptorum: II Geometria, 97. In: Fuss, P., Fuss, N. (eds.) Opera Postuma Mathematica Et Physica Anno 1844 Detecta, vol. I, pp. 494–496. Eggers & C, Petropolis (1862)
49. Floyd, R.W.: Algorithm 97: shortest path. Commun. ACM **5**(6), 345 (1962)
50. Franzen, T.: Gödel's Theorem: An Incomplete Guide to its Use and Abuse. Peters, Wellesley (2005)
51. Garey, M., Johnson, D.: Computers and Intractability: A Guide to the Theory of Np-completeness. Freeman and Company, New York (1979)
52. Gluck, H.: Almost all simply connected closed surfaces are rigid. In: Dold, A., Eckmann, B. (eds.) Geometric Topology. Lecture Notes in Mathematics, vol. 438, pp. 225–239. Springer, Berlin (1975)
53. Gödel, K.: On the isometric embeddability of quadruples of points of r_3 in the surface of a sphere. In: Feferman, S., Dawson, J., Kleene, S., Moore, G., Solovay, R., van Heijenoort, J. (eds.) Kurt Gödel: Collected Works, vol. I, pp. 276–279. Oxford University Press, Oxford (1986). 1933b
54. Gonçalves, D., Mucherino, A.: Discretization orders and efficient computation of cartesian coordinates for distance geometry. Optim. Lett. **8**, 2111–2125 (2014)
55. Gortler, S., Healy, A., Thurston, D.: Characterizing generic global rigidity. Am. J. Math. **132**(4), 897–939 (2010)
56. Graver, J.: Rigidity matroids. SIAM J. Discret. Math. **4**, 355–368 (1991)
57. Graver, J., Servatius, B., Servatius, H.: Combinatorial Rigidity. American Mathematical Society (1993)
58. Henneberg, L.: Die Graphische Statik Der Starren Systeme. Teubner, Leipzig (1911)
59. Heron. Metrica, vol. I. Alexandria, ∼100AD
60. IBM. ILOG CPLEX 12.2 User's Manual. IBM (2010)
61. Jackson, B., Jordán, T.: Connected rigidity matroids and unique realization of graphs. J. Comb. Theory Ser. B **94**, 1–29 (2005)

62. Johnson, W., Lindenstrauss, J.: Extensions of Lipschitz mappings into a Hilbert space. In: Hedlund, G. (ed.) Conference in Modern Analysis and Probability. Contemporary Mathematics, vol. 26, pp. 189–206. American Mathematical Society, Providence (1984)
63. Jolliffe, I.: Principal Component Analysis, 2nd edn. Springer, Berlin (2010)
64. Kolmogorov, A., Fomin, S.: Measure. Lebesgue Integrals and Hilbert Space. Academic Press, New York (1960)
65. Kolmogorov, A., Fomin, S.: Introductory Real Analysis. Dover, New York (1975)
66. Krislock, N., Wolkowicz, H.: Explicit sensor network localization using semidefinite representations and facial reductions. SIAM J. Optim. **20**, 2679–2708 (2010)
67. Kunen, K.: Set Theory. An Introduction to Independence Proofs. North Holland, Amsterdam (1980)
68. Laman, G.: On graphs and rigidity of plane skeletal structures. J. Eng. Math. **4**(4), 331–340 (1970)
69. Langville, A., Meyer, C.: Google's Pagerank and Beyond: The Science of Search Engine Rankings. Princeton University Press, Princeton, NJ (2006)
70. Laurent, M.: Cuts, matrix completions and graph rigidity. Math. Program. **79**, 255–283 (1997)
71. Laurent, M.: A connection between positive semidefinite and Euclidean distance matrix completion problems. Linear Algebr. Appl. **273**, 9–22 (1998)
72. Laurent, M.: Polynomial instances of the positive semidefinite and Euclidean distance matrix completion problems. SIAM J. Matrix Anal. Appl. **22**(3), 874–894 (2000)
73. Laurent, M.: Matrix completion problems. In: Floudas, C., Pardalos, P. (eds.) Encyclopedia of Optimization, 2nd edn, pp. 1967–1975. Springer, New York (2009)
74. Lavor, C.: On generating instances for the molecular distance geometry problem. In: Liberti, L., Maculan, N. (eds.) Global Optimization: From Theory To Implementation, pp. 405–414. Springer, Berlin (2006)
75. Lavor, C., Firer, M., Martinez, J.-M., Liberti, L.: Preface. Int. Trans. Oper. Res. **23**(5), 841 (2016)
76. Lavor, C., Lee, J., John, L.-S.A., Liberti, L., Mucherino, A., Sviridenko, M.: Discretization orders for distance geometry problems. Optim. Lett. **6**, 783–796 (2012)
77. Lavor, C., Liberti, L., Lodwick, W., Mendonça da Costa, T.: An Introduction to Distance Geometry Applied to Molecular Geometry. SpringerBriefs. Springer, New York (2017)
78. Lavor, C., Liberti, L., Maculan, N.: Computational experience with the molecular distance geometry problem. In: Pintér, J. (ed.) Global Optimization: Scientific and Engineering Case Studies, pp. 213–225. Springer, Berlin (2006)
79. Lavor, C., Liberti, L., Maculan, N., Mucherino, A.: The discretizable molecular distance geometry problem. Comput. Optim. Appl. **52**, 115–146 (2012)
80. Lavor, C., Liberti, L., Maculan, N., Mucherino, A.: Recent advances on the discretizable molecular distance geometry problem. Eur. J. Oper. Res. **219**, 698–706 (2012)
81. Liberti, L., Lavor, C.: Six mathematical gems in the history of distance geometry. Int. Trans. Oper. Res. **23**, 897–920 (2016)
82. Liberti, L., Lavor, C., Alencar, J., Abud, G.: Counting the number of solutions of ^{k}DMDGP instances. In: Nielsen and Barbaresco [103], pp. 224–230
83. Liberti, L., Lavor, C., Maculan, N.: A branch-and-prune algorithm for the molecular distance geometry problem. Int. Trans. Oper. Res. **15**, 1–17 (2008)
84. Liberti, L., Lavor, C., Maculan, N., Mucherino, A.: Euclidean distance geometry and applications. SIAM Rev. **56**(1), 3–69 (2014)
85. Liberti, L., Lavor, C., Mucherino, A.: The discretizable molecular distance geometry problem seems easier on proteins. In: Mucherino et al. [102], pp. 47–60
86. Liberti, L., Lavor, C., Mucherino, A., Maculan, N.: Molecular distance geometry methods: from continuous to discrete. Int. Trans. Oper. Res. **18**, 33–51 (2010)
87. Liberti, L., Masson, B., Lavor, C., Lee, J., Mucherino, A.: On the number of realizations of certain Henneberg graphs arising in protein conformation. Discret. Appl. Math. **165**, 213–232 (2014)
88. Lovász, L., Yemini, Y.: On generic rigidity in the plane. SIAM J. Algebr. Discret. Methods **3**(1), 91–98 (1982)
89. Makhorin, A.: GNU Linear Programming Kit. Free Software Foundation (2003). http://www.gnu.org/software/glpk/
90. Man-Cho So, A., Ye, Y.: Theory of semidefinite programming for sensor network localization. Math. Program. B **109**, 367–384 (2007)
91. Maxwell, J.: On reciprocal figures and diagrams of forces. Philos. Mag. **27**(182), 250–261 (1864)
92. Maxwell, J.: On the calculation of the equilibrium and stiffness of frames. Philos. Mag. **27**(182), 294–299 (1864)
93. Menger, K.: Untersuchungen über allgemeine Metrik. Mathematische Annalen **100**, 75–163 (1928)
94. Menger, K.: New foundation of Euclidean geometry. Am. J. Math. **53**(4), 721–745 (1931)
95. Menger, K. (ed.): Ergebnisse Eines Mathematischen Kolloquiums. Springer, Wien (1998)
96. Moré, J., Wu, Z.: Global continuation for distance geometry problems. SIAM J. Optim. **7**(3), 814–846 (1997)
97. Mucherino, A.: On the identification of discretization orders for distance geometry with intervals. In: Nielsen and Barbaresco [103], pp. 231–238
98. Mucherino, A., de Freitas, R., Lavor, C.: Preface. Discret. Appl. Math. **197**, 1–2 (2015)

99. Mucherino, A., Lavor, C., Liberti, L.: A symmetry-driven BP algorithm for the discretizable molecular distance geometry problem. In: Proceedings of Computational Structural Bioinformatics Workshop, pp. 390–395. IEEE, Piscataway (2011)
100. Mucherino, A., Lavor, C., Liberti, L.: The discretizable distance geometry problem. Optim. Lett. **6**, 1671–1686 (2012)
101. Mucherino, A., Lavor, C., Liberti, L.: Exploiting symmetry properties of the discretizable molecular distance geometry problem. J. Bioinform. Comput. Biol. **10**(1–15), 1242009 (2012)
102. Mucherino, A., Lavor, C., Liberti, L., Maculan, N. (eds.): Distance Geometry: Theory, Methods, and Applications. Springer, New York (2013)
103. Nielsen, F., Barbaresco, F. (eds.): Geometric Science of Information, vol. 8085. LNCS. Springer, New York (2013)
104. Papadimitriou, C.: Computational Complexity. Addison-Wesley, Reading, MA (1994)
105. Recski, A.: A network theory approach to the rigidity of skeletal structures. Part 2. Laman's theorem and topological formulae. Discret. Appl. Math. **8**, 63–68 (1984)
106. Rojas, N., Thomas, F.: Application of distance geometry to tracing coupler curves of pin-jointed linkages. J. Mech. Robotics **5**(2), 021001 (2013)
107. Saxe. J.: Embeddability of weighted graphs in k-space is strongly **NP**-hard. In: Proceedings of 17th Allerton Conference in Communications, Control and Computing, pp. 480–489 (1979)
108. Schlick, T.: Molecular Modelling and Simulation: An Interdisciplinary Guide. Springer, New York (2002)
109. Schoenberg, I.: Remarks to Maurice Fréchet's article "Sur la définition axiomatique d'une classe d'espaces distanciés vectoriellement applicable sur l'espace de Hilbert". Ann. Math. **36**(3), 724–732 (1935)
110. Singer, A.: Angular synchronization by eigenvectors and semidefinite programming. Appl. Comput. Harmon. Anal. **30**, 20–36 (2011)
111. Singer, A., Zhao, Z., Shkolnisky, Y., Hadani, R.: Viewing angle classification of cryo-electron microscopy images using eigenvectors. SIAM J. Imaging Sci. **4**(2), 543–572 (2011)
112. Sit, A., Wu, Z., Yuan, Y.: A geometric build-up algorithm for the solution of the distance geometry problem using least-squares approximation. Bull. Math. Biol. **71**, 1914–1933 (2009)
113. Sitharam, M., Zhou,Y.: A tractable, approximate, combinatorial 3D rigidity characterization. In: Fifth Workshop on Automated Deduction in Geometry (2004)
114. Spivak, M.: Calculus On Manifolds. Addison-Wesley, Reading, MA (1965)
115. Stoll, R.: Set Theory and Logic. Dover, New York (1979)
116. Tay, T.-S., Whiteley, W.: Generating isostatic frameworks. Struct. Topol. **11**, 21–69 (1985)
117. Tenenbaum, J., de Silva, V., Langford, J.: A global geometric framework for nonlinear dimensionality reduction. Science **290**, 2319–2322 (2000)
118. Thorpe, M., Duxbury, P. (eds.): Rigidity Theory and Applications. Fundamental Materials Research. Springer, New York (2002)
119. Venkatasubramanian, S., Wang, Q.: The Johnson-Lindenstrauss transform: an empirical study. In: Algorithm Engineering and Experiments, vol. 13, pp. 164–173. ALENEX. SIAM, Providence (2011)
120. Wu, D., Wu, Z.: An updated geometric build-up algorithm for solving the molecular distance geometry problem with sparse distance data. J. Glob. Optim. **37**, 661–673 (2007)
121. Wüthrich, K.: Protein structure determination in solution by nuclear magnetic resonance spectroscopy. Science **243**, 45–50 (1989)
122. Wüthrich, K., Billeter, M., Braun, W.: Pseudo-structures for the 20 common amino acids for use in studies of protein conformations by measurements of intramolecular proton-proton distance constraints with nuclear magnetic resonance. J. Mol. Biol. **169**, 949–961 (1983)
123. Xu, H., Izrailev, S., Agrafiotis, D.: Conformational sampling by self-organization. J. Chem. Inf. Comput. Sci. **43**, 1186–1191 (2003)

Index

Symbols
$=$, 98
$E(G)$, 109
$E[U]$, 109
$G[U]$, 109
$N(\cdot)$, 109
$V(G)$, 109
$\cap$, 97
$\cup$, 97
$\delta(\cdot)$, 109
$\exists$, 97
$\forall$, 97
$\in$, 97
$\wedge$, 97
$\leftarrow$, 97
$\neg$, 97
$\neq$, 98
$\not\in$, 98
$\subseteq$, 98
$\subsetneq$, 98
$\supseteq$, 98
$\supsetneq$, 98
$\rightarrow$, 97
$\vee$, 97

A
Abelian, 107
Abstract rigidity, 76
Abstract rigidity matroid, 78
Accuracy, 88
Adjacency
  structure, 81
Adjacency matrix
  weighted, 82
Adjacent, 33, 109
  edge, 109
  predecessor, 43
Affine, 30, 76
Affine dependence, 100
Affine hull, 73, 101
Affinely independent, 100
Affine space, 23, 100
  dimension, 100
Algebraically
  dependent, 75
Algebraic numbers, 115
Algorithm
  BP, 44
  class, 30
  clustering, 89
  exponential, 63
  graphical, 3
  polytime, 35, 42, 52, 112
  recursive, 37, 38
  termination, 111
Alignment, 41
Almost all, 74, 75
Ambient space, 86
Amplitude, 7
Anchor, 8, 36
Angle, 9, 29, 37, 70, 72
  incidence, xi
  right, xii
Approach
  brute-force, 60
Approximability, 95
Approximate
  distance matrix, 85
  realization, 86
Approximation, 83, 89
  quality, 88
  simplest, 82
Arbitrarily close, 68, 69
Arc, 109
Architecture, 2
Area, 31
Array, 101
  rectangular, 12
Assignment, 89
Associativity, 98, 99, 106
Asymptotically smaller, 111
Atom, xii, 4
  distance, 43
  pair
    consecutive, 37
Autonomous
  vehicle, 7

© Springer International Publishing Switzerland 2017

L. Liberti and C. Lavor, *Euclidean Distance Geometry*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-60792-4

Axiom, 98, 99
  closure, 77
  metric
    first, 11
Axiomatization, 1
Axis
  horizontal, 74
  vertical, 72

B
Backbone, xii, 37
  isomeric, xii
  protein, 37
Bar, 4, 7, 70
  rigid, 67
Bar-and-joint, 2, 67
Barycenter, 83
Basis, 77, 78, 100
  cardinality, 77
  standard, 100, 115
Battery, 4
Battery consumption, 4
Biclique, 62, 110
Big-oh, 111
Bijection, 52, 100
  vertex-rank, 34
Biology
  structural, 93
Bipartite, 110
Bit, 111
Bitmap data, 88
Blumenthal, xii
Blumenthal, L., 1
Bond, 4
  angle, 4
  covalent, 4
Bound
  worst-case, 111
Branch, 44
  infeasibility, 44
Branch-and-Bound, 63
Branch-and-Prune (BP), 39, 44, 49, 51–55, 95
Branching, 39, 53
  no, 48, 51

C
Call
  polynomial number of, 14, 57
  recursive, 41, 44
  trace, 41
Cardinality, 60, 77, 98, 100
  infinite, 13
  minimum, 47, 107
Cauchy, xiii
Cauchy, A.-L., 3, 93
Cayley, xiii
Cayley, A., 1, 93
Cayley-Menger determinant, 32, 35
CDGP, 42
Cell, xi, 37
  surface, 37
Cellphone, 8
  pair, 8
Center, 9
Centroid, 89
  closest, 89
Certificate, 113
  YES, 63
Chain
  clique, 47
  triangle, 45, 54
Challenge
  open, 79, 82
Chemical
  bond, 4
Chemical reaction, 37
Chirality, xii
Choice
  deterministic, 27
  random, 42
Circle, 72
  unit, 108
Class
  complexity, 14, 58
Clique, 19, 20, 27, 29–32, 49, 57, 60, 61, 67, 73, 74, 76, 79, 81, 110, 113
  chain, 47
  feasible, 30
  initial, 21, 27, 35, 36, 38, 40, 44, 52, 60, 67
  realization
    isometry, 68
  subgraph, 78
  weighted, 44, 83
Clock, xi, 3
  atomic, 6
  synchronization, xi, 94
Clock synchronization, 3
Closed form, 55
Closure, 77, 78, 98, 99, 106
Cluster, 2
  representative, 89
Clustering, 88
Color, 88
  depth, 89
Column, 12, 105
  basic, 23, 27
  linearly dependent, 23, 27
  linearly independent, 23, 27
  nonbasic, 23, 27
  orthogonal, 85
  zero, 24
Column vector, 101
Combinatorial condition, 75
Communication
  cellular, 7
Commutativity, 98, 99, 107
Complement, 113
Complete, 114

Completion, 67, 69, 72, 78, 81, 82
  approximate, 82
  realization, 78
Complexity, 22, 41, 95
  time, 111
  worst-case, 60, 111
Component, 100
Composition, 47, 68
Configuration, 10
  incongruent, 95
Congruence, 46, 68, 69, 71–73, 76, 79
Congruent, 26
Conjecture
  Euler, 3, 93
Connected, 59, 109
  component, 11
  graph, 11
Connectivity, 11
Connelly, B., 3, 93
Consecutive
  pair, 37
  triangle, 37
  triplet, 37
Constant, 7, 33, 53, 69
  fixed, 61
Constraint, 63
Contiguous, 57, 61
  order, 43
Contradiction, 10, 15, 35, 41, 59, 61, 62, 100
Coordinate, 27, 106
Cost
  computational, 89
Couple, 108
CPLEX, 63
CPU, 62
CPU time, 89
Crippen, xii
Criterion
  combinatorial, 79
Cryo-EM, 94
CSYP, 5, 6
CTOP, 61, 62
  instance, 63
Curve
  spiral-like, 5
Cut, 109
Cutset, 109
  nontrivial, 109
Cycle, 15, 115
  Hamiltonian, 110, 113
  simple, 14, 18, 110

D
Data
  high-dimensional, 2
  inexact, 94
  noisy, 94
  packet, 4
  visualization, 1
Database, 5
Data visualization, 86
DDGP, 37, 38, 40, 43, 44, 67
  graph, 40, 57
  instance, 38
Decision variable, 63
Decoding, 94
Degree, 16, 109
  2, 13
  highest, 61
Degree of freedom, 70–72
Dense, 74
Dependence
  affine, 100
Depth-first search, 39
Derivative, 69
Descartes, 1
Determinant, 104
  Cayley-Menger, 1, 32, 33, 35
Deterministic, 27, 39, 41
DGP, 9, 11, 12, 14, 57, 82, 95
  certificate, 12
  instance, 12, 14, 15, 18, 35, 37, 43
    YES, 74
  solution, 12
  trilaterative, 35, 37
DGP instance
  feasible, 27
Diagonal, 52, 81, 102
Diagonal entry, 85
Diagonal matrix, 105
Difference, 16
  absolute, 8
Digraph, 108
Dimension, xii, 11, 27, 74, 76, 82, 100
  affine hull, 41
  approprite, 79
  huge number of, 88
  low, 89
Dimensionality, 86
  decrease, 6
  inherent, 86
Directed, 108
Direction, 15
Discrepancy, 82
  scaled, 87
Discretization
  edge, 40
  group, 48
Distance, 6
  constraint, 7
  data, 27
  discretization, 50
  distortion, 82
  Euclidean, 2, 12, 88, 106
  feasibility, 39
  feasible, 37, 53
  inter-atomic, xi, 1
  missing, 33, 82
  non-negative, 12

pairwise, xi, 9, 86, 93, 95
all, 11, 69
path length, 59
preserved, 6
set, 5, 68
shortest, 59
to target, 7
value, 11, 20, 49
zero, 34
Distance Geometry(DG), 1, 43
Euclidean, xiii
Distance matrix, 81, 86
approximate, 85
Euclidean, 2
partial, 81
Distance subset, 11
Distance threshold, 4
Distortion, 82
Distribution
Gaussian, 88
normal, 2
Distributivity, 98, 99
DMDGP, 43
Drone
swarm, 94
Drug, 4
DVOP, 57

E
Echo, 7, 94
Edge, xii, 11, 20, 29, 35, 65, 72, 76, 109
disconnected, 76
discretization, 40, 49
incident, 62
length, 13
missing, 32
parallel, 11, 109
pruning, 40, 49, 52, 53, 55, 57
periodic, 54
set, 110
sharing, 54
spanning subset, 78
weight, 11, 34, 110
random, 36
unit, 18, 29, 30, 42, 54
weight function, 13, 14, 21, 26, 27, 44, 47, 49, 57, 67, 70, 73–75, 78, 110
EDM, 81, 85, 90, 95
approximate, 87
squared, 83
EDMCP, 81, 82, 86
Efficient, 67
Ege
pruning, 50
Egypt, 1
Eigenvalue, 85, 105
distinct, 105
nonnegative, 85
nonzero, 105
positive, 86, 87
real, 105
zero, 105
Eigenvector, 85, 87, 105
orthogonal, 105
Embedding
proximity, 87
Embedding space, 19
Encoding, 94
Equation, 13
difference, 24
quadratic, 20, 33, 41
system, 16
Equivalence
class, 107
relation, 107
Error, 16
approximation, 18
large, 18
relative, 17
tolerance, 2
zero, 16
Error-prone, 94
Error-tolerant, 83
Euclid, 1
Euclidean distance, 106
Euclidean space, 11, 106
Euler, xiii
Euler, L., 3, 93
Execution time, 111
Execution trace, 113
Exponential
complexity, 41
Expression
closed-form, 23

F
Face
triangular, 54
Feasibility problem, 63
Field, 98, 99, 101
0, 98
1, 98
electromagnetic, 7
FindTOFromClique, 60
FindTrilaterationOrder, 61
Finitely many, 13
Fixed, 61
Fixed point
argument, 2
Flag
infeasibility, 61
Fleet, 7
Flexibility, 69, 74, 75, 78
infinitesimal, 74
Flexible, 2, 67, 69, 73, 74
almost all, 74
infinitesimally, 72–74
Flexible framework, 68

Floyd–Warshall, 83
Force diagram, 3
Formula
  Heron, 1
Formulation, 16
  ILP, 63
FPT, 52–55, 61
Fréchet, M., 2
Fraction, 16
Framework, 67, 69, 70, 73, 75
  bar-and-joint, 67
  flexible, 68
  generic, 75
  generically rigid, 75
  rigid, 68, 69, 78
Frequency, 7, 8
  difference, 8
Function
  exponential, 61
  objective, 63, 112
  of time, 69

G
Gödel, xiii
Gödel, K., 2
Gaussian
  distribution, 88
Gaussian projection, 90
Gaussian random projection, 88
General
  realization, 76
Generality, 76
General position, 76
Generator, 48, 50, 52, 107
Generic, 75, 78
  Graver, 75, 76
Generically rigid, 75
Genericity, 75, 76
Geodesic, 2
Geometric build-up, 95
Geometry
  Euclidean, 1
Givens
  matrix, 103
Global optimization, 16
  methods, 17
Global optimum, 16
Gödel, xiii
Google, 88
  Images, 88
  ranking, 88
GPS, 7
Gram matrix, 83
Gramian, 83
Graph, 11, 108
  class, 65
  closest, 16
  complement, 110, 113
  complete, 13, 19, 29, 31, 33, 34, 40–42, 52, 78, 83, 110
    weighted, 21, 22
  connected, 58, 109
  cycle, 14
  DDGP, 40
  directed, 108
  disconnected, 11
  drawing, 17
  edge, 69
  empty, 72, 110
  flexible, 75, 78
  induced, 19
  input, 39, 57
  K-laterative, 37, 41
  ^{K}DMDGP, 43–45, 47, 49, 51, 79
    minimal, 49
  large, 81
  minimally trilaterative, 41
  non-simple, 109
  property, 75
  protein, 49
  protein backbone
    artificial, 37
  random, 17
  realizable, 29
  rigid, 75, 78, 79, 95
  rigidity, xiii
  simple, 11, 109
  simple undirected, 57, 67
  spiral
    random, 62
  stable, 110
  triangle, 13, 18
  trilaterative, 34, 40, 79
    minimal, 34
    minimally, 41
  undirected, 62, 108
  unweighted, 110
  weighted, 11, 16, 45, 81, 110
Graph rigidity, 76
Group, 106
  Abelian, 47, 107
  action, 46, 107
    transitive, 108
  axiom, 106
  cyclic, 47, 108
  discretization, 48, 49
  finite
    nontrivial, 108
  generated, 107
  generator, 107
  homomorphism, 108
  isomorphism, 108
  pruning, 50
  table, 107, 108
  transitive, 46, 108
Growth
  exponential, 52
Grpah, 95

H
Half-space, 48
  intersection, 3
Hamiltonian, 62, 110
HAMILTONIAN PATH, 62
Hard, 14, 58, 114
  subproblem, 43
Hardness, 57, 62
Heron, 1, 93
Heron's formula, 31
Heuristic, 2, 89
HP, 62
Hull
  affine, 73, 101
Hyperplane, 28, 47, 48, 74, 75, 102, 104

I
Idempotency, 52
Idempotent, 47, 107, 108
Identity, 47, 106
Ill-defined, 9, 10
ILP, 63
  formulation, 63
Image, 2, 88, 102, 105
  ranked, 88
  RGB, 2
  size, 88
Immediate
  predecessor, 43
Incident, 109
Independence
  algebraic, 75
Independent, 77
  algebraically, 75
Independent set, 110
Induced, 109
Induction, 14, 44, 48, 59, 60
  argument, 59
  hypothesis, 48, 59
  start, 59
Inequality
  triangle, 82
Infeasibility
  detection, 44
Infeasible, 21
Infinite
  uncountably, 13
Infinitesimal, 78
Infinitesimally flexible, 72
Infinitesimally rigid, 72
  not, 75
Information, 94
Initial
  clique, 63
Initial clique, 21
Injective, 52
Input, 5, 86
Instance, 63, 112, 113
  graph
    YES, 67
  NO, 15, 82
  PARTITION, 15
  TDGP, 36
  YES, 14, 15, 44, 57, 58, 113
Integer, 16, 53, 57, 59, 98
  nonnegative, 98
  smallest, 89
Intensity, 94
Intersection
  line with sphere, 24
  no, 25
Invariance, 99
Invariant, 48, 49
Inverse, 21, 98, 99, 106
  unique, 106
Inverse problem, 9, 94
Isomap, 90
Isometry, 46, 67, 68, 76, 79
  local, 68–70, 73, 79
Iteration, 27
Iterative construction, 19

J
JL, 88, 89
Johnson, B., 2, 93
Johnson–Lindenstrauss lemma, 88
Joint, 4, 7, 70
  movable, 67

K
K-CLIQUE, 57, 58
K-dimensional, 100
^{K}DMDGP
  graph, 43, 44, 55, 61, 67
Kernel, 71, 74, 102, 105
Kernel and image
  theorem, 102
K-lateration, 22
k-means, 2, 89
K-volume, 31

L
Label
  negative, 48
  positive, 48
Lagrange
  little theorem, 52
Laman's theorem, 78
Lattice
  incidence, 3
Leaf, 37, 39
Learning rate, 87
Lebesgue measure, 31, 74
Lemma
  Johnson–Lindenstrauss, xiii, 88
Length, xi, 31, 106

Level, 48
BP tree, 49
Lindenstrauss, J., 2, 93
Line, 23, 27, 29, 30, 46, 100, 101, 106
orthogonal, 30
real, 19, 21
Linear algebra, 77, 81
elementary, 1
Linear combination, 100
Linear dependence, 75
Linear function, 28
Linearly dependent, 99
Linearly independent, 21, 75, 99
Linear transformation, 102
List, 109
of lists, 12
Local isometry, 68
Localization, 1
Logarithmic dependence, 88
Loop, 11, 44, 81, 109
Lossy, 94
Lowest energy state, 4

M
Machine Learning, 88
algorithm, 88
Macromolecules
spectroscopy, xiii
Manhattan, xii
Manifold, 71, 73
Material science, 93
Mathematica, 3, 4, 17, 28, 40, 45, 61, 64, 91
Mathematical Programming, 63
formulation, 63
solver, 63
Matrix, 101, 107
adjacency
weighted, 81, 83
column, 101
completion, 81
component, 101
decomposition, 27
determinant, 104
diagonal, 105
distance, 10, 17, 81, 82
approximate, 85
Euclidean, 83
partial, 81
Givens, 103
Gram, 83, 85, 87
approximate, 87
identity, 102
inverse, 21, 102
maximum rank, 75
nonsingular, 23, 27, 102
nullity, 102
of eigenvectors, 105
positive semidefinite, 2, 85
product, 102
by scalar, 101
random, 2
rank, 102
rectangular, 23, 27
rigidity, 69–71, 73, 74, 78
rank, 69
row, 74, 101
singular, 102
square, 101
structure, 21
sum, 101
symmetric, 81, 105
transpose, 101
zero, 101
Matroid, 77, 78
abstract rigidity, 78, 79
axiom, 78
basis, 77
closure, 77
independence, 77
infinitesimal rigidity, 78
matrix, 78
oriented, xii
rank, 77
Maxwell, J.C., 3, 93
MDGP, 5, 43
instance, 43
classic, 83, 87
Mean, 88
zero, 2, 88
Measurable, 31
Measure
error, 82
Lebesgue, 74
Memory, 111
Menger, K., 1, 93
Meta-data, 88
Method
approximate, 35, 82
direct, 31
discrete, 31
precise, 35
Metric, 11, 105
Euclidean, 105
Metric space, 105
Microphone, 94
Minor, 75
nontrivial, 75
trivial, 75
Missing
pair, 82
Model
precise, 43
Modulus
constant, 69
Molecule, xii, 37
Movement
continuous, 13, 67, 69
solver, 63
Multi-Dimensional Scaling, xiii, 2, 85, 86, 90

classic, 83, 87
Multiplication
right, 12
Multi set, 109
Multivariate, 13

N
Neighborhood, 109
Network, 4, 8, 36
sensor, xi
wired, 36
wireless, xi
wireless sensor, 93
Next, 21, 22, 26, 29, 33, 35, 37, 39, 41, 48
NMR, 4, 37
sectroscopy, xiii
Nobel Prize, 93
Node, 37, 109
exploration, 39
leaf, 37, 39
non-leaf, 39
Noise
reflection, 94
Noise source, 94
Nondeterministic, 35, 39, 41
Nonidentity, 108
Nonnegative, 105
Nonsingular, 21, 102
Norm, 105
Euclidean, 88, 105, 115
NP, 57, 113
NP-complete, 57, 60–62, 114
NP-completeness, 57
NP-hard, 14–16, 43, 57, 60, 62, 114
by inclusion, 43
Nullity, 74, 102
Number
complex, 98
irrational, 35
natural, 8
of solutions, 7
rational, 98
real, 98
Number of solutions, 95

O
Objective function, 63
no, 63
OEIS, 42
Off-diagonal, 81, 102
zero, 81
Oil
spillage, 7
1-clique, 62
1D, 72, 100
Operation
elementary, 111
Operator, 26
Optimality
guarantee, 16, 89
Optimization, 94
Optimization process, 63
Optimum
global, 16, 17
local, 17
Oracle, 114
Orbit, 107
representative, 108
Order, 34
K-lateration, 34, 35
^{K}DMDGP, 43
rank, 34
trilateration, 38, 57–59, 61, 62
contiguous, 43, 47, 49, 61, 62
trilaterative, 44
vertex, 30, 33, 57, 61, 95
alternative, 37
0-lateration, 62
Orientation, 48
Origin, 29, 41, 100, 101, 106, 108
Orthogonal, 30
Output, 5

P
P, 113
Pair, 6
ordered, 108
point name, 11
set, 108
unordered, 108
Pairwise distance, 4
Parameter, 63
fixing, 57
Partial, 52
Partial reflection, 55
PARTITION, 14, 15, 18, 107, 110
instance, 14
Path, 37, 39, 52, 62, 115
continuous, 69
Eulerian, 65
Hamiltonian, 62, 65, 110
leftmost, 39
length, 59
shortest, 59, 83
simple, 18, 110
P-complete, 114
Peer-to-peer, 4
Periodic, 94
Physical device, 94
Ping, 94
sonar, 7
Pixel, 88
array, 89
Plane, 23, 54, 101, 106
congruence, 72
Euclidean, 29
rotation, 70

translation, 70
Plane vector, 108
Point, xiii, 1, 2, 9, 73, 76, 95
configuration, 9
feasible, 53
perturbation, 75
quadruplet, 2
set, 67, 68
single, 21
Polyhedron
rigid, 3
Polynomial, 32, 53, 115
multivariate, 13, 16
quadratic, 33
system, 13
Polynomial case, 41
Polytime, 15, 31, 35, 41, 42, 48, 57, 60, 61, 79, 112, 113
Position, xi, 6, 7, 20
accuracy, 7
feasible, 37, 41
general, 76
geometric, 6
relative, xi, 7
spatial, xi, 94
two, 26, 39
unique, 15, 26, 41
Positive
semidefinite, 85
Power, 16
Predecessor, 33, 34
adjacent, 34, 37, 43, 50, 57, 59, 61
all, 37
contiguous, 63
set, 39
contiguous, 34
immediate, 34, 43, 44, 50
Principal component, 87
Principal Component Analysis (PCA), 87, 90
Probability 1, 75
Problem
complete, 114
decision, 112, 115
direct, 9
easy, 35, 113
feasibility, 63
fundamental, 11
hard, 115
hardest, 114
ill-defined, 9
instance, 112
inverse, 9
optimization, 112
hardness, 115
tractable, 35, 42
Product, 16, 48
Cartesian, 47, 52
linear, 102
operation, 106
scalar, 102
Programming
integer linear, 63
mathematical, 63
Projection
random
Gaussian, 6
Property
closure, 78
hinge, 78
Protein, 4, 37
backbone, 43, 49
conformation, xi
instance, 41
structure, 1, 37
Protocol, 4
ProximityAdjustment, 87
Pruning, 39, 40
edge, 40
Pruning edge, 55
PSD, 85, 95

Q

Quasi-clique, 32, 42, 67, 79
weighted, 32, 33
Query, 88

R

Radical, 16
Radio
base, 8
Radius, 24
unit, 108
Range, 37
Rank, 21, 27, 34, 43, 52, 77, 78, 102
full, 70, 102
lowest, 86
maximum, 73, 74
order
unique, 63
smaller, 75
Rank and nullity
theorem, 102
Rational, 98, 115
Real number, 105
Realizable, 29
Realization, 11, 12, 18, 35, 55, 68, 70, 72, 75, 81, 85
all, 31, 38, 40, 41, 46
approximate, 16, 82, 86, 87
collinear, 73, 74
complete graph, 23
current, 38
distinct, 13, 36, 45
feasible, 27, 37, 41, 70
finitely many, 67
general, 76, 78
improvement, 87
incongruent, 2
iterative, 19

manifold, 72
none, 29
one, 38
partial, 38
planar
partial, 49
plane, 45, 54
random, 36
regular, 70, 74, 75, 78
singular, 70, 73, 74
space, 72
triangle, 23
two, 34
unique, 19, 95
valid, 14
vector, 48
Realizeation
first, 51
RealizeClique, 27, 29, 35, 36, 39, 41
RealizeComplete, 21, 22, 52
RealizeDDGP, 38, 39, 41, 44
RealizeDDGPRecursive, 38
RealizeDMDGP, 44, 45, 48, 49
implementation, 48, 51
RealizeDMDGPRecursive, 44
RealizeDMDGPSymm, 51
RealizeTrilaterative, 35
Reals, 98
Recursion, 37
start, 38
Recursive, 19
Reduction, 113, 114
inverse, 113
nontrivial, 62
polynomial, 14, 57, 58, 113
trivial, 62
Reflection, xii, 7, 13, 19, 26, 45–49, 67, 68, 79, 102
operator, 47
partial, xii, 46–48, 55, 67, 79
Regular, 70, 73, 78
Regularity, 75
Relation
equivalence, 107
Resonance
magnetic, xi
RHP
instance, 62
Right multiplication, 12
Rigid, 2, 67, 69, 70, 73–75
almost all, 74
framework, 75
generically, 75
infinitesimally, 72, 73
not, 75
Rigid framework, 68
Rigidity, 74, 75, 77, 78, 93, 95
generic, 75
infinitesimal, 72, 74
matrix, 71
Rigidity matrix, 69
full rank, 75
rank, 77
Robotics, 93
Room
shape, 94
Root, 16, 25, 37, 39, 115
complex, 25, 27
distinct, 25
distinct real, 41
non-distinct, 36
real, 25
distinct, 36
Rotation, 7, 13, 19, 26, 40, 41, 45, 46, 67, 68, 72, 102
center, 72
clockwise, 45
counterclockwise, 45
modulo, 46, 54, 55
planar, 106
Route, 94
Routing, xi
Row, 12, 21
independent, 74
linearly dependent, 27, 75
Row vector, 101

S

Sample, 88
uniform, 28
Scalar, 99–101
product, 99
Scaling
poor, 27
Schoenberg, I., 2, 93
Scip, 64
Search tree, 39
Segment, xi, 19, 21
Semantic context, 88
Semidefinite Programming (SDP), 95
Sensor
mobile, 36
network, 36
wireless, 1
Sensor Network Localization (SNL), 4, 36
Sentence
tag, 88
Sequence, 109
alternating, 62
empty, 72
integer, 42
Set, 97, 99
bounded, 31
cardinality, 98
closed, 31
compact, 28
difference, 98
edge, 78
element, 97, 108
equality, 98
finite, 108

generator
minimal, 107
independent, 110
initial, 46
intersection, 97
large, 88
linearly independent, 78
nonconvex, 3
of points, 9
point, 68
singleton, 109
solution, 38
union, 97
Shape, xi, xii, 4
Shortest path
all, 83
Side
length, 1, 44
negative, 27
positive, 27
Side chain, xii, 37
Signal
correct, 94
loss, 8
Signal processing, 94
Simple, 109
path, 62
Simplex, 26, 31, 42, 44, 54
flat, 26, 32
Simplex inequality
strict, 43
Singleton, 109
Singular, 21, 70, 73, 102
Singularity, 21
Smartphone, 4
SNLP, 5, 7, 36
Solution, xii, 7, 9
algorithm, 113
approximate, 115
at least one, 44
closed-form, 16
countably many, 13
distinct, 7
multiple, 36
no, 13
number, 36
set, 16, 28
cardinality, 13
current, 38
uncountably many, 13
unique, 13, 36, 95
uniqueness, 94
valid, 7
Solution algorithm, 14, 57
Solution set, 38
Solver
MP, 63
Sonar, 7
Sound, 7
Space, xi, 2, 106
3D, 54
affine, 23, 100
ambient, 75, 82
Euclidean, 79
metric, 105
tangent, 71
dimension, 74
Span, 77, 78, 100, 105
Spanning, 109, 110
Sparsity, 21
Sparsity structure, 17
SPE, 87, 90
Sphere, 2, 28
flexible, 3
K-dimensional, 24
surface, 2
Spiral, 72
Spline, 2
Square
unit, 108
Square root, 44
Stable, 110, 113
Standard deviation
unit, 88
Star, 109
Statics, 2, 93
String
search, 88
Structure, xiii
bar-and-joint, 1
Subgraph, 11, 109, 113
induced, 109
K-CLIQUE, 57
rigid, 77
spanning, 109
Subgroup, 107
Submarine, xi, 7
unmanned, xi
Submatrix, 104
Subnode, 39
Subproblem, 42
hard, 43
Subset, 14, 89, 98
all, 48
dense
open, 74
discrete, 67
instances, 15
maximal independent, 77
strict, 52, 98
Subspace, 88, 100, 105
1-dimensional, 73
affine, 30, 76
lower dimensional, 88
Sum, 16
Superset, 98
strict, 98
Surface, xi, 2, 71
triangulated, 91
Symbol

parameter, 63
Symmetric matrix, 105
Symmetry
partial reflection, 95
System
linear, 21–25, 69

T
Tag, 88
human, 88
image, 88
Tangent, 24, 71, 73
set, 73
space, 71
Tangent space, 71
Taxicab, xii
TDGP, 35
Term, 16
Termination, 41, 111
Tetrahedron, 19, 23, 26
flat, 23, 26
reflected, 26
Theorem
Asimow and Roth
first, 73, 74
second, 73
Gluck, 74, 75
kernel and image, 102
Lagrange
little, 52
Laman, 78
rank and nullity, 73, 102
3D, 67, 100
Thumbnail, 2
Time, 7
computational, 21
CPU, 62
exponential, 48, 51, 57
interval, 7
lag, 94
Tomography, 94
Tool
industrial, 7
TOP, 14, 57, 58, 61
Tractable, 42
fixed parameter, 52
Transformation
polynomial, 14, 57
polytime, 15
Transitive, 48, 50, 108
Transitivity, 52
Translation, 7, 10, 13, 19, 26, 40, 41, 45, 46, 67, 68, 72, 73, 101, 102
line, 76
modulo, 46, 54, 55
Transpose, 12, 101
Tree
binary, 37
complete, 40
BP, 39, 52
branch, 39
height, 52
level, 37, 39, 52
node, 37
root, 37
search, 39
width, 52
bounded, 53
exponential, 53
Triangle, 19, 21, 23, 54
chain, 45
flat, 23
inequality, 10
side, 31
special, 23
Triangular inequalities, 44
Trilateration, 19, 22, 36
Trilaterative, 35, 37
Triplet
consecutive, 37
2D, 9, 67, 100

U
UAV, 7, 93
Underwater
vehicles, 7
Undirected, 108
Uniform sample, 28
Uniqueness, xii, 95
Unit circle, 108
Unit radius, 108
Unknown, 20, 23
Unmanned
submarine, 7
Update, 89

V
Value
nonnegative, 16
rational, 35
unique, 11
Variable
basic, 25
binary, 63, 89
decision, 63
nonbasic, 25
Variance, 2, 39
Vector, 1, 2, 89, 99
column, 12, 101, 102
component, 27, 35, 100
data, 89
difference, 21, 88
pairwise difference, 100
random, 88
row, 101
tangent, 71
Vector space, 11, 99

0, 99
Vehicle
underwater, 1
Velocity, 69
Vertex, 11, 14, 15, 74, 109
adjacent, 59, 109
first, 41
fixed, 13
incident, 62
index, 38
label, 13
order, 37, 54
position, 26
predecessor, 33
rank
minimum, 53
relabeling, 74
set
induced, 109
single, 62
subsequent, 57
subset, 60
translation, 73
unique, 63
unlisted, 59
Vertex order
alternative, 37
Vertex-rank
bijection, 34
$V(G)$, 109
Vienna Circle, 1
Volume, 31, 41, 42
complex, 44
non-negative, 44
nonzero, 26
real, 44
simplex, 35
squared, 44
zero, 2, 32

W
Wüthrich, xiii
Wüthrich, K., 93
Walk, 15
Wave
electromagnetic, 7
superposed, 7
Weight, 110
random, 29
Weighted, 110
graph, 82
Well-defined, 11
WIFI, 4
Word
tag, 88

Y
Yemini, Y., 93
YES
instance, 14

Z
Zero
mean, 88